SOCIAL NETWORKS

NPTEL Online Course | Prof. Sudarshan Iyengar, IIT Ropar

Week 4 - Lecture Transcripts

Lectures 41-53 | 13 lectures | 142 pages (source transcript pages 391-531)

INDEX

LECTURE	TOPIC	PAGE IN THIS FILE
L41	Introduction to Homophily - Should You Watch Your Company?	2
L42	Selection and Social Influence	11
L43	Interplay Between Selection and Social Influence	21
L44	Homophily - Definition and Measurement	36
L45	Foci Closure and Membership Closure	50
L46	Introduction to Fatman Evolutionary Model	54
L47	Fatman Evolutionary Model - The Base Code (Adding People)	58
L48	Fatman Evolutionary Model - The Base Code (Adding Social Foci)	73
L49	Fatman Evolutionary Model - Implementing Homophily	89
L50	Quantifying the Effect of Triadic Closure	99
L51	Fatman Evolutionary Model - Implementing Closures	103
L52	Fatman Evolutionary Model - Implementing Social Influence	119
L53	Fatman Evolutionary Model - Storing and Analyzing Longitudinal Data	131

Social Networks
Prof. S. R. S. Iyengar
Department of Computer Science
Indian Institute of Technology, Ropar

Lecture - 41
Strong and Weak Relationships (Continued) and Homophily
Introduction to Homophily - Should you watch your company?

(Refer Slide Time: 00:05)



Remember the age-old parental advice. Our parents keep telling us keep good company do not get into bad company.

(Refer Slide Time: 00:11)



Does this statement even mean anything? How does it matter who your friends are? Do not you think it is you and only you who make makes your character, how on earth will it matter if I have a spoiled brat on my as my friend. It does not matter if my morals or in place, I think I will do well in my life. Is this true? Do you think your friends really make or break your life? Do you think the age-old parental advice that I told you is really true? There is a very well-known proverb it says tell me who your friends are I will tell you who you are - how much of truth does this proverb carry.

(Refer Slide Time: 00:52)



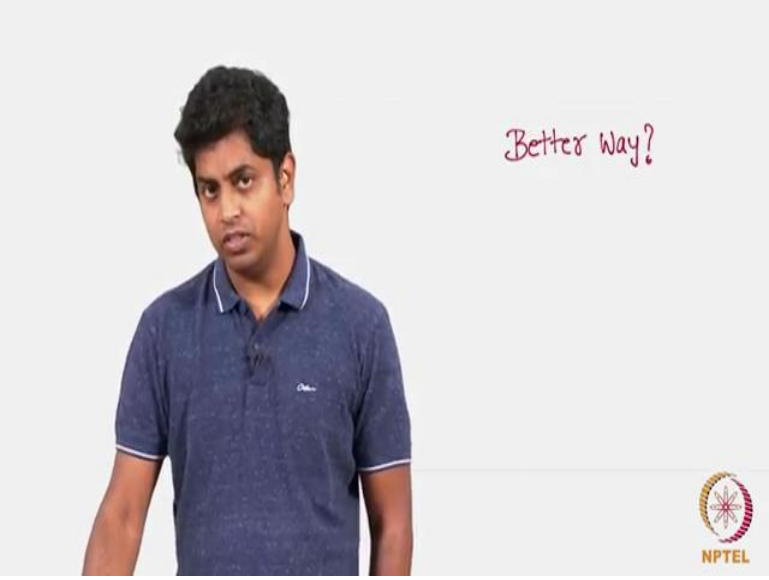
Now, how can one even answer such a question as to how this is true?

(Refer Slide Time: 01:04)



One way of doing it is to go and look at many people and ask who their friends are, look at their traits. For example, I will go and ask this person who are your friends, what are they doing, what is their social status or what is your social status and try to see whether friendships and the character of the person is sort of related, I can go on doing this right.

(Refer Slide Time: 01:30)



(Refer Slide Time: 01:35)



Is there a better way to do this? Yeah there is there is there is a much better way to do this as a very fascinating study that I am going to tell you people.

(Refer Slide Time: 01:40)



There is a paper that was published in 2007 by Christakis and Fowler which said obesity is contagious what do I mean by this.

(Refer Slide Time: 01:48)



If I am friends with people who are obese, I enter the danger of becoming obese. How do you think this is true? Probably it some fictitious statement that that one can make just like that, can we back it up with some evidence.

(Refer Slide Time: 02:13)

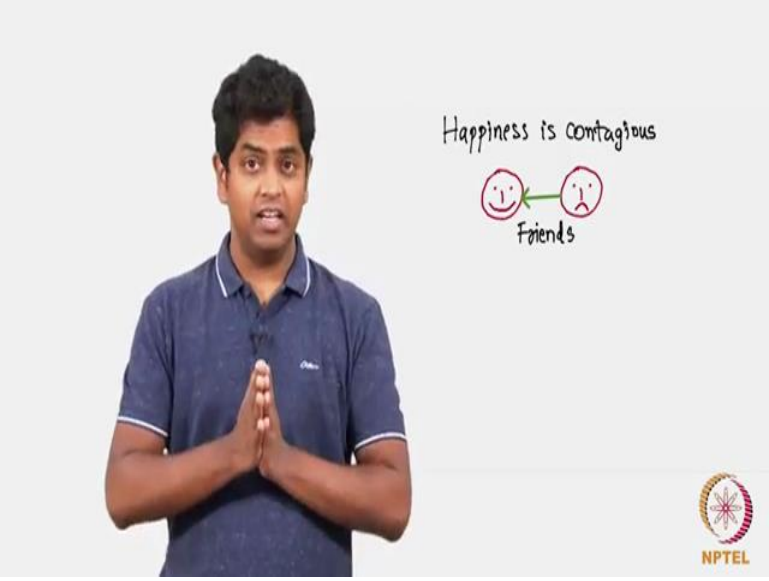


Scientists also observed that it is not just obesity that is contagious, there are several other things that are contagious too on our friendship network and it is the inferences beyond something being contagious. This eye-opening piece of research which says happiness is contagious, takes this to a brand-new level what is it mean? It goes without

saying that if your amidst high energy happy people, you also tend to be happy. You see this is especially true of dogs if you have a pet you will realize that when you are happy and energetic it is very contagious the dog also becomes incredibly happy and energetic and vice versa sometimes.

If you meet an old friends who is sort of very who has a smiling face and who is always positive optimistic you miss being with him or her right I mean it is in all I am trying to say that this happiness and energy levels are very contagious. Now what if we repeat this experiment of the obesity being a contagion, with obesity replace by happiness, do you think happiness also is contagious? In the previous question we saw how obesity was contagious, but now if we ask the same question if I am happy is it because of someone who is happy around me, well again a longitudinal study said people who tend to be happy by again by longitudinal study I mean study across several years. This was some 10 to 12 years study of some 5000 people and they observed that happiness is also very contagious, and it spreads in social networks.

(Refer Slide Time: 03:56)



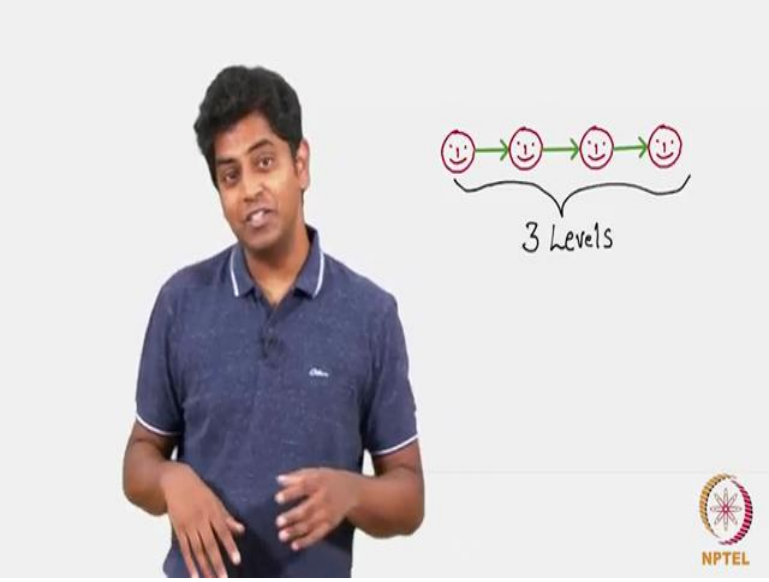
Not just that lot more fascinating than just this.

(Refer Slide Time: 04:03)



They observed, but this happiness is contagious up to three levels which means if you are happy your friends are happy and their friends are happy and their friends are happy I mean that is why the startling, that it is not just at one level, it is are up to three levels.

(Refer Slide Time: 04:07)



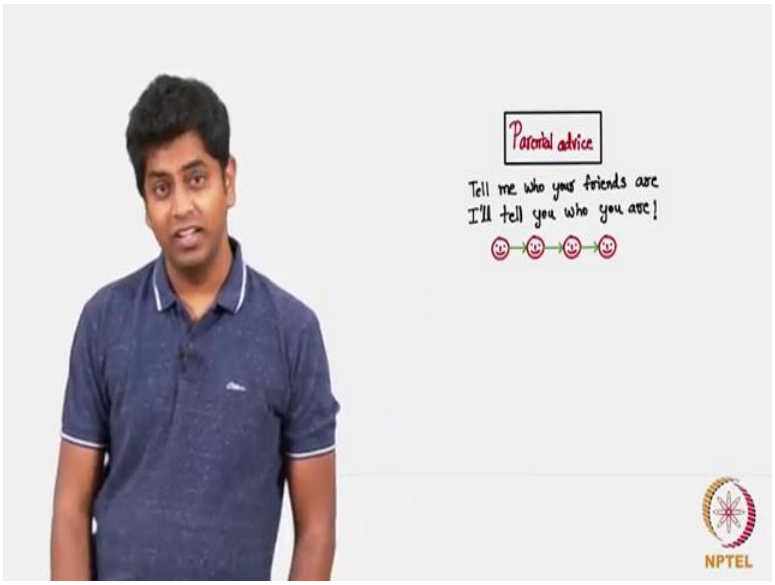
Assume I have 100 friends, 100 friends have 100 other friends and they have 100 more friends. So, up to the last level people are happy if I alone is happy. So, which means happiness is it percolates from the top to bottom one person is happy all his friends are happy on so on and so forth.

(Refer Slide Time: 04:23)



Very tough to believe, but yeah research says this is true. At least to a particularly good extant that many of your friends are happy when you are happy.

(Refer Slide Time: 04:48)



Getting back to the parental code that I said, keep good company this also means something more than that, not just keep good company work on understanding what is the company that your company keeps because their happiness implies his happiness which is your friend implies your happiness. We should watch up to several levels. So, the proverbial code of tell me who is your friend I will tell you who you are seem to be

partially true in the sense that there is a lot more to it that it is not just your friends it is your friends' friends who can influence you and their friends who can influence you. Maybe a modern code after this kind of a research should be tell me who your friends are who you are friends' friends or who you are friends' friends are and I will tell you who you are right.

So, this eye-opening research establish a brand-new line of understanding in social networks that we are not just connected, we permeate our thoughts, we spread our thoughts like a contagion. Just the way yarning is contagious, happiness is contagious. Just the way flues spreads obesities spread unbelievable but true. We will see in the forth coming lectures how we can program this and then make our observations.

(Refer Slide Time: 06:09)



I will give you a fictitious data set on which you will be writing a piece of code and you will be inferring all these facts. The fact that obesities contagious, happiness is contagious, I will give a big network and I will give you the longitudinal network data across the timeline and you will infer that yes obesity has become contagious from here to here. Happiness does spread to people across several levels.

Social Networks
Prof. S. R. S. Iyengar
Department of Computer Science
Indian Institute of Technology, Ropar

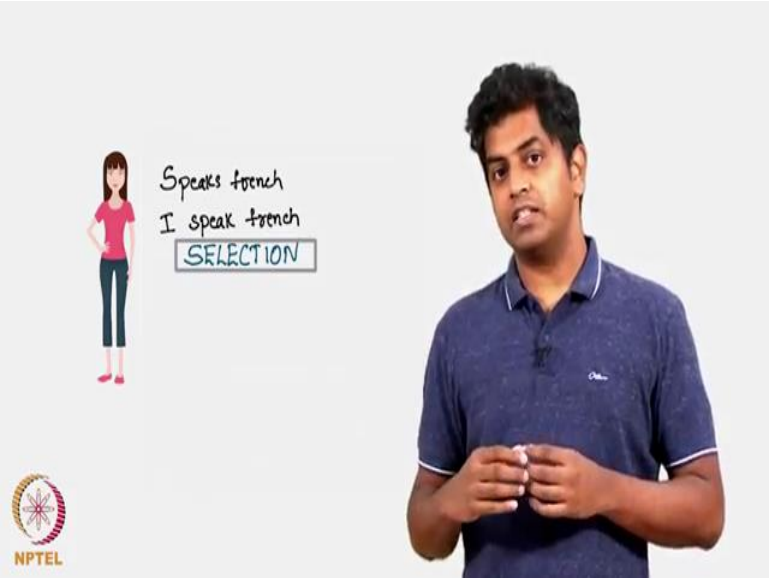
Lecture - 42
Strong and Weak Relationships (Continued) and Homophily
Selection and Social Influence

(Refer Slide Time: 00:12)



We just now saw a nice, question the question was how we pick friends? Do we pick friends who are our types, or do we pick friends and then we become their types? Let us ask a few questions through examples.

(Refer Slide Time: 00:25)



First question I meet this girl who speaks French I also speak French, we start speaking in French and we become friends. See it is very unlikely that in a place like Chennai I mean it is someone who speaks in French while I also know French this is some common interest, we become friends. This is called selection; we select friends based on the fact that they are our types they are just like us.

(Refer Slide Time: 00:58)



This goes like, you probably know of this code which goes like this - Birds of the same feather flock together, correct. This may not always be true. Let us look at examples and classify these examples into selection or social influence.

(Refer Slide Time: 01:08)



(Refer Slide Time: 01:11)



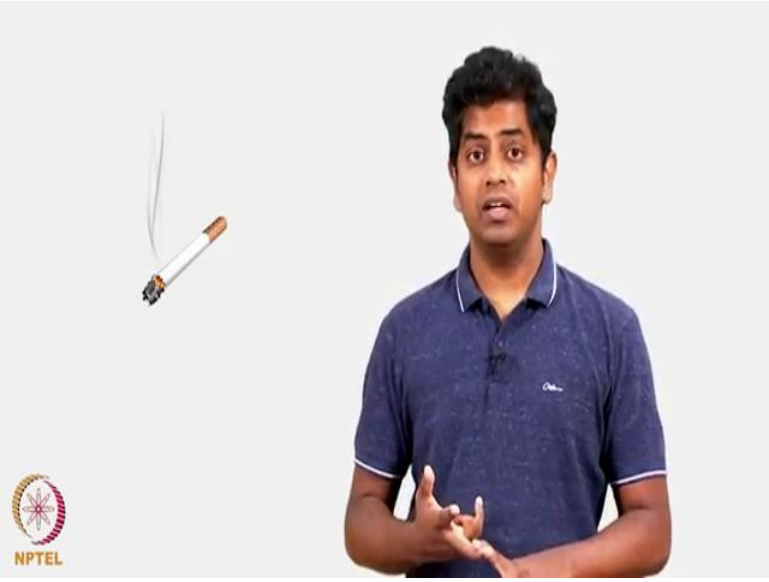
A good example of social influence is smoking. A good example of selection is a girl example I gave you.

(Refer Slide Time: 01:15)



When I meet a person it is know that generally people start smoking because of friends. I meet a friend who is a smoker and he influence me to smoke I start smoking. Same with drinking too, people do not start drinking because they are alone rather, I mean they do not pick up this habit all by themselves, they generally pick up this habit because of their friends.

(Refer Slide Time: 01:56)



So, example number one: I meet a French girl, I know French we start talking I select her as my friend because we both are similar and then there is this friend who asks me to try

smoking I try smoking I pick up this smoking, habit after becoming friends with him I become like him this is called social influence. Let me repeat there is one thing called selection another thing called social influence. The question that you just now saw was what exactly is in play is it selection or social influence. Let us see more examples.

(Refer Slide Time: 02:27)



What about social status? Do you think I become friends with someone whose social status is higher than mine and my social status rises? Not really.

(Refer Slide Time: 02:48)



In fact, two people of the same social status they become friends this again falls under the selection category. Class topper if someone is friends with him, he or she is also a class topper, if she is a class topper if he or she is friends with him he is also class topper, this here again selection is implied.

(Refer Slide Time: 03:03)



Eating habits, you see a person who goes, and eats junk every second day, I probably cannot be friends with him unless I also eat junk right. Here it is social influence. I do not pick a person who eats junk as my friend. I start developing this habit if I am with friends with people, with the person who eats junk.

(Refer Slide Time: 03:30)



Partying, what is it fall under? I am friends with someone who parties a whole lot I also start partying a whole lots social influence. Smoking as I told you social influence.

(Refer Slide Time: 03:45)



(Refer Slide Time: 03:48)



So, we can classify these things into two broad categories namely social influence and selection. One of the biggest questions is what amount of friendships have social influence in them and what amount of friendships have selection factor into them. Let us investigate more of this slowly.

(Refer Slide Time: 04:05)



So, we all know that they say love is blind I do not know how far that is true, I am not even sure if there is any study which says it is true, but for sure parents do not agree with a fact that they would say it better not be blind, love better not be blind when you fall in

love keep your eyes wide opened and then youngsters definitely agree to this, they say oh yeah love is blind right.

But then if you look at psychologies behavioral economies, evolutionary biologist they have claimed that there is a lot of selfishness in love of course, that aside we probably will not debate more on how people fall in love and what happens in love and things like that.

(Refer Slide Time: 04:46)



But in friendships when people make friendships the dynamics of these friendships seems to be very well understood at least under the domain of social networks.

(Refer Slide Time: 04:49)



So, we will be observing, we will be investigating, we will be seeing how friendships happen and what exactly is in play when friendships happen. As we discussed just now is it social influence or selection or both, if it is both what amount of fit has social influence part what amount of it has the selection part.

Social Networks
Prof. S. R. S. Iyengar
Department of Computer Science
Indian Institute of Technology, Ropar

Lecture – 43
Strong and Weak Relationships (Continued) and Homophily
Interplay between Selection and Social Influence

(Refer Slide Time: 00:08)



So, we saw the big question, what is the difference between social influence and selection; which one of the two is happening, are both happening. Now, it is very difficult for us to go ahead and look at the real-world dataset of friendships and ask this question. If we ask this question of, can we have some very concrete dataset where this experiment can actually be conducted, and the facts be unraveled.

(Refer Slide Time: 00:48)



There was one such data set which gave some very promising observations. So, our motive for this video chunk is to understand this piece of research on Wikipedia.

(Refer Slide Time: 00:57)



Let me now motivate the concept by explaining; how Wikipedia works, number one and number two by explaining what one means by similarity measure; these two are important for us to go any further.

(Refer Slide Time: 01:08)



(Refer Slide Time: 01:12)



So how does Wikipedia work? Wikipedia basically is a blank slate; you can basically anyone can go write anything on Wikipedia. For example, you can open the Wikipedia entry of India; edited. If you think some information is incorrect there, delete it add the right content; that you think is right and if someone else thinks; what you have entered is wrong, they will come and edit it and change it to what they think is right this goes on and on.

(Refer Slide Time: 01:42)



One frequently asked question is, can we really trust such a database, where anyone can come and enter anything. Very intriguing, but it is known to be true that when a lot of people get together and start discussing and debating, when they have conflicts of their ideas; true knowledge is known to emerge.

Initially of course, there will be some false information here and there but a lot of people watching it, will converged to the right answer. So, Wikipedia is actually believed to be very trustworthy these days; that is about Wikipedia.

(Refer Slide Time: 02:14)



But what happens in the background of Wikipedia? There are many people who will be coming and editing it.

(Refer Slide Time: 02:20)



If there are conflicts on their belief of what should be put and what should not be put, they talk in the background with each other and the background talking is also visible for all of us; they are called user talk page. One can click on this tab and then take a look at what all discussions they have about the content.

Now, this big dataset is available to our disposal and one can conduct any kind of research on this data set.

(Refer Slide Time: 02:57)



I am now going to explain one such research which actually helped us understand the inter play between social influence and selection.

(Refer Slide Time: 03:09)



As I told you, just now I helped you understand how Wikipedia works; that was the first prerequisite. The second prerequisite is understanding what one means by a similarity measure. So, let me motivate what is the similarity measure now, so let me define what is the similarity measure; it is actually quite intuitive. Let me motivate it with a good example.

(Refer Slide Time: 03:22)



Assume, I like the following dishes I like upma, let us say idli, dosa, pizza and let us say fruit punch; this is the five things that I like upma, idli all those things are South Indian dishes.

(Refer Slide Time: 03:46)



(Refer Slide Time: 03:56)



And let us say my friend Priya she likes pizza, burger let us say pasta and idli; these four things. As you can see, we do not have a lot of commonalities, but we have; we like pizza and idli that is common between both of us. So, what I do is in the denominator I put the total number of items that we like, whatever I like and whatever Priya likes; both put together as you can see is 7 items, out of which we both like 2 items.

(Refer Slide Time: 04:35)



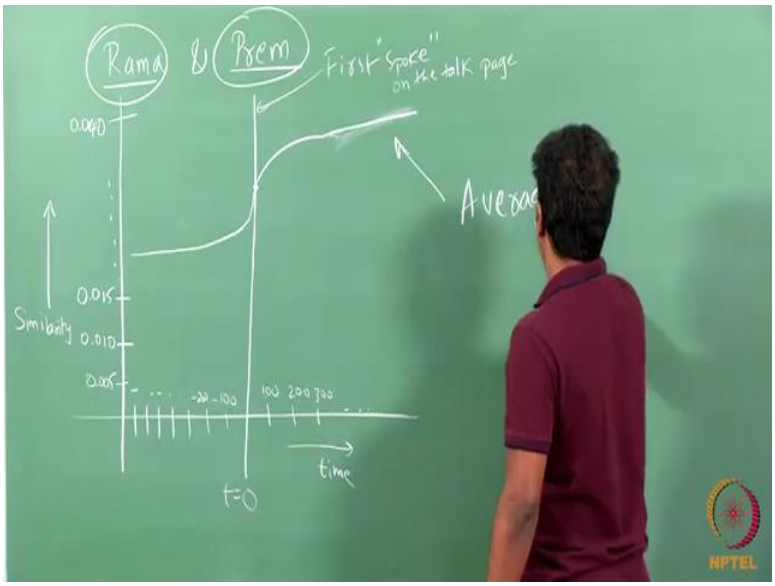
So, I say the similarity measure between me and Priya is 2 by 7; this is what we mean by similarity measures. Now, I am going to use this on my Wikipedia dataset that I was

telling you. So, what do I do? There are two people two authors who are editing Wikipedia pages randomly here and there let us say; I am editing the Wikipedia entry of India and let us say Chennai floods and Wikipedia entry on iPhone 8 and things like that; different Wikipedia pages I am editing. And a friend of mine let us say Peter is editing Wikipedia pages, again some other Wikipedia pages, some ten other Wikipedia pages.

The similarity measure between me and Peter is the total number of Wikipedia pages that we both are editing, total number of unique Wikipedia pages that we both are editing in the denominator; and in the numerator we write those Wikipedia pages we both have edited, just like the similarity measure that I defined between me and Priya. The similarity measure between me and Peter on Wikipedia is total number of Wikipedia articles that we both have edited in the numerator; divided by the total possible unique Wikipedia pages that we have edited, both put together; that is in the denominator. This gives a good measure of how similar we both are on our Wikipedia editing transactions.

So, now that you understand what is; how Wikipedia works, and you also understand the definition of similarity measure. I am going to use these two things in answering the big question of social influence versus selection. So, here it goes; I will do the following, I will look at two people who have spoken to each other on Wikipedia. I told you people do talk in the talk pages, in the background; if I observe that two people have spoken to each other, please note I have a dataset here; in that data set I observe if two people have spoken to each other.

(Refer Slide Time: 06:52)



If let us say two people; I will say Rama and Prem have spoken to each other on the wiki talk page, then what I do is I plot this. What is the plot? The plot is the following, this is my Y axis, this is my X axis, I will see when exactly they spoke. Let me call that time T equal 0 and I will observe, what were they doing before and what were they doing, what did they do after this time equal 0, What do I mean by that?

Let me define what is the X and Y axis properly; on the Y axis, I will put their similarity measure. Their similarity measure is given by let us say 0.005, 0.010 so on and so forth; 0.015 so on up to let us say 0.040; I am doing this on the Y axis. This is my similarity and this is my time; I repeat Rama and Prem have started editing some Wikipedia articles and their similarity measure, I am seeing when they first spoke; this line denotes the time when they first spoke; by spoke let us be clear.

I mean first time they spoke on the talk page and I observed what was their similarity measure before and what is their similarity measure after, it was observed that the plot looks something like this; it looks like this. Let me correct this, it just goes steep it just goes like this that is it. Point to note is the steep increase and further increase and then it becomes sort of constant. What is happening here? Let us just pause and observe; let us revise time, similarly, measure Rama and Prem.

Their similarity measure was; so, this is time when they started let us say some few days before and the first time they met, this probably some minus something minus 100,

minus 200 or whatever is the scale and the time when they first met and then time t equals 100 and then 200 and then 300 so on. Whatever you want, 200 minutes or 200 seconds or you can have any scale here, but this is the time when they first spoke to each other.

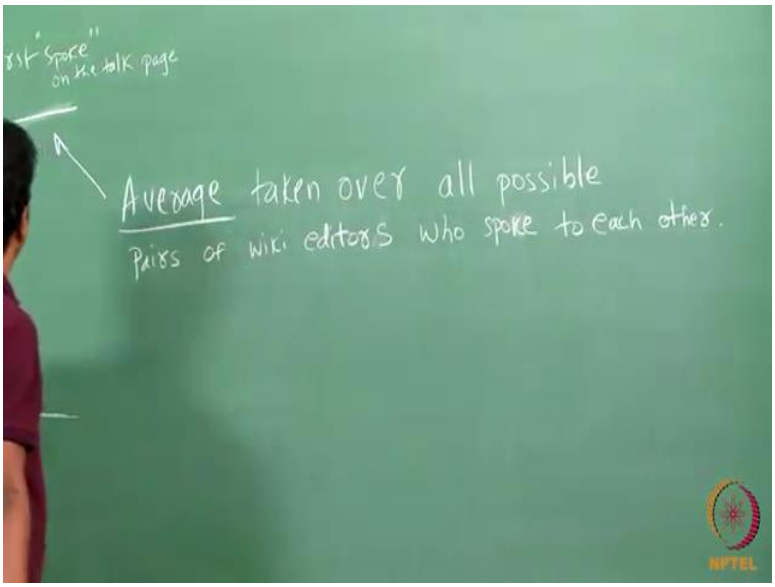
Do you see something happened here? Something fascinating, what does the curve tell you. The curve tells you that before they first spoke to each other, there was a sudden surge of similarity between them. These two people were similar because of which they spoke maybe, because of which they had a talk on the spoke on the talk page maybe. Do you see; after they spoke there was increase in their similarity measure. What does this translate to? This translates the following because that their interest were common, that is what similarity measure means because that their similarity was high, their interest were common; they actually spoke to each other, after speaking there was further increase in their similarity.

This translates the following: two people become friends, if they agree on lot of; let us say dishes in my previous example of me and Priya; maybe if we have 2 by 7 similarity, we may not become friends. If we agree on a lot of food interest, maybe we will become friends; I do not know whether if this is true or not, but assume that people become friends because they have a lot of common food interest. What will happen after we become friends? Once we become friends, we tend to have all the more common food interest, we will probably will hang out together and eat more often the same kinds of dishes, give each other our testimonials of this kind of food in this kind of place and we may want to try these things in each other's presence.

So, the fact that we are similar makes us talk; the fact that we have spoken right now makes us become all the more similar. Fine, this plot was given to you for reasons of better explaining things to you, but the fact here is; if you look at the textbook, this plot is the plot of the average. By average, I mean they have looked at all possible people who have actually spoken and they observed; what was their similarity measure before and what was their similarity measure after. This is the average plot that you see in the text book; which says that on an average people tend to become more similar and then talk and then become all the more similar not just this.

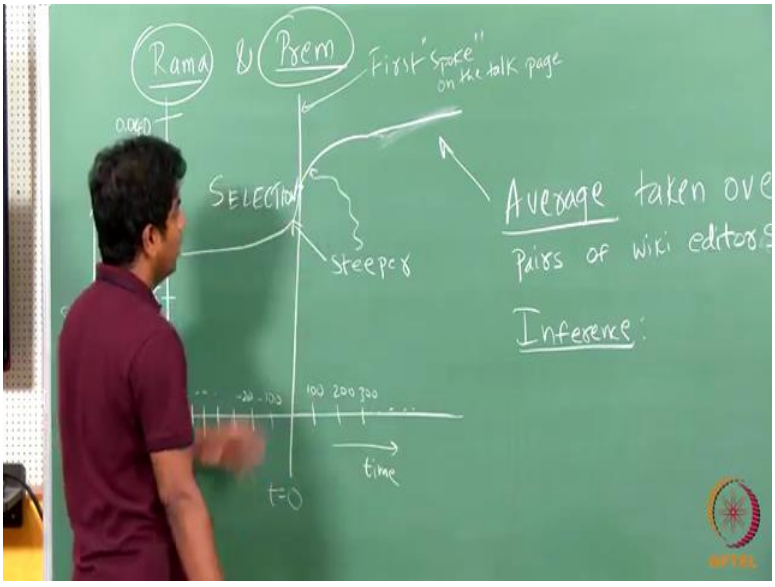
This; what is coming next is what makes the research all the more interesting. They observed; I repeat on an average people become similar; people because this is average, I told you.

(Refer Slide Time: 12:59)



This is average taken over all; let me write that down, average taken over all possible pairs of wiki editors; who spoke to each other, sorry who spoke to each other and we observed that; when they speak at this time, there is sudden increase in their similarity measure before speaking and after speaking; this is the average curve.

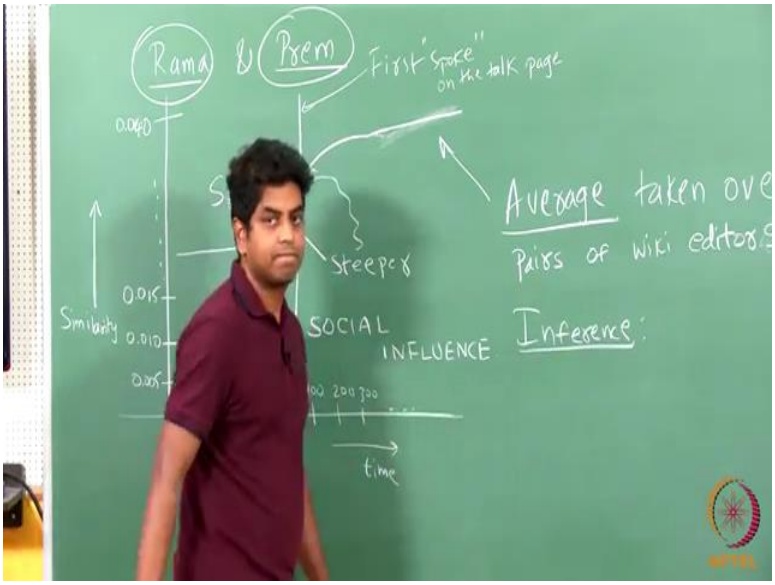
(Refer Slide Time: 13:49)



A mega inference that one can make here is that this is steeper than this, now that is the climax point in the piece of research. This is steeper than this; let me write that, this is steeper as compared to this. This is steeper; than this, what does that mean? That means, that you become similar and then you talk and then you continue to become better and better similar. So, what is this and what is this; take a minutes pause and observe, what is this and what is this in our language that we are motivating from the past few minutes.

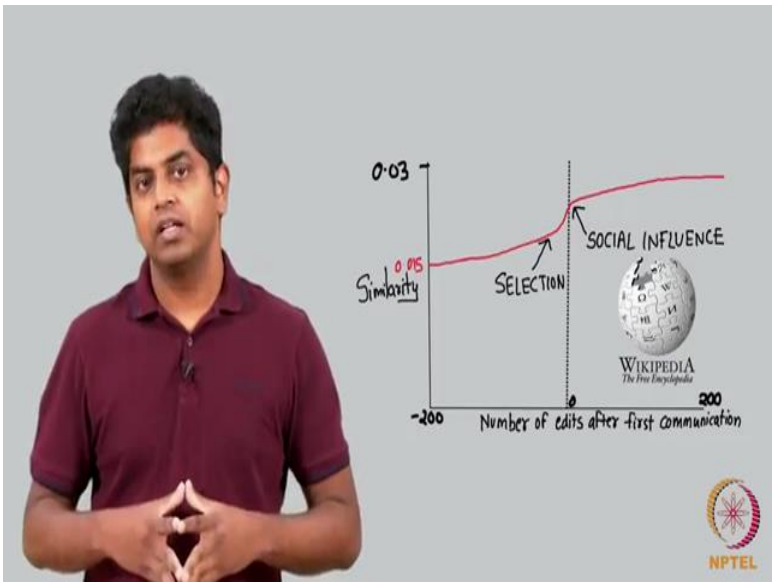
Which one is social influence, and which one is selection? When do we select? We select when people are similar. So, I am sure you would have figured out the answer right now, two people select each other to talk when they are similar. This is selection here happening on this part of the vertical T equals 0 line selection and on this part; once they spoken to each other, maybe they will share more common interest and they will started eating pages together, that is what is happened, that is what this means; there is steep increase in the curve here as well and this is going to be our social influence.

(Refer Slide Time: 15:21)



So, we just now saw the plot and we observed the interplay between selection and social influence on a particular type of dataset.

(Refer Slide Time: 15:34)



Now, let us get a little critical and ask this question; fine the plot is very clear, it says that initially there is a lot of selection happening here and then social influence is also happening, selection seems to be more in play than social influence while both actually happening; that is with this dataset. Maybe this kind of a observation is very context dependent, dataset dependent.

(Refer Slide Time: 16:16)



Now, instead of taking Wikipedia dataset, if I were to take some other data set, let us say the way smokers become friends with each other. Maybe there is a lot of social influence

factor there, the un-selection factor correct and in fact, obesity research says otherwise whatever we saw here it is the reverse of that. Here, there is more selection than social influence, while in the obesity research what we inferred was there is more social influence than selection.

This is a very context dependent observation the people have made in fact; this opens up brand new questions namely. Which is more in play in the society? So, be it with for bad habits, is it more of social influence than selection. For good habits, is it selection and less of social influence; one can ask many such questions and thankfully we do have some datasets on which we can experiment these questions and then find our answers.

Social Networks
Prof. S. R. S. Iyengar
Department of Computer Science
Indian Institute of Technology, Ropar

Lecture – 44
Strong and Weak Relationships (Continued) & Homophily
Homophily – Definition and Measurement

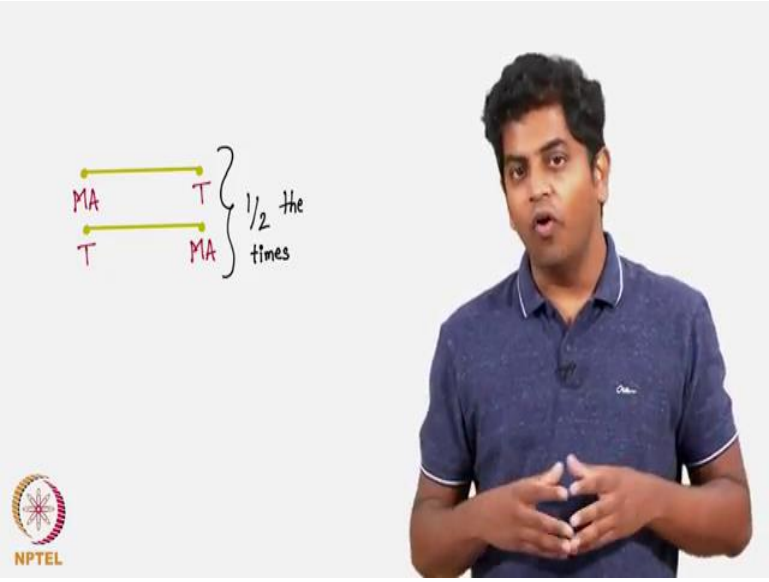
Let us now consider an example assume there is a party going on with 50 teenagers and 50 middle aged people. Now there is some kind of a friendship that gets formed between these two people these two groups of people, and now if I pick a friendship let say uniformly it random I should see on an average one person should be the middle aged person, another person should be the teenage person right. So, this is analogous to you toss a coin twice you will expect a head and a tail or a tail and a head or will be head and a head and tail and a tail, but you observe that head and a head comes with a probability with 1/4 head and a tail comes with a probability.

(Refer Slide Time: 00:51)



1/2 by that I mean a head and a tail and tail, and a head comes with probability 1/2 a tail and a tail come with probability 1/4.

(Refer Slide Time: 01:06)



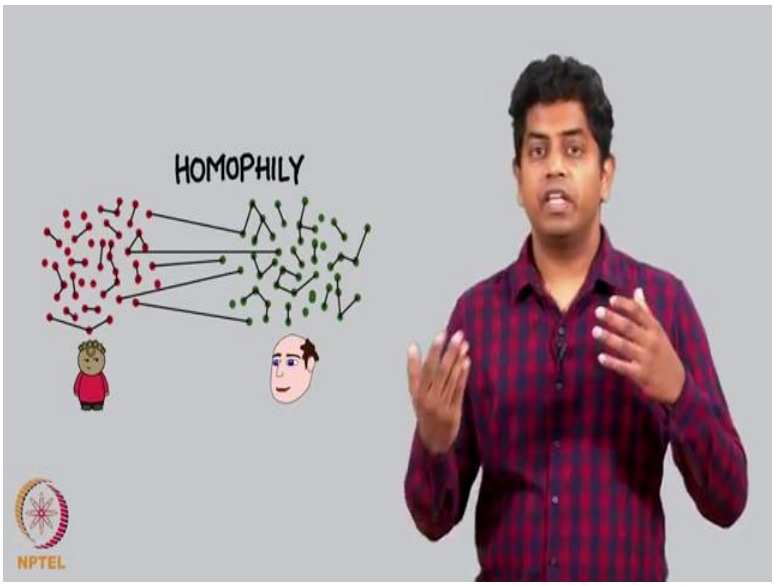
So, it is very likely that if you pick some friendship out of this 100 people, that friendship will have one middle aged and one teenage person half the times that is how I think mathematically it appears to us.

(Refer Slide Time: 01:21)



But you all know the teenagers may want to talk to only teenagers, and middle-aged people.

(Refer Slide Time: 01:31)



May want to talk to only middle-aged people this is called homophily again the code that I was telling in a previous lecture videos.

(Refer Slide Time: 01:38)



Birds of the same feather flock together.

(Refer Slide Time: 01:42)



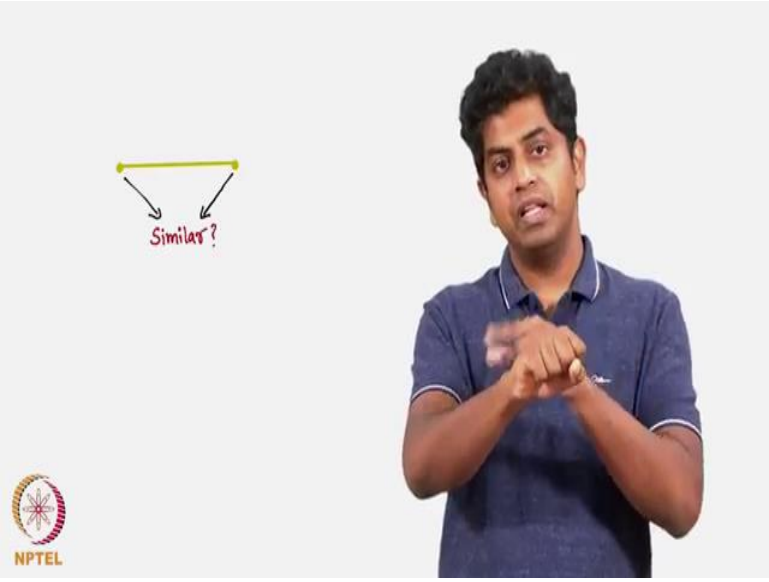
Now, let me ask this question do you think this is true if there is a bunch of people let say 100 people of the same age group and they are sort of there are 50 men and 50 women half men, and half women and they are sort of in a dating spry they want to talk with each other then definitely across genders you will observe that any friendship will have opposite genders definitely for sure.

But then the previous example of middle-aged men and teenagers you saw that tend to cluster middle aged men would be friends with middle aged men, teenagers would be friends with teenagers correct.

(Refer Slide Time: 02:27)

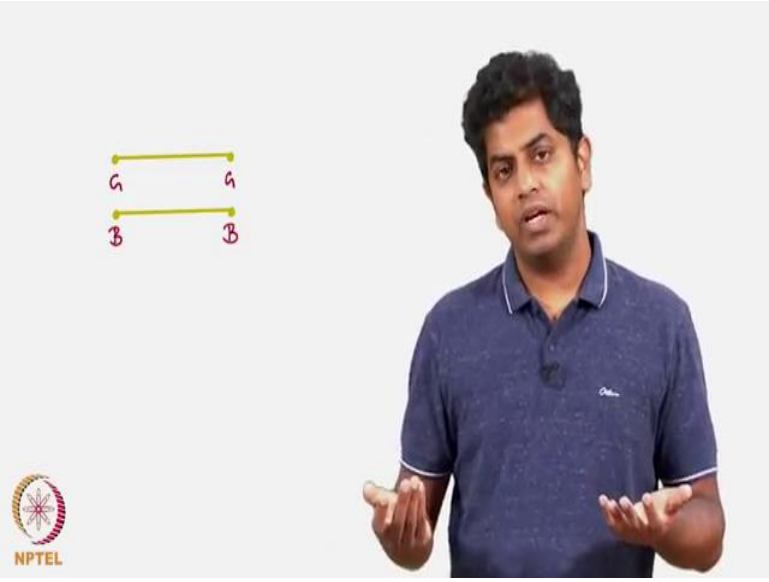


(Refer Slide Time: 02:38)



So, what will do, what will now do is given a network will observe, does it exhibit homophily what do you mean by this? By this I mean given a network if you pick a friendship an edge across the edge are people similar.

(Refer Slide Time: 02:47)



For example, in a classroom if you go and observe that girls and girls are basically friends with each other boys and boys friends with each other girls and boys are also friends, but quite comparatively rare right unless it is a romantic relationship.

(Refer Slide Time: 03:01)

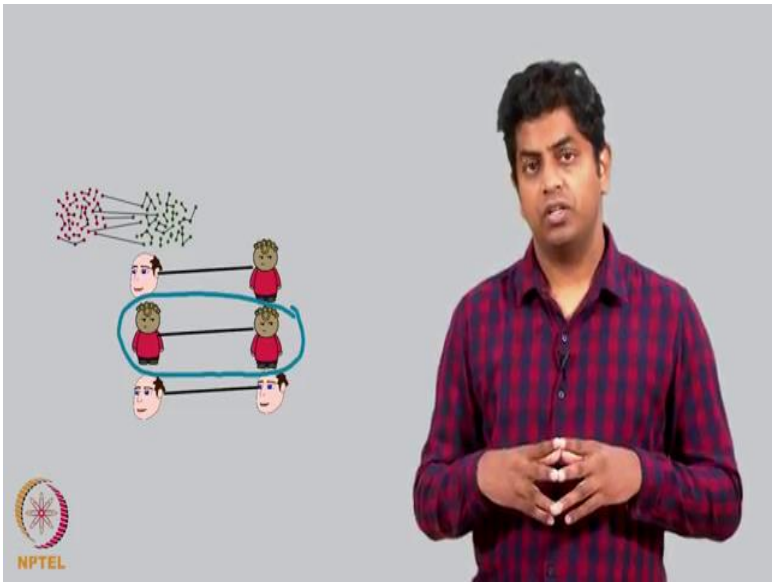


So, we will now try to quantify; what do I mean by quantifying this? First question that we can ask is does the network exhibit homophily this is like asking do you have milk in the vessel the answer is yes or no. If the answer is yes, then you ask another question how much milk you have in the vessel. Similarly, you ask this question does the network

exhibit homophily if the answer is yes you ask the next question to what extent does it exhibit.

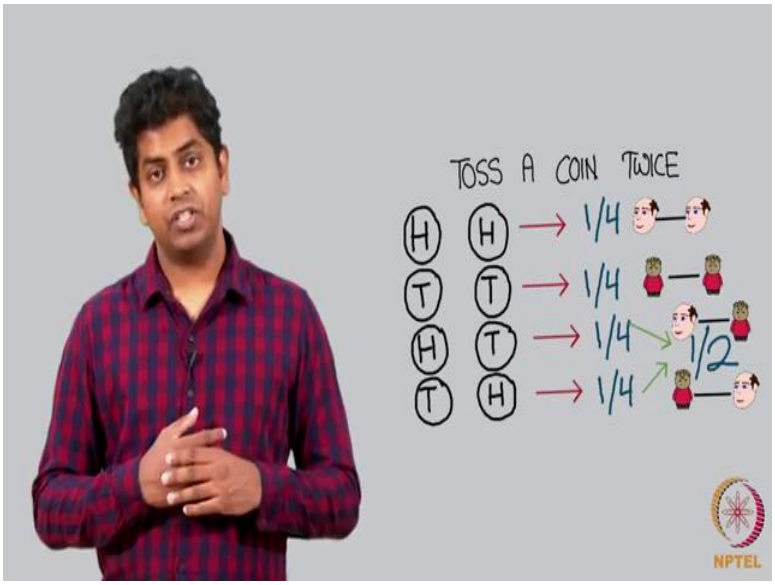
Let us now define what is homophily rigorously? Let me use the same example assume there are 100 people in a party hall, 50 of them are middle aged people and 50 of them are teenagers. Now there is friendship between these 100 people which is actually classified as teenagers and middle-aged people. Now what do I expect? I expect that all of them talk to each other become friends with each other irrespective of whether they are teenagers or middle ages.

(Refer Slide Time: 04:17)



Let me take one such friendship and then observe whether this friendship I pick a random friendship and I see whether this friendship is across a middle-aged person and a teenager or between two teenagers or between two middle aged people. Now what do you suspect would be the answer for this? If I ask you what is the possibility that if you pick a friendship, uniformly at random, that friendship is actually between two teenagers, it is in fact easy to see that this is very similar to a coin tossing experiment.

(Refer Slide Time: 04:53)



I toss a coin twice let say and I get what are the possibilities that I will get a head and a head a tail and a tail a head and a tail or a tail and a head right all these 4 possibilities are equiprobable

Now what is the possibility that I get head and head 1/4 pretty obvious, tail and a tail 1/4 a head and a tail and a tail and a head both put together is 1/4 + 1/4 which is 1/2. Now is in this what is happening in our example where I said a party with some 100 people half of them are teenagers, half of them are middle aged people and friendships between them is what we are observing; half the friendships should be between middle aged and teenager people, and quarter of them should be between teenagers and quarter should be between middle aged and middle aged.

Now, here are two contrasting examples.

(Refer Slide Time: 06:03)



One example is middle aged and teenagers in a party hall, what do you expect? You will definitely expect there is a whole lot of friendships between teenagers and between middle aged people by that I mean the expected probability of half which is the friendships between middle aged and teenagers across that is half right that will not divisible infact that will be a lot less it will be some 18. If the friendships across these two parities are less than half, then we say we observe homophily here this makes perfect sense you see let me rephrase.

Party with 100 people half of them 50 of them are teenagers, 50 of them are middle aged people you pick friendships and observe what kind of friendships people have, you will observe that the friendships between teenagers and middle aged people are a lot less what do you mean by a lot less? You expect it to be half the friendships out of all the friendships half of them should be between middle aged and teenagers, but you observe that less than half is between middle aged and the teenagers, which means there is homophily happening here, there is the that the same old court I have been repeating birds of the same feather flock together is what is happening here.

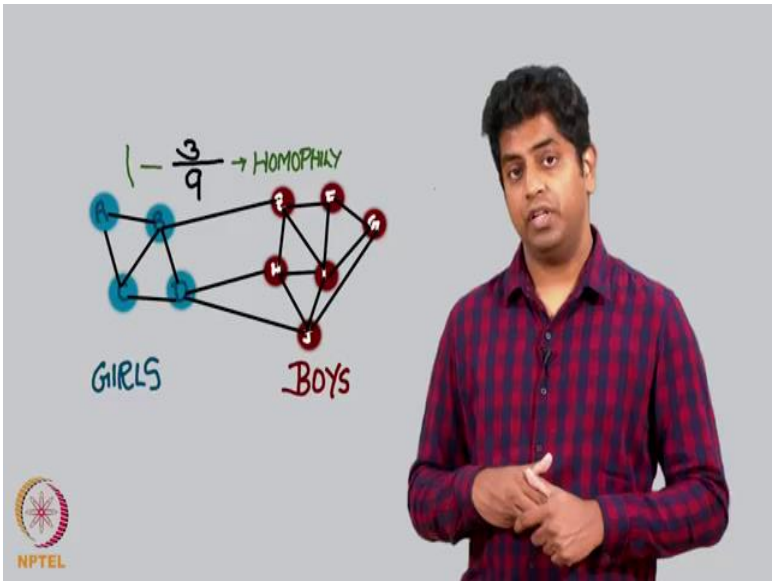
(Refer Slide Time: 07:43)



Now, let us look at the contrasting example; the contrasting example is this assume there is a party hall full of Bollywood hero and heroines right. So, a 50 of them are Bollywood heroes 50 of them are Bollywood actress. Now what kind of friendships will happen here? I suspect that the actors have the tendency of going and talking to the actress more than actors themselves. An actress has the tendency of talking to an actor than a fellow actress. Now if I look at the kind of friendships that happen in a party hall between a Bollywood actors and actresses what would you observe think for a moment. You will observe the exact opposite of the previous example right.

Now, there will be very less friendships within the actresses and there will be a very less friendships within the actors there will be a lot of friendships across. Now what is happening here? There is no homophily here absolutely if anything there is very less homophily here correct. Why? Pick an edge out of all friendships that happen in this party hall of Bollywood actors and actresses, pick all the friendships and see how many of them are actually between the actresses and between the actors, you will observe that a lot less is between the same type of people, a lot more will be across right which means every friendship has different types of people across the edge, which means homophily is a lot less.

(Refer Slide Time: 09:25)



So, let us now look at a nice example; look at this network of 10 people let say in a class of 10 there are four girls namely a b c and d and then 6 boys e f g h i and j, and this is the network between them there is some friendships there is a friendships fine done. Now let me see what the nature is of friendships. Let me look at one such friendship let say between a and b as I can see friendship between a and b is present and a and b are both girls. If I randomly pick a friendship and look at who are on the other side of the friendship do, I mostly see two boys or two girls or do I also see a boy and a girl now let me quantify this properly. Let me first count the total number of edges on this network as you can see there are 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 edges here, which means 18 friendships are there out of this, 18 friendships I expect half of them as we are being discussing I expect half of them to be between boys and girls.

Now, let see how many of the friendships are actually between boys and girls. I observe that 1 2 3 and 4 these are the only four friendships between boys and girls, no not four actually it is three one two and three just 3 friendships between boys and girls, the rest is either between boys or between girls.

So, I now try to define what is homophily. By Homophily I mean take look at this fraction in the numerator you put what are the actual number of friendships across the divisions boys and girls that is in the numerator, in the denominator I put the expected number of friendships between boys and girls. Since there are 18 friendships, I expect

the number of friendships between boys and girls to be half of 18 which is 9. Under the numerator is the actual friendship, across boys and girls which is 3. So, 3 by 9denotes homophily correct. Lesser this fraction more is the homophily think about it, more this fraction lesser the homophily why is that? If you think the numerator is very small, then the fraction becomes very less. What do you mean by numerator is very small? By numerator small I mean that the actual number of friendships across boys and girls is very small, which means there are lot of friendships happening within them which means there is a lot of homophily.

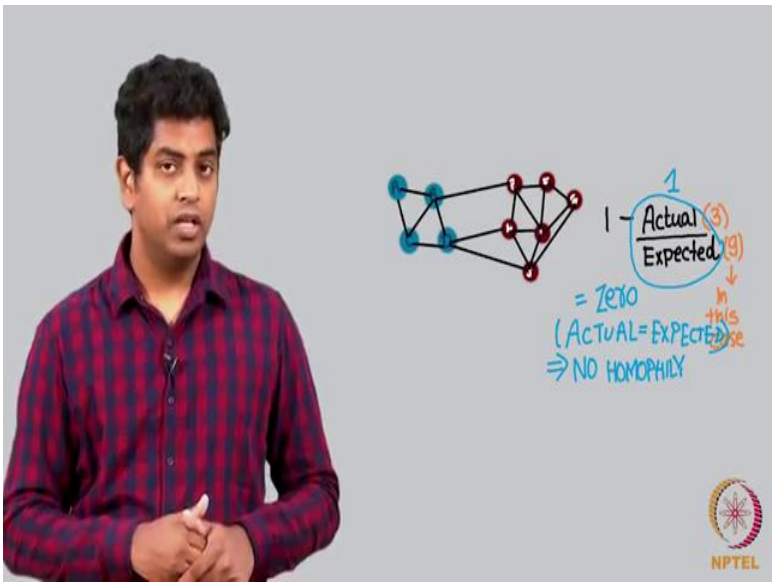
Now, what we do is a typical trick that we do in math physics, computer science in general is try to subtract this from one. So, what I do is I take 1 minus this fraction, actual number of edges divided by expected number of edges what does this denote? This simply denotes that as this number goes higher there is more homophily, as this number goes lower there is lesser homophily.

(Refer Slide Time: 13:03)



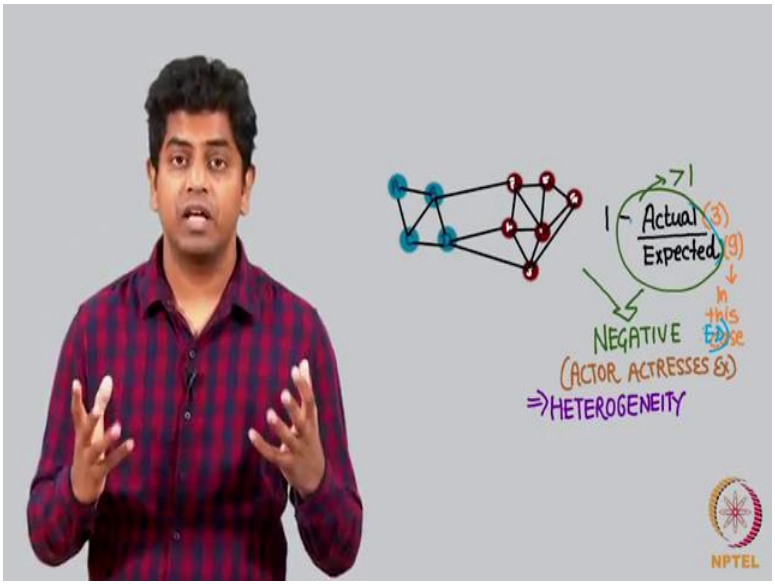
Let us look at this number do you look at the fraction? The fraction can actually become equal to 1 and hence 1 minus 1 can be 0 what does 0 denote here?

(Refer Slide Time: 13:10)



0 denotes that the number of edges across these two parities boys and girls, is the expected number of edges which means homophily is not happening here. Please note there is also possibility that this big number this entire number 1 minus this fraction can become negative; when is it negative? It is negative when the actual number of edges are more than the expected number of edges.

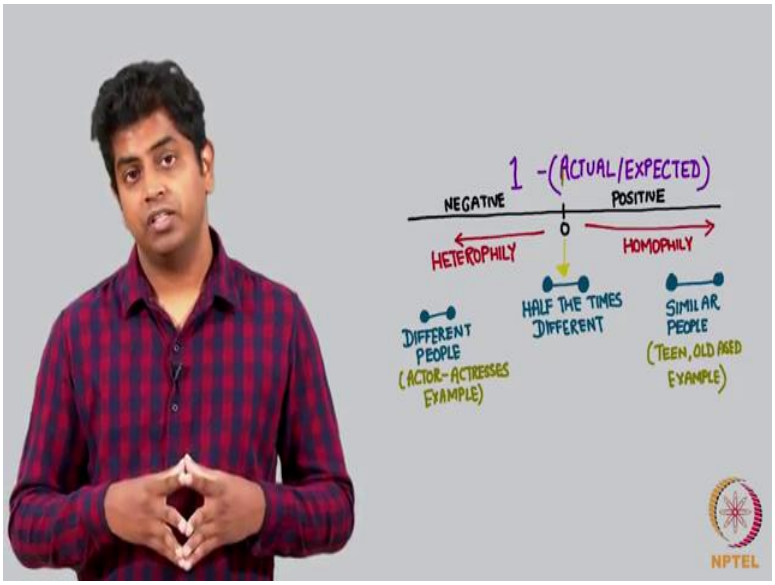
(Refer Slide Time: 13:36)



Do you see the actor actresses party example that I gave you, where friendships are mostly between two people of the opposite gender? Now what will happen? The

expected number in the denominator and the actual number in the numerator the actual number will be more than the expected number in that case 1 minus of this will become negative; and the negative number denotes the heterogeneity in the network, where friendships are actually between two people who are of opposite types, and a whole lot of friendships are like this.

(Refer Slide Time: 14:31)



Now let summarize if the friendships are such that across the friendship two people are similar more often, then you call such a network as homophily the network exhibits homophily. If they are half the times different then you say the network looks very random, then this number is 0, 1 minus actual by expected is 0. When almost always the types of people across this edge are different like in the actor actresses example party example, then you say such a network exhibits heterogeneity and is no way homophilic.

Social Networks
Prof. S. R. S. Iyengar
Department of Computer Science
Indian Institute of Technology, Ropar

Lecture – 45
Strong and Weak Relationships (Continues) and Homophily
Foci Closure and Membership Closure

Neeru (Refer Time: 00:10), I wanted to ask you something (Refer Time: 00:12) I think we are batch mates.

Yeah, you are right; I was there but I remember seeing you in Delhi University.

Yeah, you are correct and see we are here in the same yoga class.

Yes.

What a co-incidence?

Yes.

I think we should catch up sometime (Refer Time: 00:25) for a coffee?

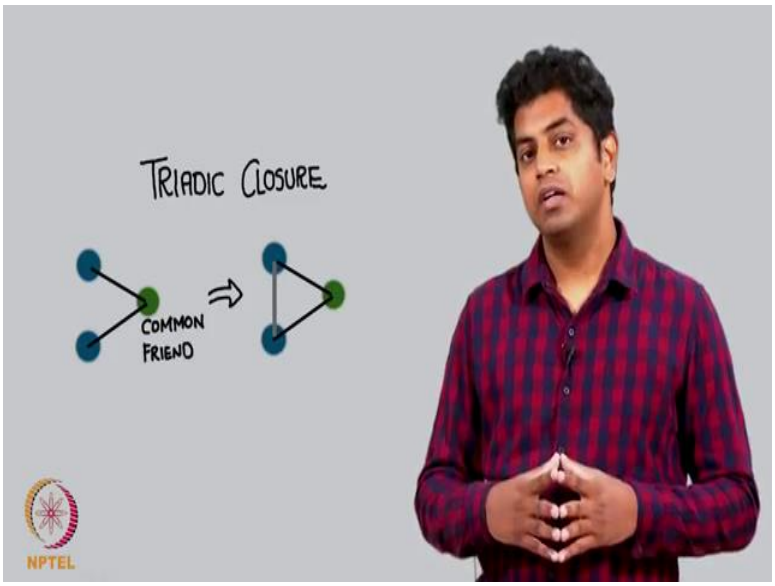
Sure; no problem.

Tell me regarding; which batch you are?

I was in (Refer Time: 00:31)

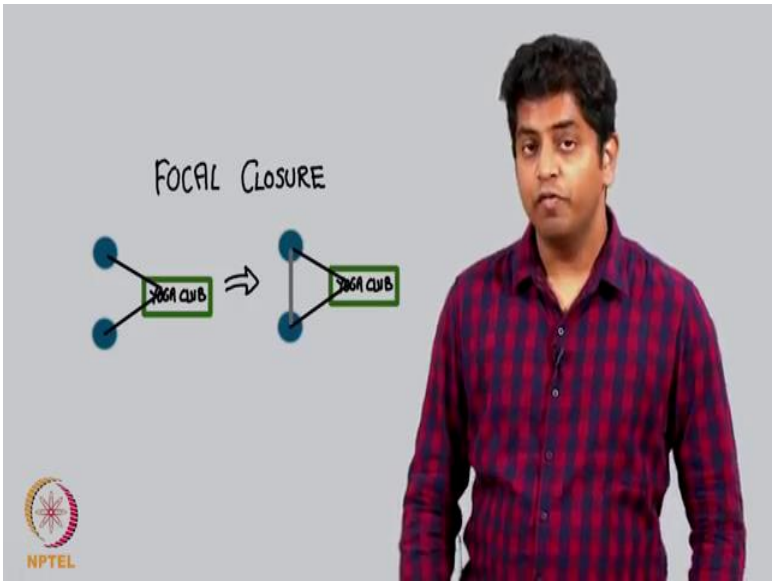
We just now saw a video clip of Amit and Neeru being part of yoga club and then they realized that they are actually part of several affiliations while, we have been looking at triadic closures; where 2 people who have a common friend trying to become friends with each other.

(Refer Slide Time: 00:47)



It is not just a common friend which makes 2 people get closure, sometimes it also affiliations. If they are part of a common focus; let us say me and you are part of kung fu club, we trying to be friends with each other. If we both are part of yoga club, we will be friends with each other. In the example between Amit and Neeru, they were part of yoga club and then they realized that they are part of many other common foci, they are now becoming friends with each other; this is called focal closure.

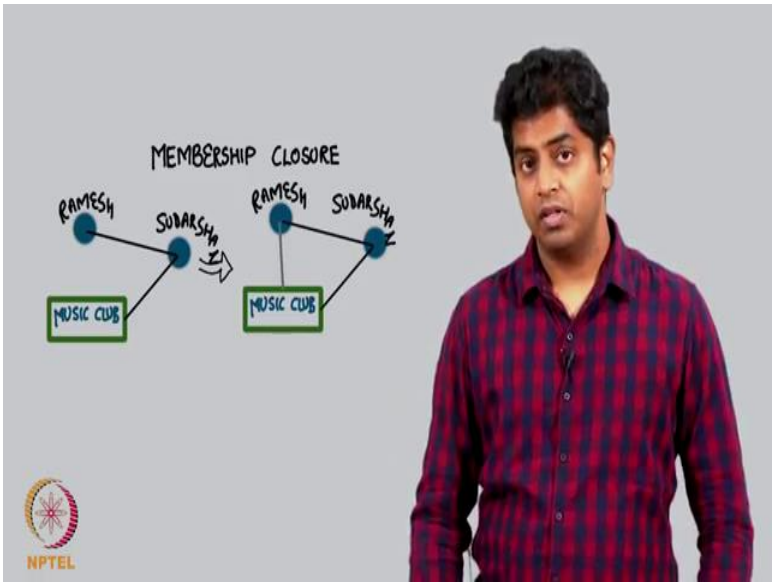
(Refer Slide Time: 01:29)



Where, we saw what a triadic closure is. Similar is focal closure, triadic closure meant common friend 2 people become friends. Focal closure means 2 people have a common place where they meet, where they talk, where they see each other and that can also result in the friendship that can happen between these 2 people that is called focal closure.

Now, there are basically three types of closures that we will now observe; first is of course triadic closure, second is focal closure that I told you, third is membership closure.

(Refer Slide Time: 02:07)



Assume me and Ramesh are 2 people who know each other and I go to the music club and now because he is friends with me, I will take him to the music club; this is called membership closure, I make my friends as members of this focal point; which is let us say music club.

(Refer Slide Time: 02:31)



So to per phrase, we saw triadic closure; common friend means 2 people can become friends, focal closure; if there is a common focus, let us say Kung Fu Club, yoga club; 2 people can meet there and become friends. The third one is membership closure; I am part of a music club; I can make Ramesh become part of the music club.

Social Networks
Prof. S. R. S. Iyengar
Department of Computer Science
Indian Institute of Technology, Ropar

Lecture - 46
Strong and Weak Relationships (Continued) and Homophily
Introduction to Fatman Evolutionary model

Now, we are going to do a very interesting screencast, programming screen cast. So, what will be doing in this programming screen cast is, to make an evolutionary model. What is an evolutionary model? We have studied 3 main concepts in this chapter.

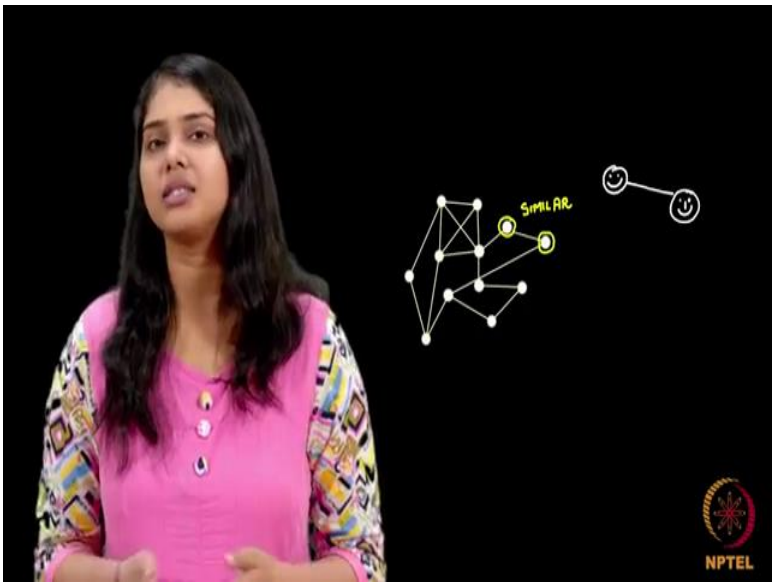
(Refer Slide Time: 00:26)



So, the first one is homophily; people who are alike similar to each other, they tend to become friends with each other. Then we looked at 3 different kinds of closures triadic closure, focaic closure, membership closure and we also looked at social influence.

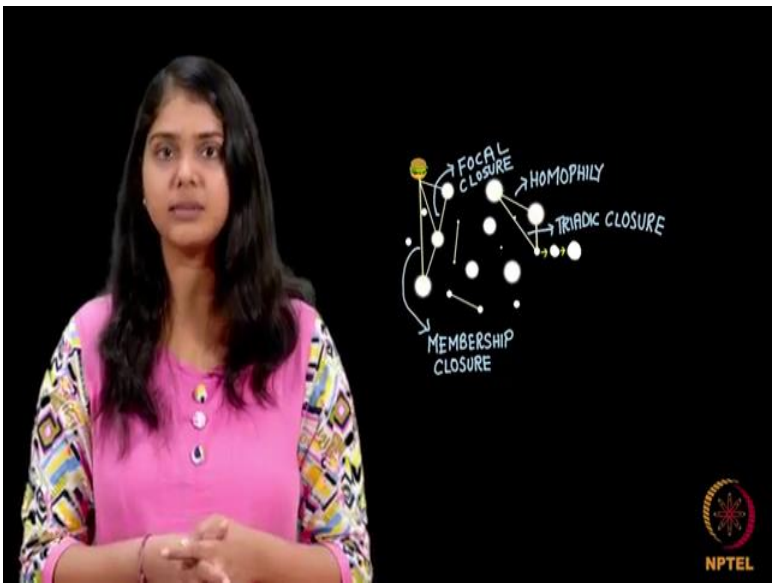
So, in a graph, in a network of these people this people keep changing, so first of all new edges keep coming in this network.

(Refer Slide Time: 00:54)



So, when 2 people look at each other and they find that they are similar to each other; they become friends with each other. So, you see that this network is evolving with time, more friendships are coming in this network and probably some friendships are getting broken also. And then people they change their properties because of social influence; for example, somebody who is not getting good marks becomes friend with a topper and starts getting good marks. So, we have also seen these kinds of things known as social influence.

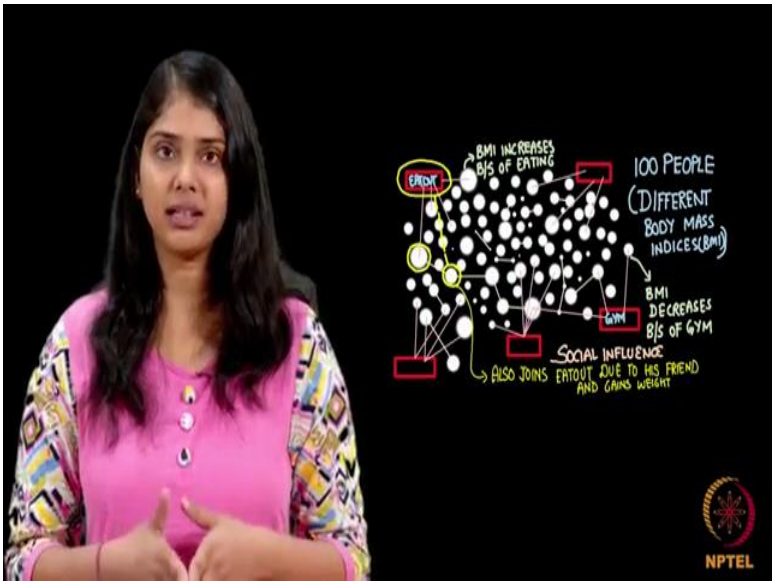
(Refer Slide Time: 01:34)



So, we will try to model all these 3 things as an evolutionary model; what will be modeling is, there is a city and this city has different-different kinds of people; kinds of people in terms of their body weight. So, I hope you remember this Fatman hypothesis which we discussed during the chapter that; beware of your fat friends, if your friend is fat the probability of you gaining weight increases. So, we will be modelling a city in which there will be people with different-different body weights, different-different BMI's and then we will look at all these 3 phenomena happening over this network. So we will see that, we will code it; mainly we will code it that people who are having similar body weight, they tend to become friends with each other. We will be looking at triadic closures also; 2 people having same friend tend to become friends with each other.

We will be looking at foci closure also; that is if 2 people they are going to the same eat out place or let us say the same gym, they tend to become friends with each other and we will also be looking at membership closure. So, assume I am a friend with somebody who regularly goes to an eat out place, so I also start going to that eat out place. So, all these different kinds of closures and we will also being modelling social influence that will be in a more subtle way, which I will tell you, so how does this programming screen cast proceed.

(Refer Slide Time: 03:04)



So, first of all we have this network and let us say there are 100 people in this network and these 100 people have some BMIs. So, we assign some random BMIs to each of these persons and then we look at; when the model proceeds, when the evolution continues people who are having similar BMI, they make more and more edges with each other. And then we also have; five, we will introduce five social focis in this system as I tell you and 2 social focis out of these will be one is an eat out and another is a gym; which has a direct impact on the BMI's of these people.

So, these people they will be linked to different-different social focis and this social foci will change the property associated with these people; that is, if you are part of a gym you will be losing weight, if you are part of an eat out place you will be gaining weight and then how do we implement social influence. One of the reasons for social influence, if you remember or the sheared context, so if one person goes to an eat out place; I will also start going to that eat out place and I start gaining weight.

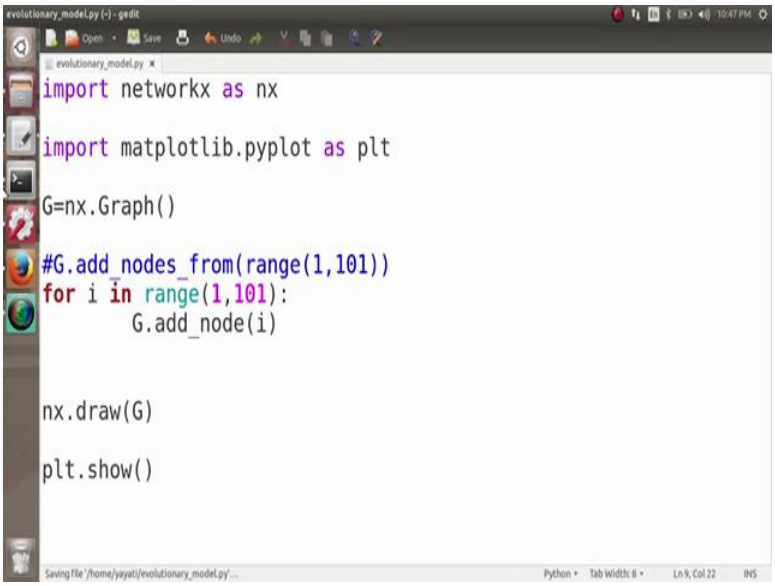
So, this is the way in which we will capture the influence of; we will capture the phenomenon of social influence. So, I might have less weight, but then over time I might become friends with somebody because of maybe let us say a triadic closure. So, we have a common friend and this common friend makes both of us friend, but now this new friend which I have made is going to an eat out place; I am in his company. So, I will also start going to this eat out place and I start gaining weight.

So, we will see how we code such a complex scenario and one assumption in this programming screen cast that we will be taking, although there are many assumptions. One main assumption is that, new edges only come with time; previous edges as in the existing edges they are not deleted, only the new edges come in this network. But you will see once when we code the entire thing, it is actually very easy and you can modify this model in many ways you can have old edges leaving the network, you can increase the number of social foci, you can increase or decrease the number of people and you can incorporate many new interesting things in this model.

Social Networks
Prof. S. R. S. Iyengar
Department of Computer Science
Indian Institute of Technology, Ropar

Lecture - 47
Strong and Weak Relationships (Continued) & Homophily
Fatman Evolutionary Model - The Base Code (Adding people)

(Refer Slide Time: 00:05)

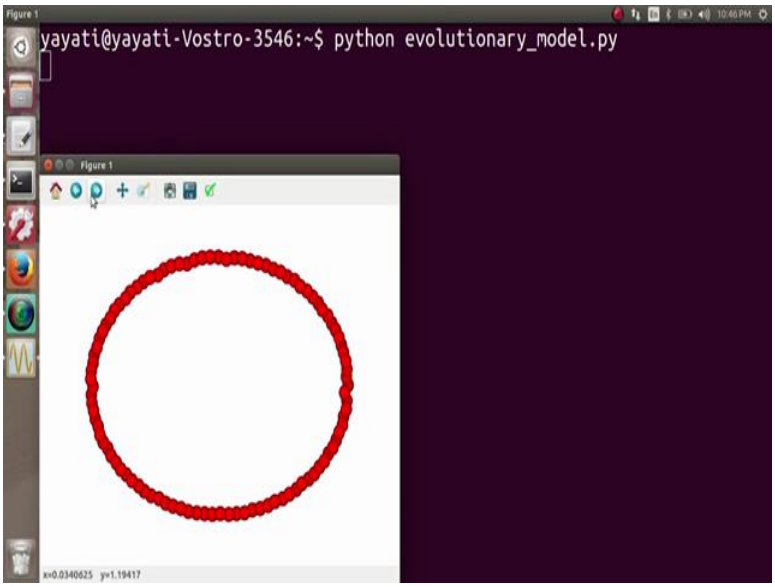


So, what do we do first is we have to create a graph on 100 nodes where everybody has a different BMI? So, mainly we have this city and the city has 100 people which we depict as 100 nodes. So, to make this graph first be import a function sorry be import a package import networkx as nx and let us save this file, let us save this file as let say evolutionary model.py. So, we save this file a evolutionary model.py. The import networkx is nx, now we have to create a graph on 100 nodes which we know is very simple we have already created many graphs.

So, what we simply need to do is first of all create a graph G = nx.Graph and then we have to at 100 nodes here. So, we write G dot add_nodes from and this function demands a list here. So, what is the list here? Range 1 to 101. So, what will this function do it will add 100 nodes to this graph with the indices being 1 2 3 4 up to 101. So, this is one we do will get will soon look at another wave of doing the same thing.

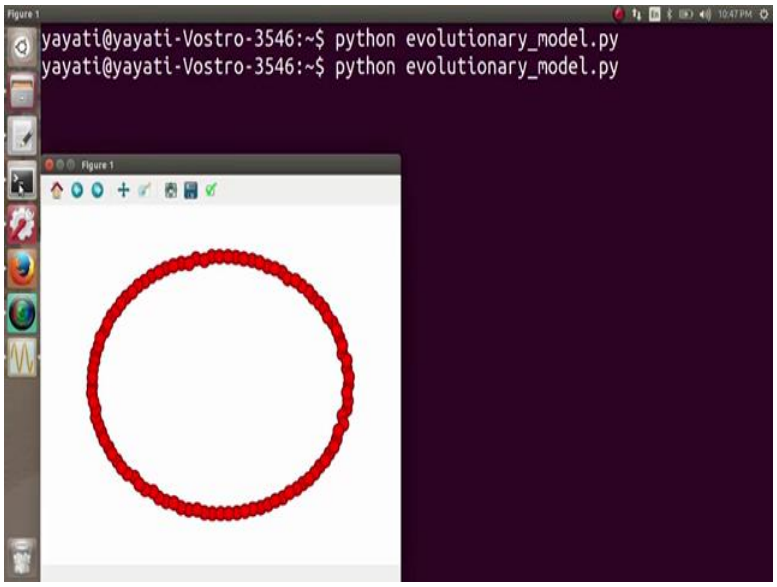
And next I want to plot this graph and see how does it look like, so far as we know for plotting we need import mat plot lib dot pyplot as plt and then we can use a function and next dot draw G and then plt.show.

(Refer Slide Time: 02:07)



I save this file let us run it and see, python evolutionary model.py and here we can see a graph with 100 nodes. Let us quickly look at another way of doing the same thing for you might be already knowing it I am just repeating it.

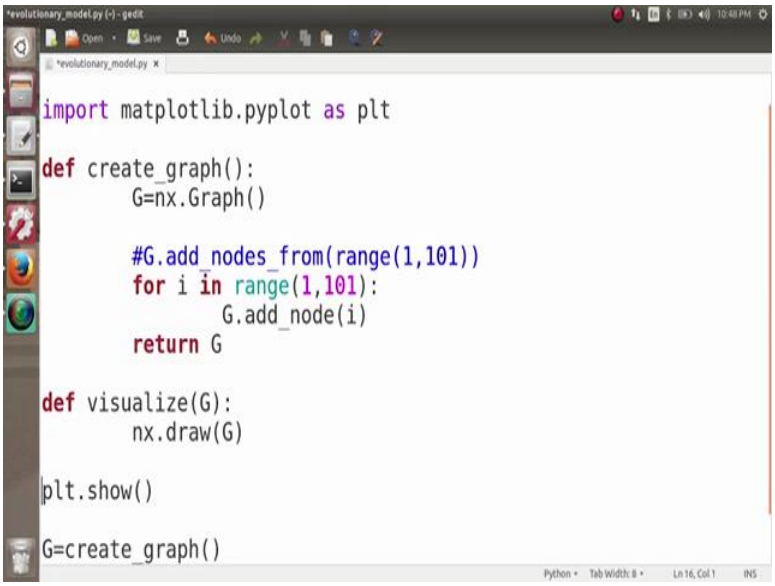
(Refer Slide Time: 02:32)



So, here what we have done is we have made a list here and then passed it what we can do is have a four loop here. So, we can do for i in range 1 to 101 what do we do G. add_node(i), and it creates the same output as before. So, when we run this file and we do python evolutionary underscore model.py we get the same graph perfect.

Next what we want to do is we want to a see some random BMIs to these people. So, for assigning random BMIs to these people what we will do is. So, we will be first before doing that we want to use a good use of we want to make a good use of functions here. So, whatever will be doing will keep writing it everything in functions only. So, this particular piece of code G = nx.Graph to add_node(i) was for creating our graph.

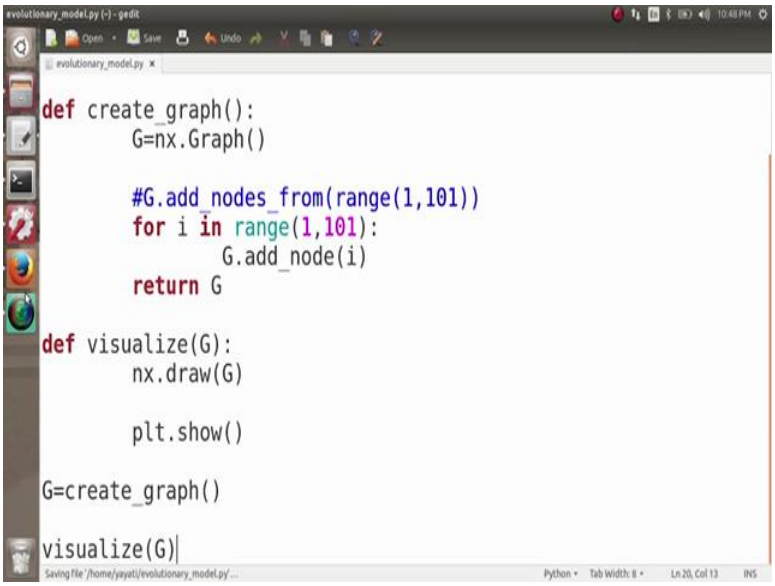
(Refer Slide Time: 03:44)



So, what we do here is we simply pass a function here G = create_graph and this entire thing we shift under the function create underscore graph. So, we have this function here - define create underscore graph and what this function does is it creates a graph haven 100 nodes and it returns the graph G.

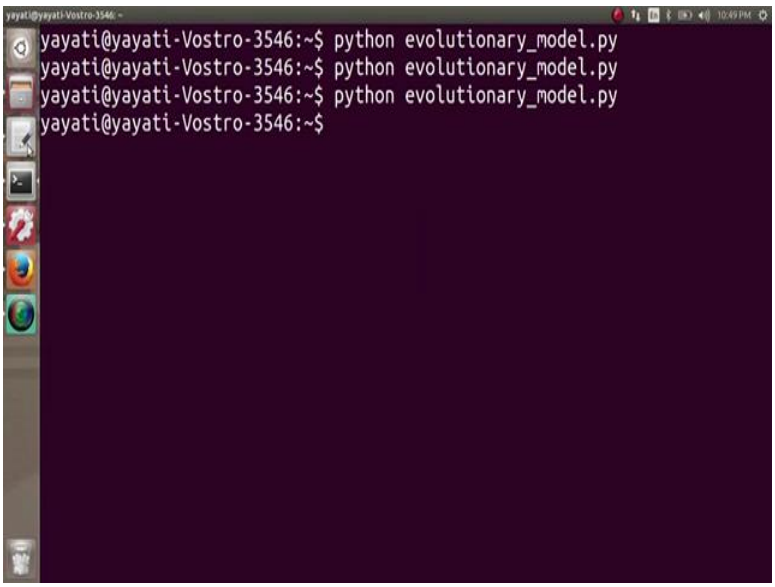
And these two statements nx.draw(G) and plt.show we are using it to visualize our graph. So, what do we do is we define another function here define and then we name this function as visualize, define visualize G what it does is it will draw or graph and showing?

(Refer Slide Time: 04:39)



So, we pass both of these statements inside this function and whenever we have to see this graphic simply called visualize G. So, let us run it and see quickly.

(Refer Slide Time: 04:47)

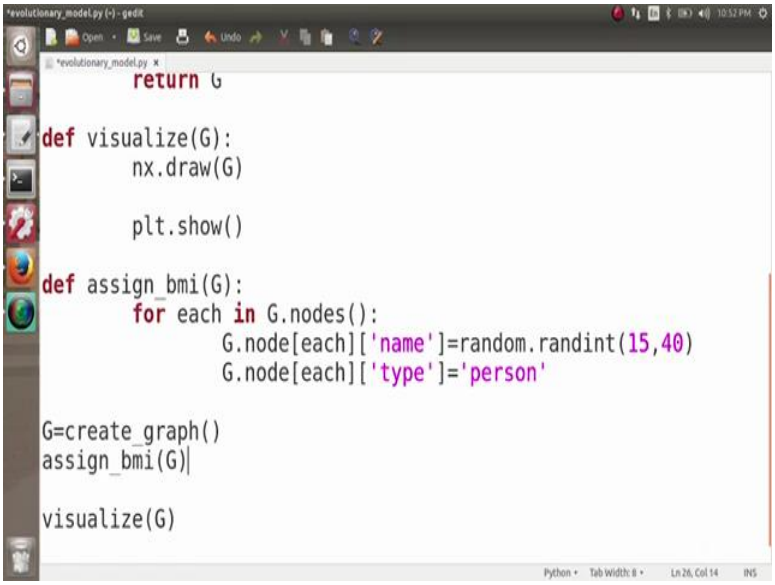


So, we get here the output which we want it. So, till now everything is perfect. So, we have a graph having 100 nodes.

Next what we want to do is we want to assign a body mass index to every person here in this graph. So, here I have 100 people and you want to assign a BMI to every person. So,

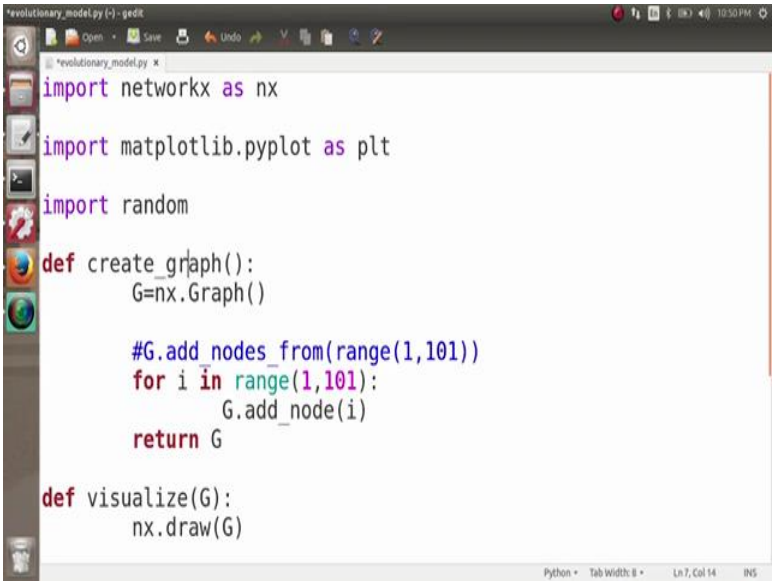
for that again we take the help of a function and we call the function assign BMI and we pass the graph G in this function.

(Refer Slide Time: 05:16)



So, let us write down the body of this function, we have define, assign, underscore BMI and we have a graph G here. How do we assign a BMI, so we know that BMI is going to be a number from 15 to 40? So, what we are going to do is for every node in the network we generate a random number from 15 to 40 and assign that number as the BMI of this node.

(Refer Slide Time: 05:56)

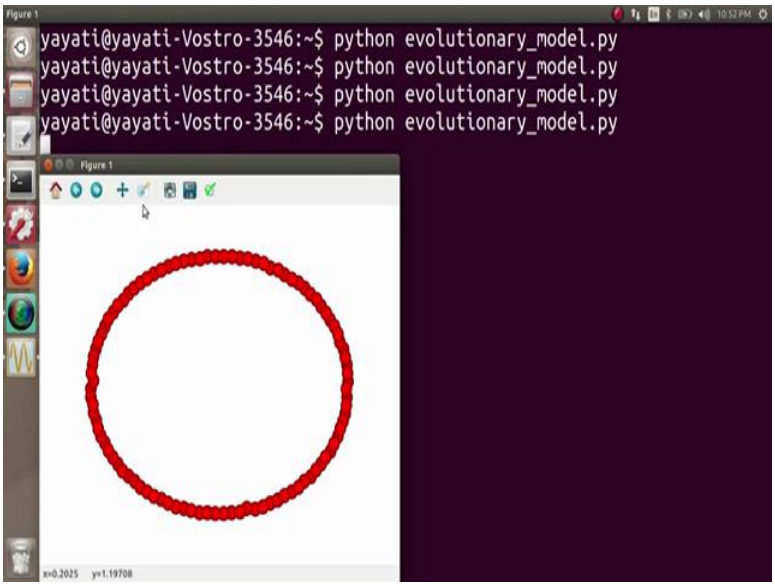


So, since we need a random number, we need derive package random here. So, we import our package random here, and we come back to our function assign underscore BMI. How do we assign a BMI to our node? So, probably you will remember we have used the attributes of the node, we have learnt about the attributes of the node. So, we are going to make the use of the attributes of the nodes. So, how do we do it for each in G dot nodes? So, we have an iterator which iterates you every node in G. So, for each in G dot nodes what do we do is we add an attribute G.node each and let us have the name of this attribute as name. So, better option was having the BMI here, but we are using the name of the attribute to be name itself have a reason why I am using the attribute name to be name here which you will understand as you see more of this code.

So, every person has here, and attribute and the attributes name are name. So, the attribute is name and the value of the attribute is random number, random integers from 15 to 40 what I do is random.randint and it should be from 15 to 40. In addition to an attribute name I want one more attribute associated with each node here and you guess what that attribute is going to be. I told you that we are going to have two types of nodes in this network. So, one type of nodes is corresponding to the people in the network and another type of node is corresponding to the social for kind. So, will need to distinguish between these two kinds of nodes, so hence I take an attribute here G dot node each and I name this attribute as type.

So, G dot node each type equals to and these are 100 people. So, I name this as I keep the value as person. So, for each of these 100 nodes there type is equal to person. So, we have assigned a BMI to every node and let us put this assign BMI function before visualizing the graph.

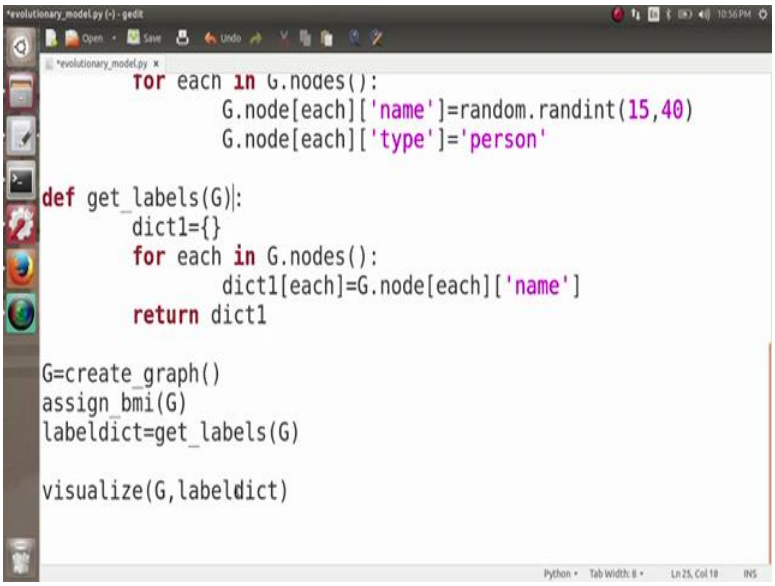
(Refer Slide Time: 08:31)



And let us run the code and see it now run the code and we see it so, but the graph looks like the previous graph only so every node here has a BMI, but we do not see any BMI here why, because currently or nodes are having no labels. So, we need to label our nodes and this particular labeling which we want is the BMI. So, every node should be labeled with its BMI. Every node should have a number written on it and that number should be the BMI of this node.

So, how do we do that? After assigning BMI is to these nodes, we want these as labels. So, mainly we need a dictionary for all the labels which can be displayed.

(Refer Slide Time: 09:20)

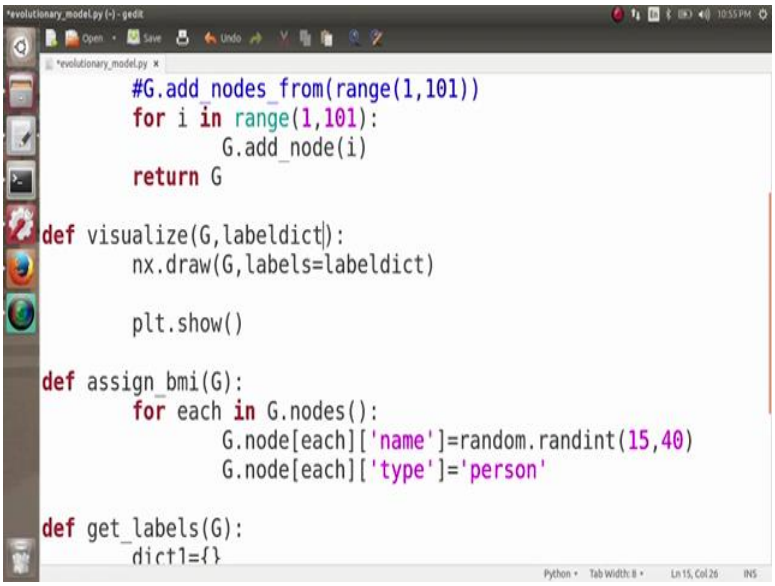


We want the dictionary here let us name this dictionary as labeldict and to get this dictionary labeldict we use a function get labels from G and this is not an implicit function get labels we need to define these function. So, what do we do? We define this function get underscore labels G and what this function is going to do first of all we create a dictionary we have a dictionary here let us name it as dict to 1 and what will add to this dictionary is the label for every node.

Hence for each in G dot nodes what we are going to do is dict1 and the value in the value for each, so each is the key here, dict to 1 of each. So, each is am node here dict1 of each should be equal to, what it should be equal to the BMI of this person, the BMI of each. So, how to access the BMI of each BMI is are attribute we do G.node each under BMI was stored in the attribute named name and at the end we return our dictionary dict to 1.

So, here you see we get a dictionary labeled dict equals to get underscore labels G and this dictionary has all the labels. Now to see these labels in our graph we look at this function visualize G, we go above will see this function visualized G. So, it was having nx dot draw G we need to add a small code here, we want the labels with the each node.

(Refer Slide Time: 11:16)

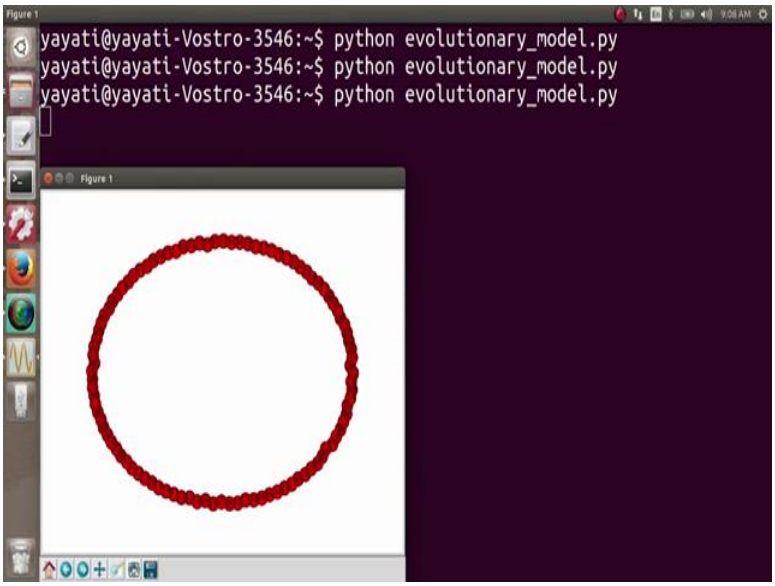


So, we add labels = and what should be the labels = a dictionary and what was the dictionary label dict. Do you see a problem here? I will see a problem here, we are using labels = labeldict how does my code node what is, does my code node what is label dict. So, you need to pass labeldict as an argument here. So, we pass this labeldict as an argument here and similarly here while visualizing the graph G we need to pass labeldict here. So, I hope that the code is clear do you.

So, I will just quickly recap what did we do is after assigning the BMI we created a dictionary the name of the dictionary is labeldict and what is the value what is this dictionary carrying is this dictionary is having one key for each of the nodes in the network and the value corresponding to that key, corresponding to that node is the BMI for that node and then we written this dictionary dict1 and its collected in label dict. Now we have a dictionary for all the labels in the graph to do visualize the graph we pass this labeled dict and here by visualizing we have an extra parameter here labels which is equal to label dict.

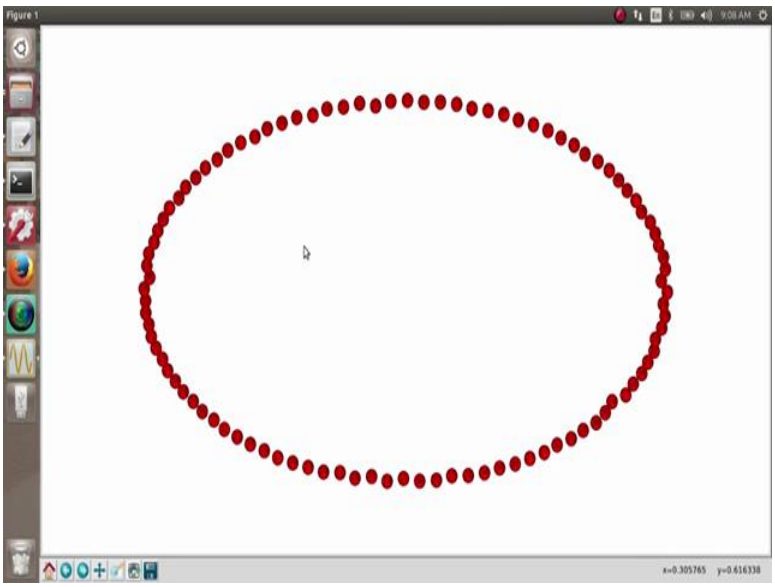
So, now we execute this code and see our graph. So, I execute our code and you see the graph here.

(Refer Slide Time: 12:42)



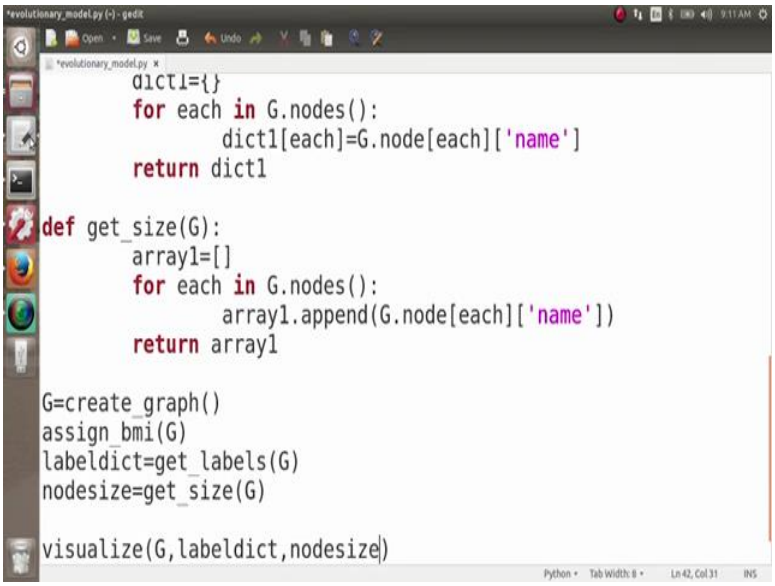
So, when we look here at this graph you can see that every node is marked with the label and the label is its BMI.

(Refer Slide Time: 12:51)



So, these are 100 nodes if you look at the labels for all these nodes the label the range from BMI that is from 15 to 40 and then you can also see that there are two nodes it is having a label 31 and this is also label 31. So, these two nodes they are having the same BMI. So, here we get the graph where we get different people and every person is labeled with his or her own BMI.

(Refer Slide Time: 13:29)

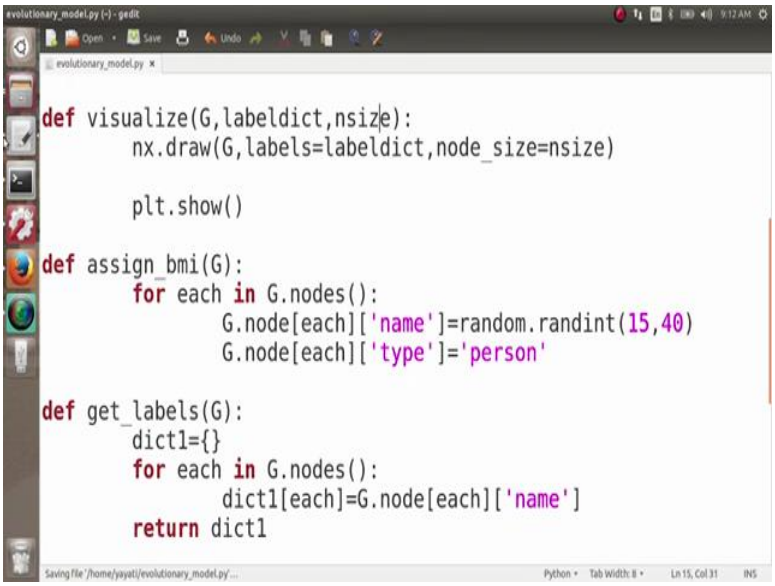


Next what am I interested in, what I want in the graph. Suppose that we want to see this difference in the form of the node size that is I want a person having a higher BMI to appear larger as compared to a person who is having a lower BMI. So, what do we do for that is I want to assign a size to every node and the size of the node should be equal to or proportional to the size of a node should be proportional do its BMI? So, what do we do for that is, I want an array here and this array consist of, this array consists of these sizes of different node? So, I take an array here node size and the value of this node size is to get the value I call a function get underscore size G.

And again, this function gets underscore size we have to define. So, we define this function here - define get underscore size G and what this function should output is an array of the sizes of different nodes whatever size we want to keep in the graph. I create an array here let us say array one and then for each in G.nodes. So, I put an iterator for all the nodes in the graph for each in G dot nodes what do we do? We do array one dot append and what do we append in this array in this array we append the BMI value extract that node and as we know how do we extract the BMI value of a node is G dot node each and then we have here name. This gives us the BMI value of each node and at the end we return array1. So, we get here and array node size which has the size of different nodes.

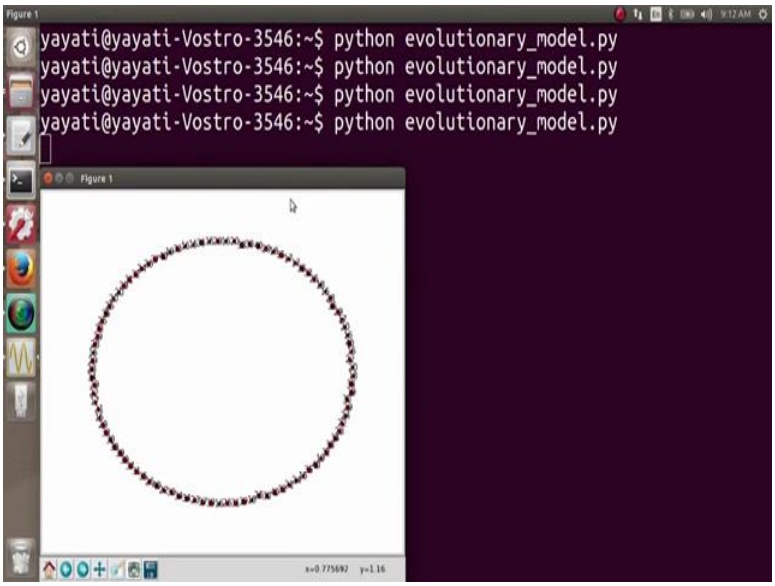
We want to see it in the graph. So, to see it in the graph we first of all pass it in the function visualize, so in the function visualize we also pass here the array node size. And then when we are visualizing the graph what I do here is I include a new parameter which says node underscore size. So, and the value for every node is to be extracted from the array node size and also, we need to pass this array here. So, let me pass node size here.

(Refer Slide Time: 15:42)

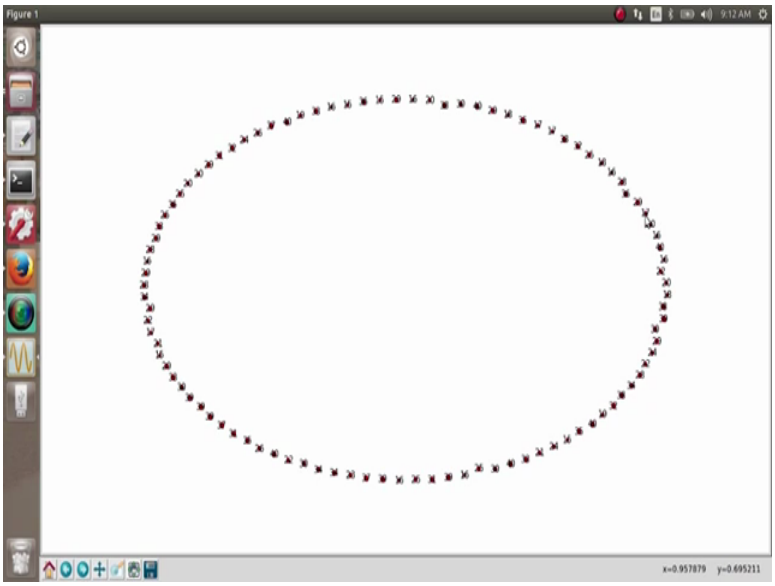


You can also give different names for like we pass there the argument node size. So, here I can actually keep it nsize and at the end I pass here n size. Just doing it this way for the sake of clarity, we can also have, we can keep the same name for the argument and we parameters and we can also give different names. Let us go back execute this code and see.

(Refer Slide Time: 16:28)

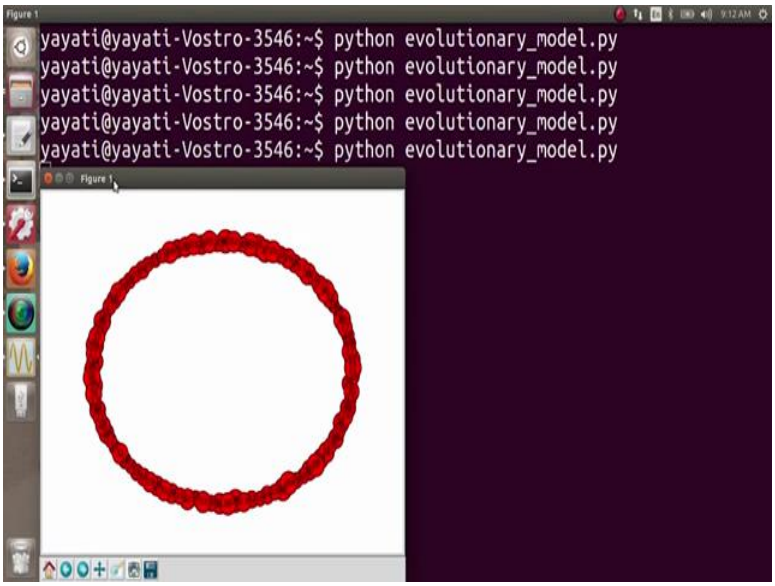


(Refer Slide Time: 16:33)



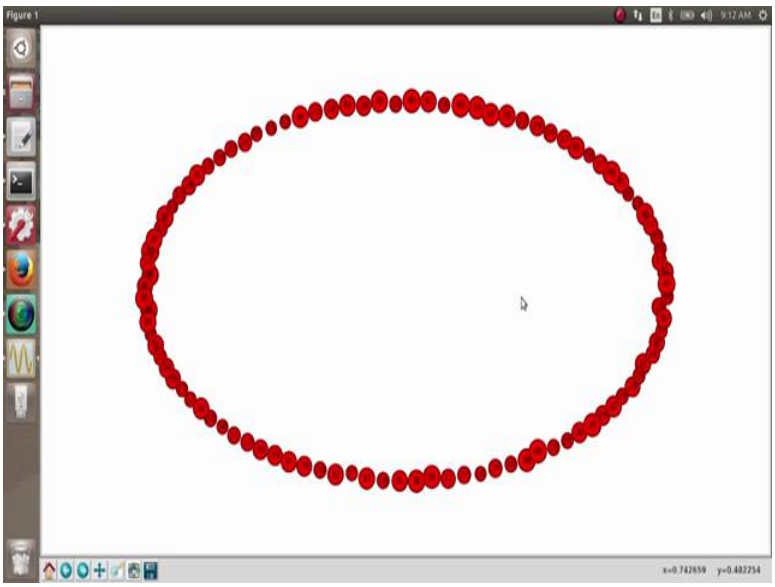
Let us execute this code and what we see here is the size of the nodes have changed, but we are not being able to. So, you can see here that some nodes are of smaller size some nodes are of larger size, but still the difference is not very clear the graph does not look very need. So, what I do is I go back to the code and for and for each value way wherever we are collecting the size here. So, for each value what I do? I multiply it by 20. So, I scale the value for the size of every node by 20. So, that the graph looks cleaner.

(Refer Slide Time: 17:10)



And then we go back and then we execute this code. So, you can see here this looks more beautiful from the previous graph.

(Refer Slide Time: 17:14)

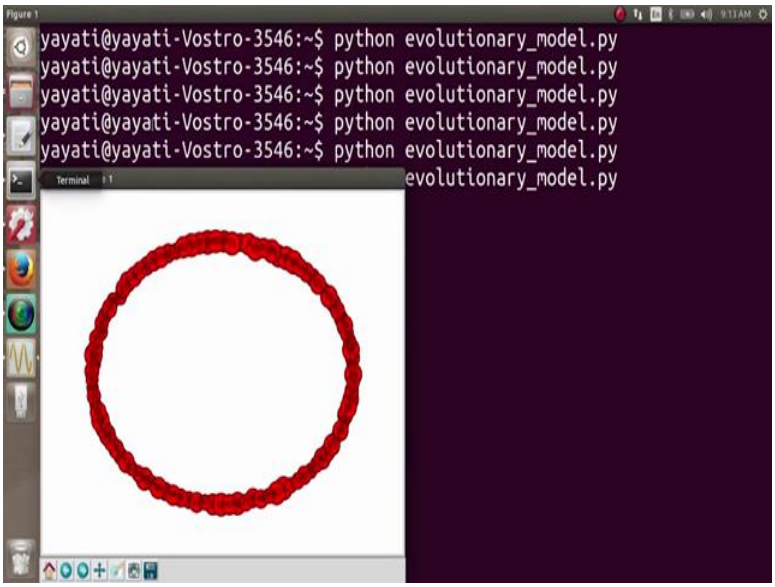


So, you can say here the nodes which are having a high BMI such as 40 they appear in the biggest; they appeared biggest in size and the nodes which are having a lower BMI such as 15 they appear smaller in size.

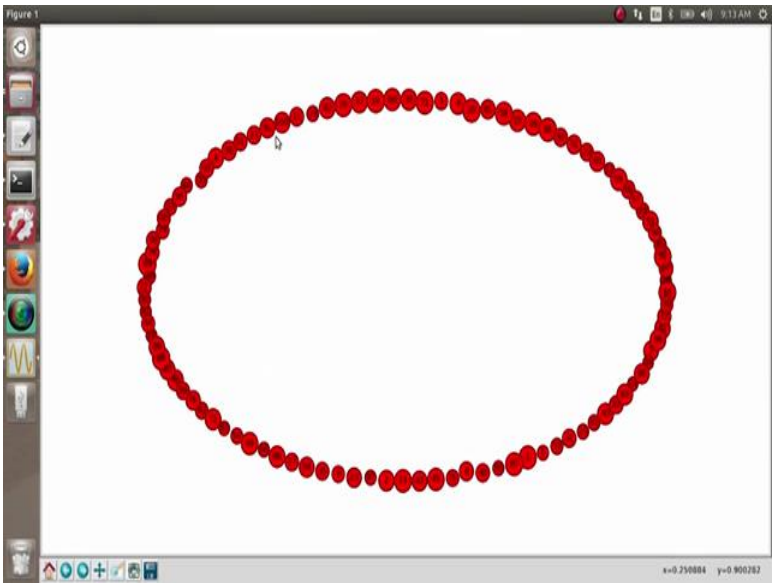
So, till now we have just created a network on 100 people where every person has a different BMI and the size of every person is proportional to his or her BMI when we

visualize this network. So, for the time being what I want to do is while visualizing my graph I will now remove the labels. So, these are the labels which shows the BMI in our graph, I will just I will remove the labels.

(Refer Slide Time: 18:04)



(Refer Slide Time: 18:07)

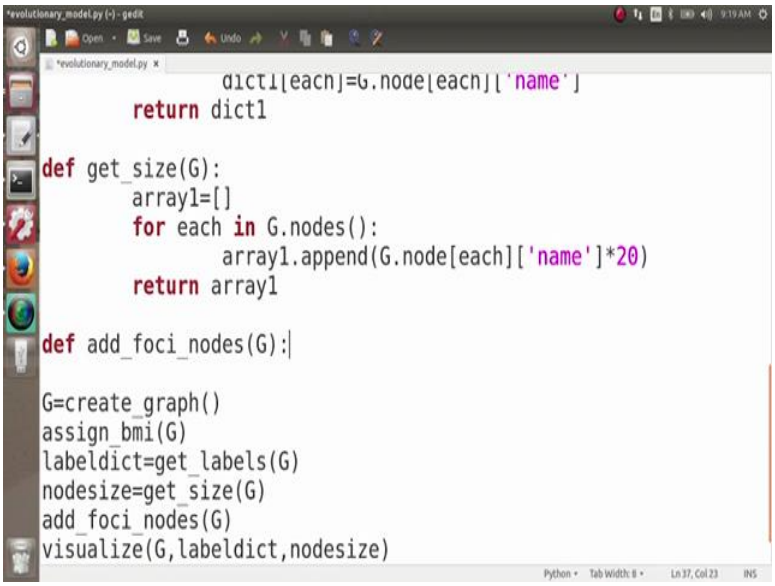


So, here we say that the node 10. So, can see that the node 10 is of the is having a very high BMI, node 87, node 13 these are all having very high BMI and the nodes like 83, 64, 66 these are having smaller BMI.

Social Networks
Prof. S. R. S. Iyengar
Department of Computer Science
Indian Institute of Technology, Ropar

Lecture - 48
Strong and Weak Relationships (Continued) and Homophily
Fatman Evolutionary Model - The Base Code (Adding Social Foci)

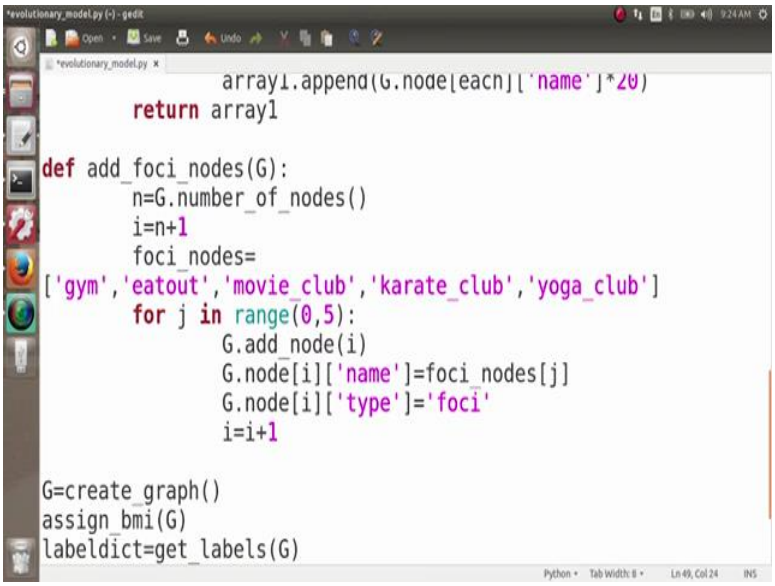
(Refer Slide Time: 00:05)



Next, what we want to do is; we want to add foci_nodes here. So, we have now all the people node here, we want different-different foci like the gym, eat out, movies, yoga etcetera. So, let us see; how do we add foci_nodes here, so as before we are writing a function for everything. So, I create a new function here and I name the function as at foci_nodes and I pass my graph G here. So, in my graph G, I have to add foci_nodes, I define here; define add foci_nodes and I pass the function G here. Now, what I have to do is as discussed before we have 5 foci, so we have to create 5 new nodes in this graph. There, the simplest way to do is we know that the nodes in our graph till now, are numbered from 1 to 100.

So, we create 5 nodes; 101, 102, 103, 104 and 105, but I do not want to do it this way, what if my graph was having 200 nodes or 300 nodes. I might not know; what is the number of nodes in my graph and how should I proceed.

(Refer Slide Time: 01:18)



So, what will we do it; we will get an iterator, let us say i or let us say a number n. So what is n? N is G.number_of_nodes. So, it returns as the number of nodes in the graph, so the number of nodes was 100. So, enhance this value 100 unit and now we want the new node to get started from 101, it means that we have to start adding nodes from the number n plus 1.

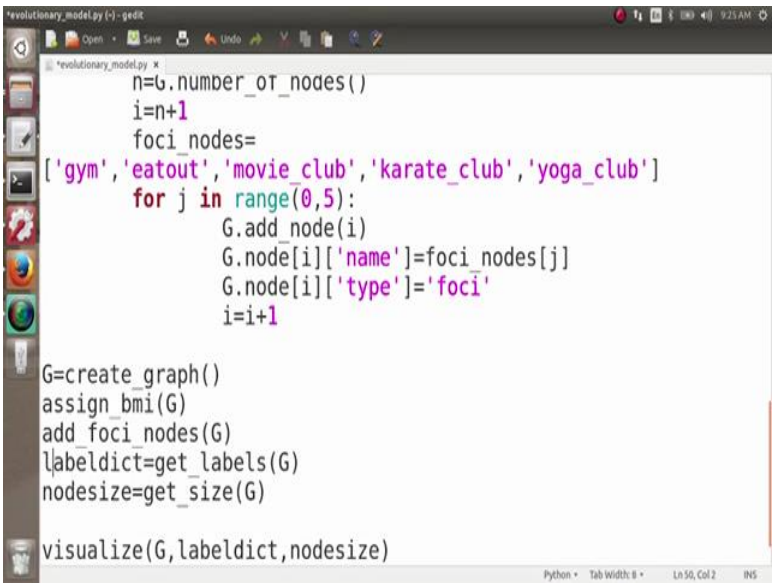
Now, you might want to note here; here the numbering of our nodes, it started from 1 to 100. If the numbering of the nodes was from 0 to 100; sorry was from 0 to 99, so still these were the 100 nodes, but the numbering and deduct 99. So, there we would have to start adding the nodes from the number n itself because n is 100 but provided now, we have the nodes from number 1 to 100. We have to start creating the new nodes from the number n + 1; so, I take the value of i = n + 1.

Next, I create an array for all of my foci_nodes; so I say foci_nodes equals to and this is an array and what all values this array has, it has a gym and then it has eat out and then it has a movie club and then it has a karate club and it also has a yoga club, so you have all these value in our foci_nodes. Now, I want to add 5 nodes in my graph; what we can do is four let us take a new variable; j in range 0 to 5, so it goes from 0 to 4. So, for j in range 0 to 5; what we have to do is, G dot add node and what should be the index of this node, index of this node should be i and then we increment i with i + 1. So, these 2 lines 0 to add node i and i = i + 1; add a new node in the graph.

Now, this new node as before should have 2 properties; the first property is the name which is the attribute of this node, which will tell you which of these 5 social foci it is. So, you see here now; so, to maintain a consistency in the person node also we have taken this attribute to be named. So when you look at a person, the main feature of a person is; his BMI which we depict as name and if you look at a social foci node its main feature is what so foci node it is; that is name. So, that is why we have used the attribute name in both the cases, so we need here 2 attributes; one is the name, and another is the type; so, that we can distinguish it from the person node.

So, what we do is; G.node[i] and what should be its name; its name should be chosen from our array. So, its name should be foci_nodes and we pass the value j here because j ranges from 0 to 4. So, it gets a name here and then we have to add a type; we do G.node[i] and then we have a type and this type should be equal to foci node, that it tell us that it is a foci node.

(Refer Slide Time: 05:57)

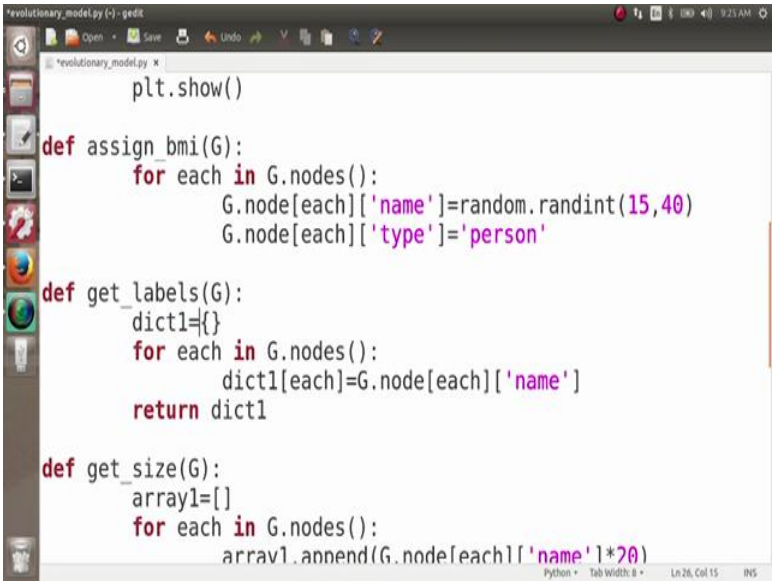


So, we have added 5 foci_nodes to our graph G and then for visualizing it as you see that; so, one thing we have to take care where we have to keep this function call is should be after assigning BMI.

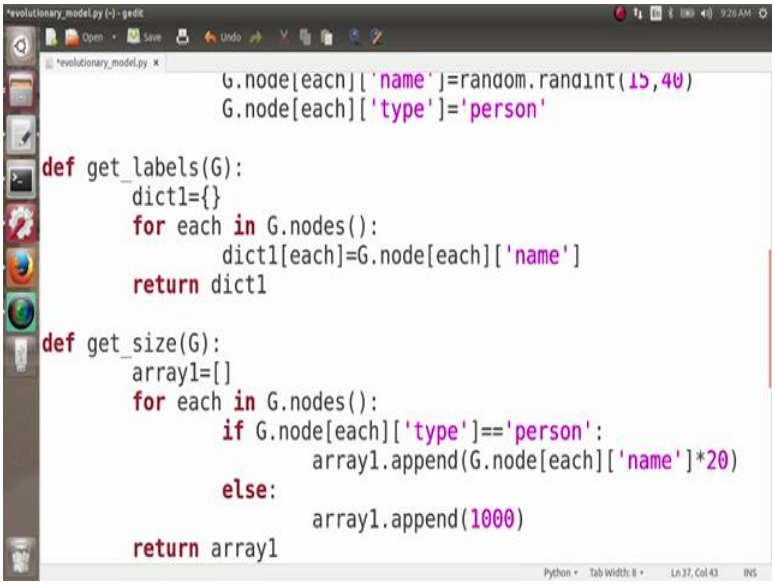
So, once we assign BMIs to the people then we add the foci_nodes, then next we get the labels and as we know labels for every node was its attribute name; so, we need not do any change here. Another was this function get under score size, so here we will have to

do a little bit of modification. So, if you see at this function get labels; we do not need any modification here.

(Refer Slide Time: 06:31)



(Refer Slide Time: 06:38)

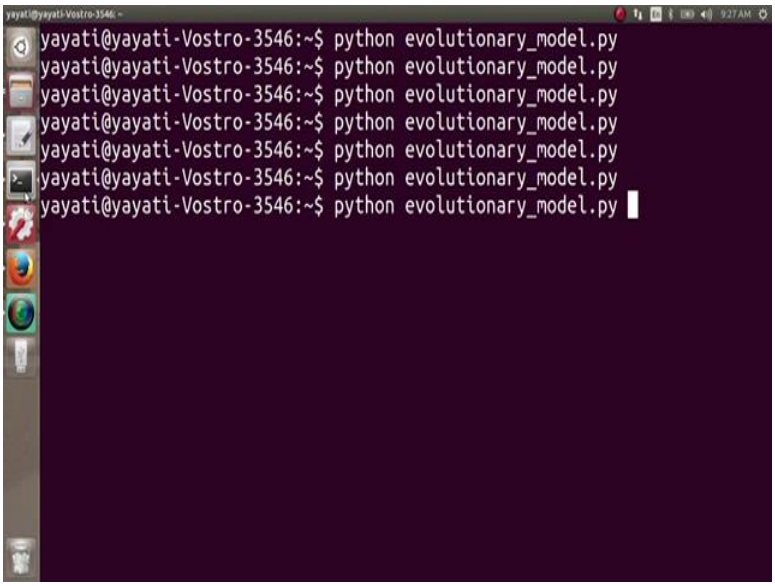


Because it is simply returning the name of every node, which is well defined for a person node as well as for a foci node, but if we look at these get size here. So, as we know that in our graph now, if we look at this name attribute for nodes. So, for the people node this name attribute returns the BMI, which is a number.

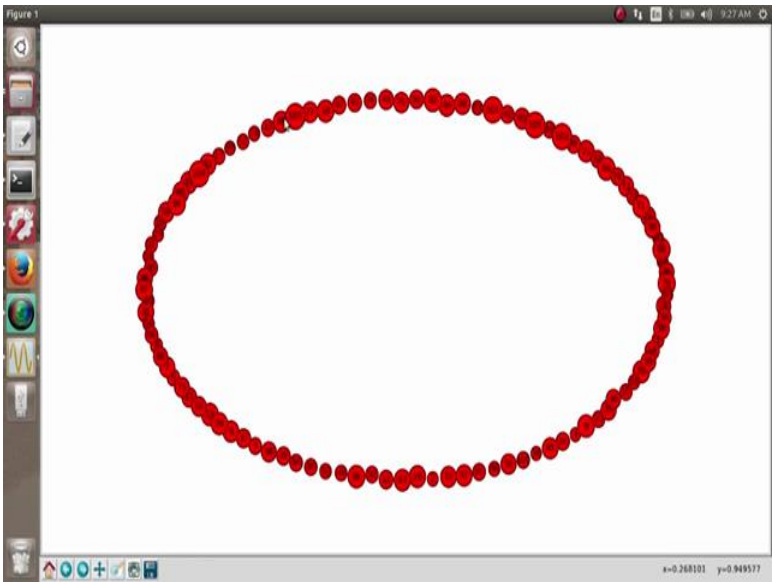
But, if we look at the social foci node; its name attribute is a string and multiplying the string by a 20 or using it in the array of size makes no sense. So, what we will do it; for each in G.nodes, if G dot each; if G.node[each][‘type’]. So, if its type == person; then what we do is, we go ahead and have this value, but else if its type is not of person; it is a foci node. Let us have some well-defined value fix here, so what we append here; array l.append and let us append a value 1000 here.

Now, if it is a person node; the size of the node is proportional to its BMI and if it is a foci node, we have a fixed size of that node which is 1000 and as before we then visualize this graph.

(Refer Slide Time: 08:02)

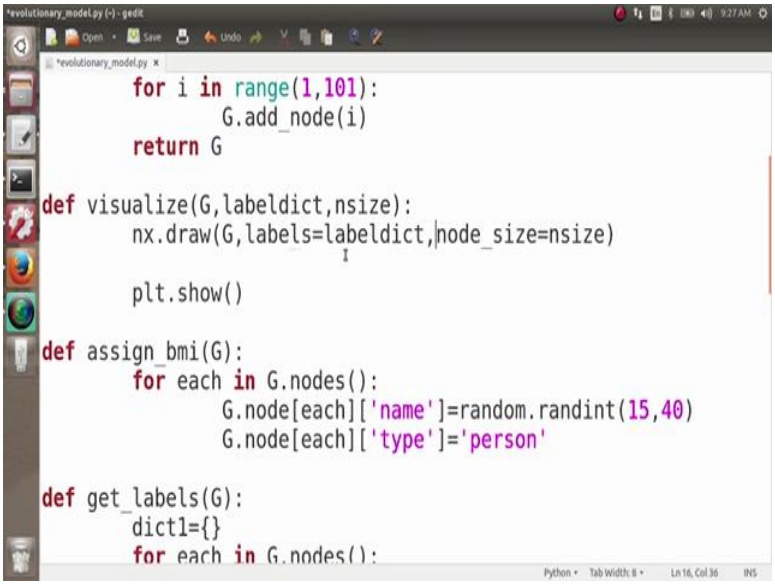


(Refer Slide Time: 08:05)



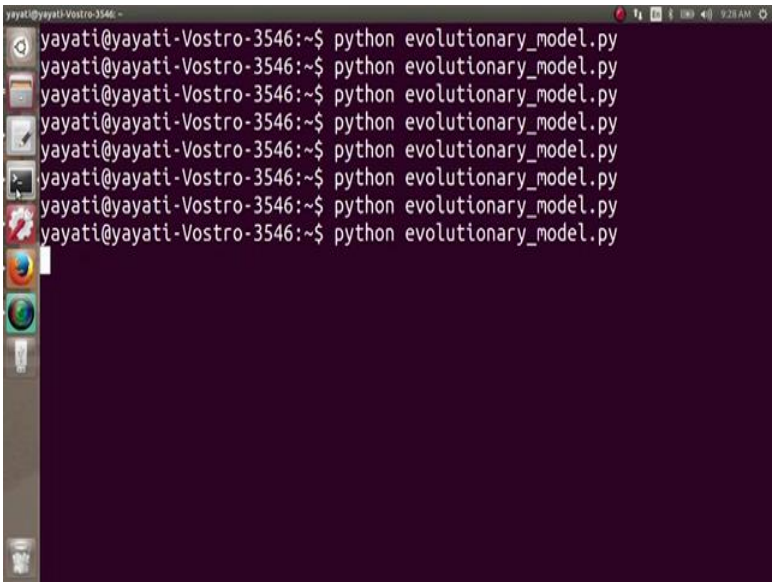
Let us execute this code, so we see this graph here; so, what is the problem here? We see all the nodes, but we do not see the names. So, what we had done previously is; we have to use this labeldict, so we were getting the label for every node; we have also passed this label and by visualizing the graph; we have actually removed that statement from here.

(Refer Slide Time: 08:28)

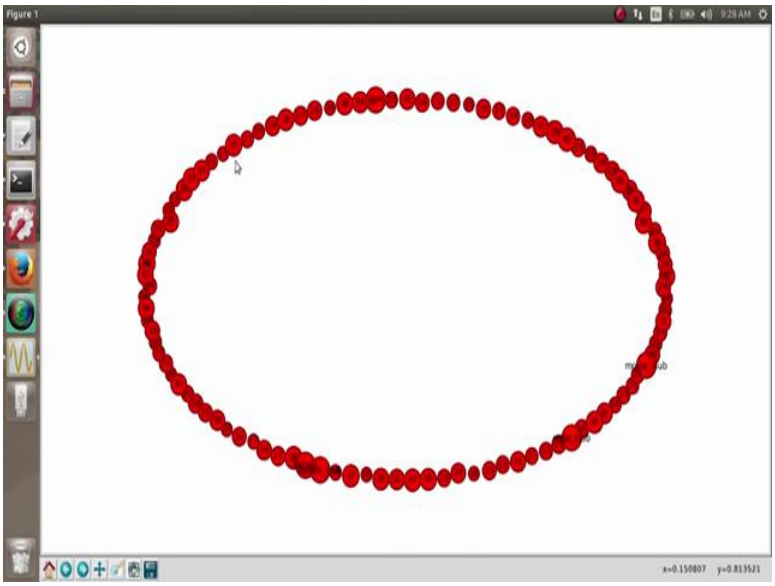


So, what we want is; we want the labels and the labels should be equal to labeldict so that we can see the labels for all the nodes, we want the labels; so, we keep = labeldict.

(Refer Slide Time: 08:47)

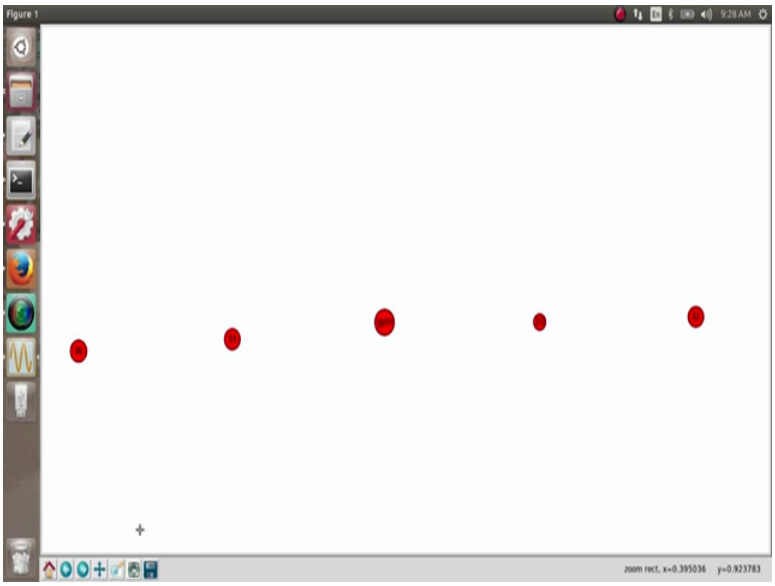


(Refer Slide Time: 08:52)

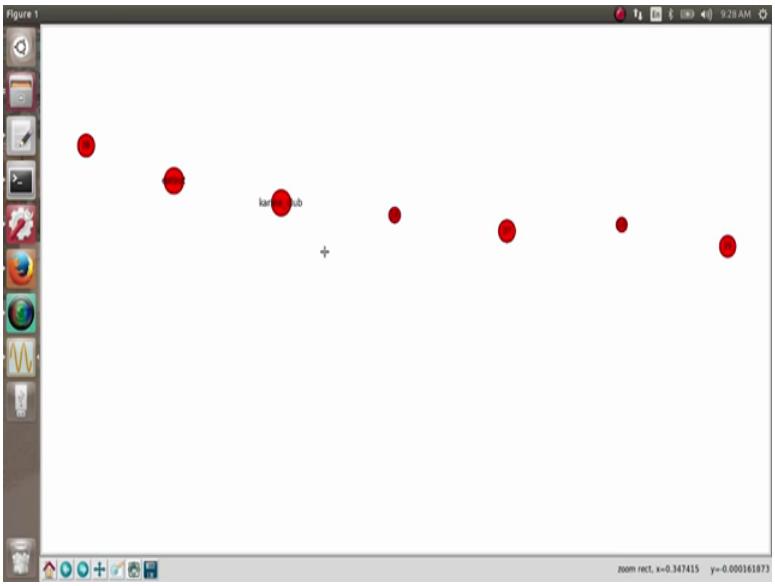


We go back then execute this code, now if we look at this graph; you can see that we have all different nodes labeled with their BMI, with their size, but here we have 5 social foci_nodes as well.

(Refer Slide Time: 09:02)

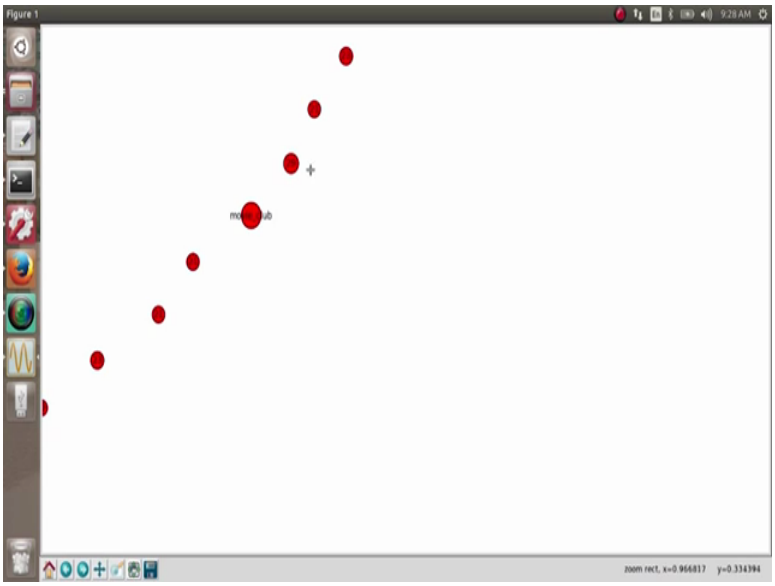


(Refer Slide Time: 09:07)



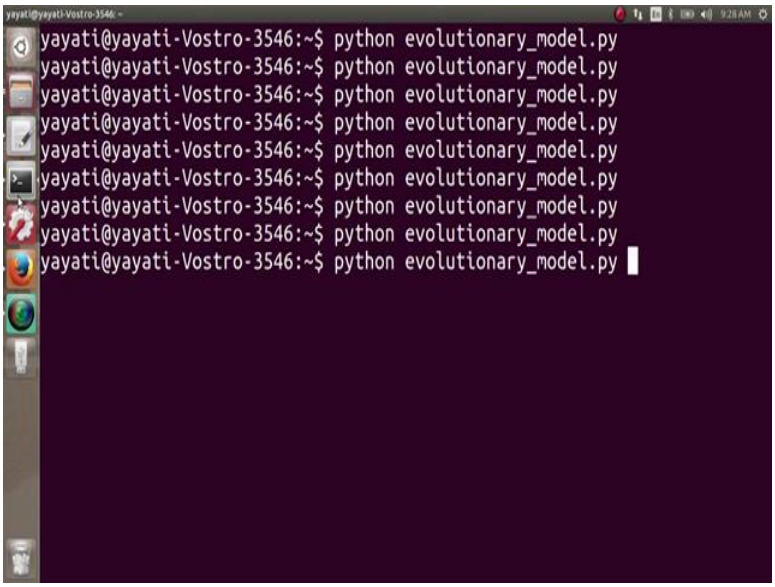
So, here we have gym node and then here we have eat out and we have karate club and here we have movie club.

(Refer Slide Time: 09:11)

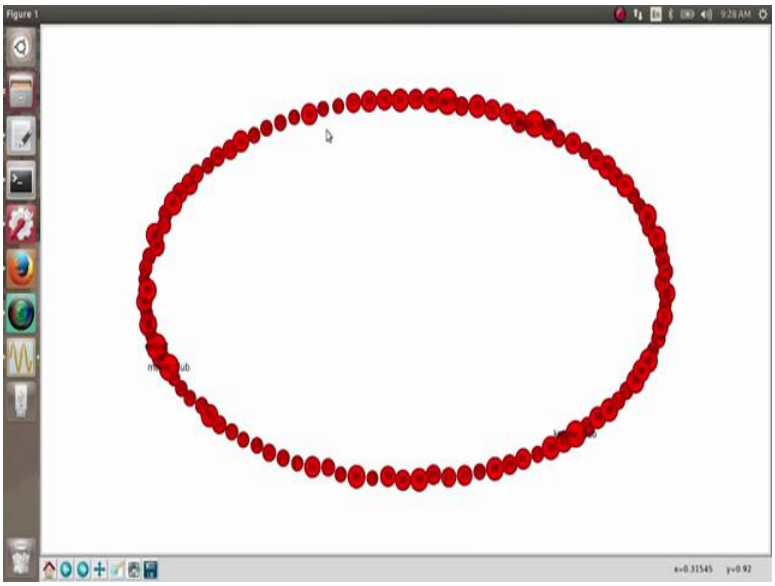


So, we have all the social foci_nodes also here in this graph; so, we have 100 nodes for people and 5 nodes for social foci. What is more interesting to do next, so something which is coming next is all the way more interesting; what do I want?

(Refer Slide Time: 09:37)

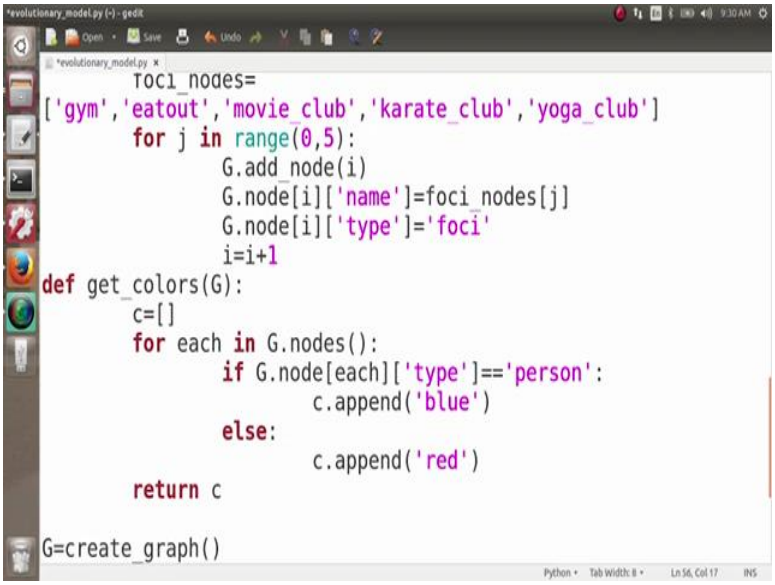


(Refer Slide Time: 09:39)



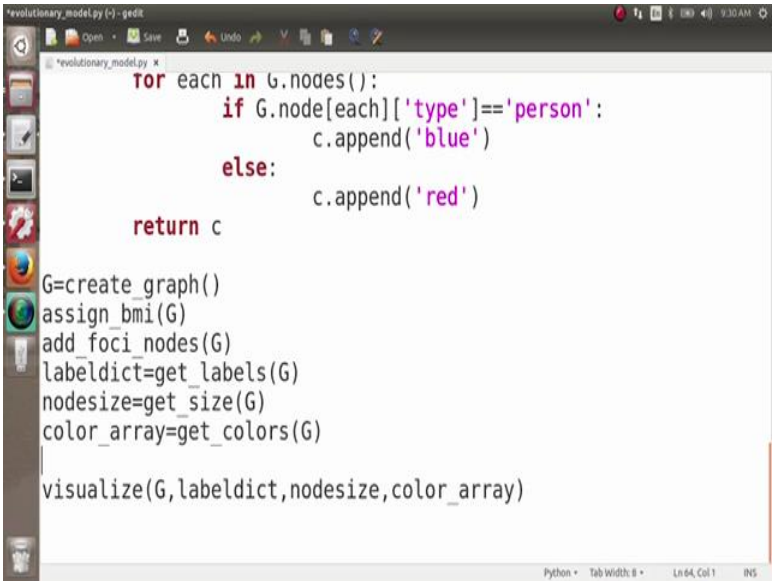
So you have seen that when we looked at this graph here; everything comes everything is in a red color. So, it makes difficult for me to identify the social foci_nodes here; I have to search for them. Next, what we will do is; we write a piece of code, so that the social foci_nodes appear in red color, but all the other nodes that is the people nodes; they appear in blue color and this is very simple. So, like we did for the labels and for the node size, we also do a similar thing.

(Refer Slide Time: 10:06)



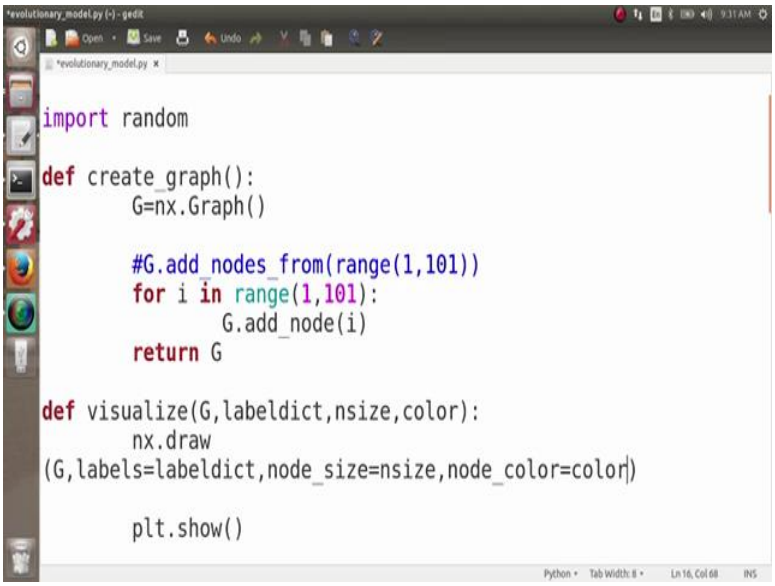
Here, we make an array; color array and get this array from the function get colors G and then we define this function get colors G here; define get colors G and what do we do here is, we have an array let us say c, then for each in G.nodes; if G.node each, type, so if its type == person, What do we do? We append a blue in this array else we append a red in this array and then we return this array.

(Refer Slide Time: 11:05)



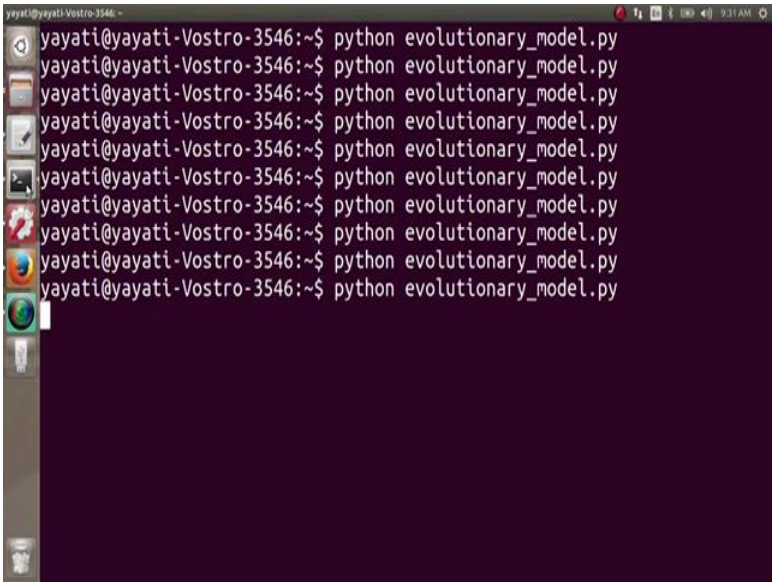
So, we get here color array, so this color array has a blue color if a node is a person and it has a red color, if the node is a social foci and what do we do here is we pass this color array here and the visualize function.

(Refer Slide Time: 11:27)

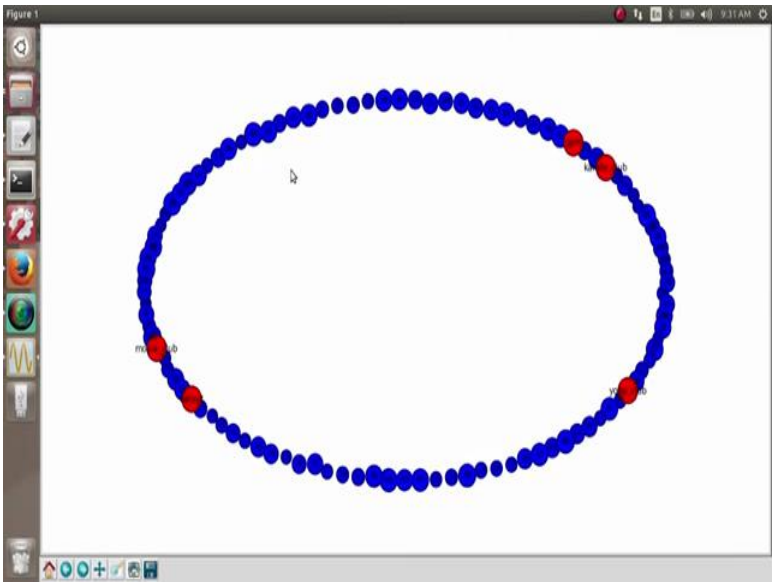


We go back to the definition of the visualize function; we have an extra array color here let us say color. So, what we do here is we add one more parameter which is node underscore color and from where every node takes its color is our array color, so we are done; go back.

(Refer Slide Time: 11:45)

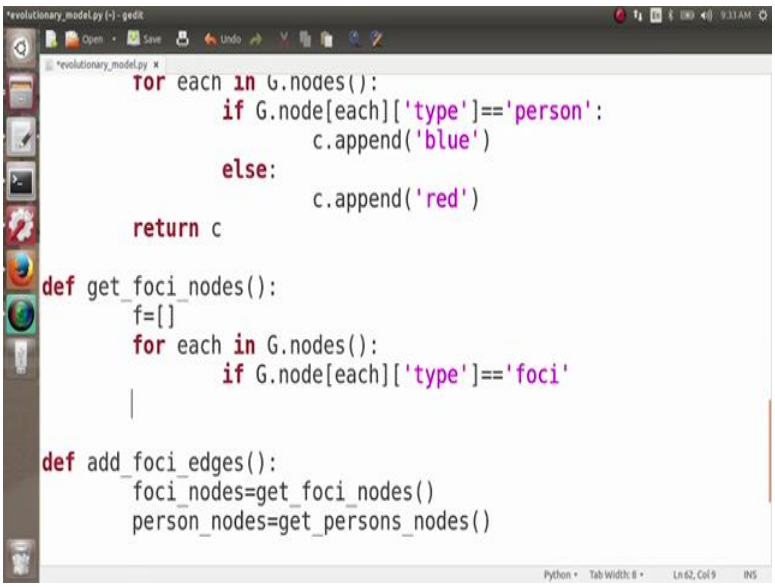


(Refer Slide Time: 11:51)



Execute our code and you can see now; every social foci node comes in a red color and all the people node, it comes in a blue color; so far, so good. Now, we have a city and the city has 100 people and the city has 5 social foci. Next, what we want to do is in the beginning every person should be a part of exactly one social foci. So, now we must add edges between the people and the social foci.

(Refer Slide Time: 12:26)



So, again we write a function for that and let the name of the function be add_foci_edges. So, we want to add some edges from every person to a social foci; so,

we have here define add; foci edges and now what we want to do is, we chose one person one by one and from this person, we add an edge to every foci node. So, for that first of all we need all the nodes which are the foci_nodes at one place, so that we know to which nodes we have to make connections to. Although, we know that the foci_nodes are from 101 to 105, but that might not be the case with every graph given to you. We have made this graph this way, so let us try to get an array which consists of all the nodes, which are the foci_nodes.

So, we make an array foci_nodes equals to and then and let us make an array; people node. Rather, a better approach is to use functions for both of these; so, for foci_nodes equals to get. So, the array foci_nodes = get_foci_nodes and the array person_nodes = get_persons_nodes and then we quickly define both of this functions here; define get_foci_nodes. So, what do we do is; we take an array f and then few of we want to make the foci_nodes for each in G.nodes; if G.node each and if its type is equal to equal to what? If its type == ‘foci’, then what do we do?

(Refer Slide Time: 14:42)

```
                        c.append('red')
        return c

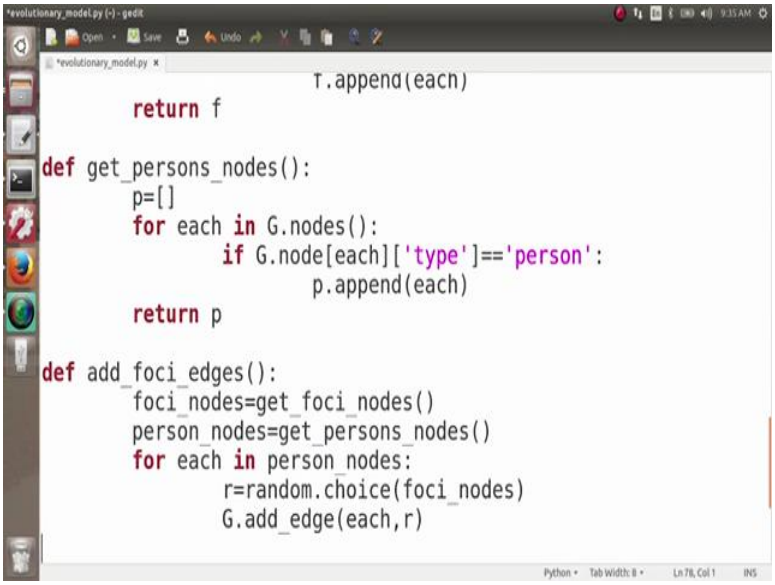
def get_foci_nodes():
        f=[]
        for each in G.nodes():
                if G.node[each]['type']=='foci':
                        f.append(each)
        return f

def get_persons_nodes():
        p=[]
        for each in G.nodes():
                if G.node[each]['type']=='person':
                        p.append(each)
        return p
```

Then what do we do is; f.append; f.append each and then you return f. So, this particular function will return you an array or foci_nodes and we do exactly the same thing for the person’s nodes. So, instead of writing the code again; let us just copy paste this code and make the changes. So, get_foci_nodes and here we have get persons nodes and what we do not want to do here is let this array be p. So, for each in G.nodes, if

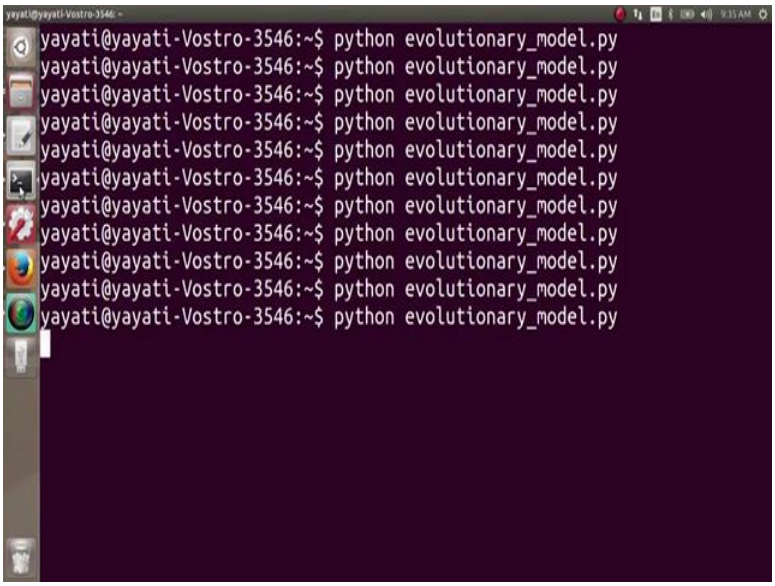
G.node[each][‘type’] equals to equals to and we have here person, then we append this in array p and then we return p.

(Refer Slide Time: 15:37)

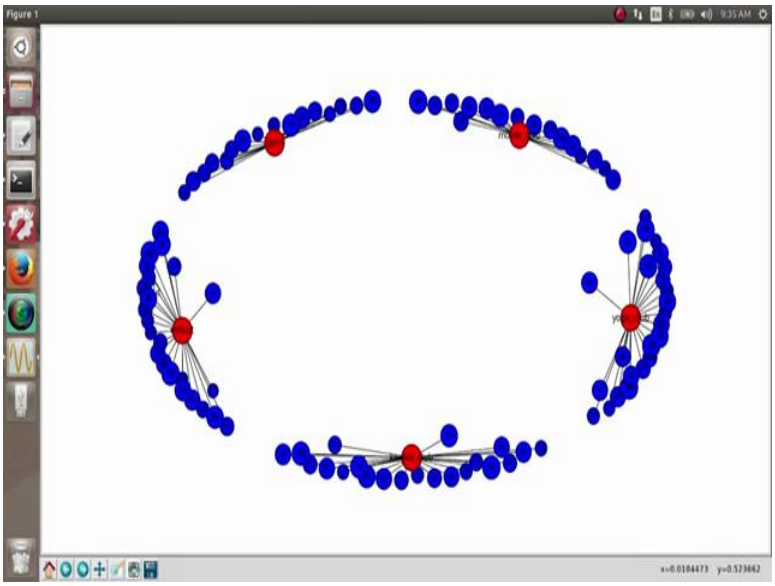


So, here we have got an array one is for the foci_nodes and another is for the person’s node. Now, what we want to do is add an edge, so what do we do for adding an edge is for each in and we want to add an edge from every person nodes. So, for each in person nodes, what we do is, we choose a random node from the foci_nodes. So, r = random.choice, so random.choice is used to get a random value from an array; r equals to random dot choice from foci_nodes and then what we do is G.add_edge and we add edge from each to r. So, for every node in the person’s node, we choose a node randomly from the foci_nodes and add an edge between the 2.

(Refer Slide Time: 16:26)



(Refer Slide Time: 16:31)

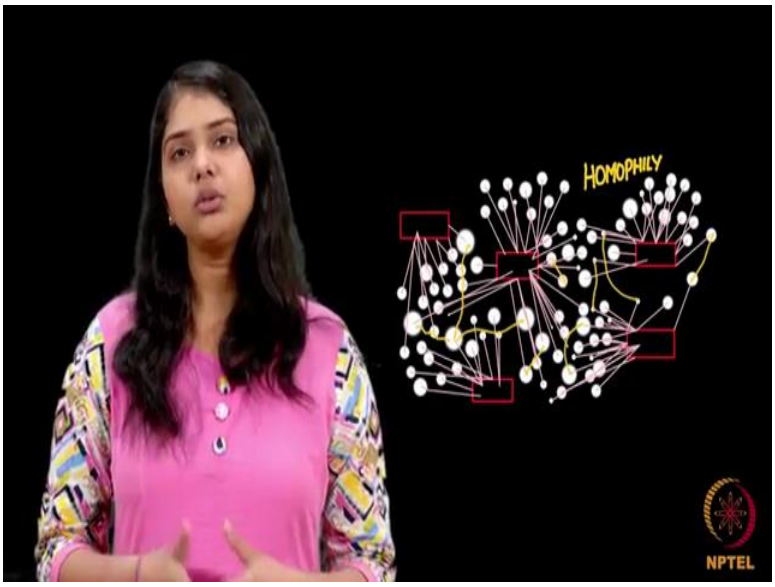


So, let us go back and look at the output; so, you can see here what we have is; every person has chosen here one foci_nodes. So, these people here are part of gym, these people are part of movie club, these people are part of yoga club, karate club and eat out.

Social Networks
Prof. S. R. S. Iyengar
Department of Computer Science
Indian Institute of Technology, Ropar

Lecture – 49
Strong and Weak Relationships (Continued) and Homophily
Fatman Evolutionary Model - Implementing Homophily

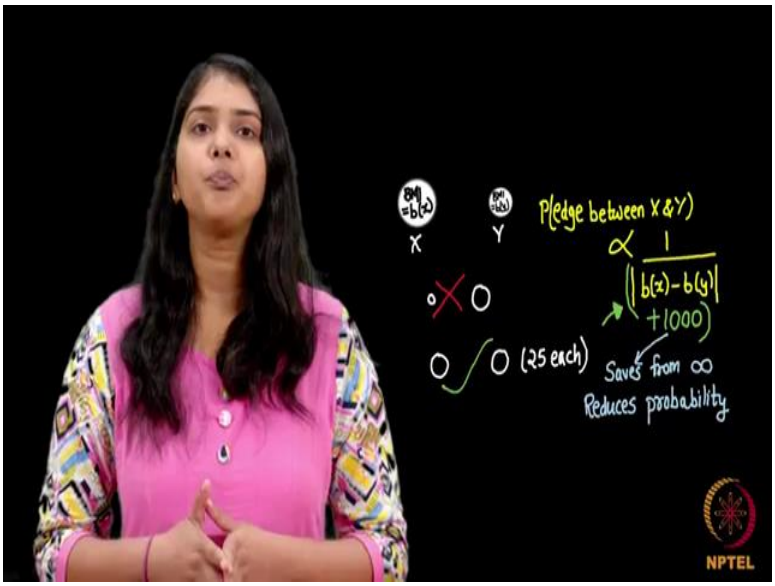
(Refer Slide Time: 00:10)



Till now we have made a network which has 100 people of different body weights and then we have 5 social focis including a gym and an and eat out place and we have seen that initially everybody is a part of exactly one social foci and with time people will start becoming the parts of more social foci because of membership closer.

Next what we want to implement in this coding what we want to happen in this network is homophily that is the people who are having singular BMI should become friends with each other. So, we are going to use a very simple method to do it. So, we see that the probability of two people becoming friends with each other is inversely proportional to the difference between their BMIs.

(Refer Slide Time: 00:53)



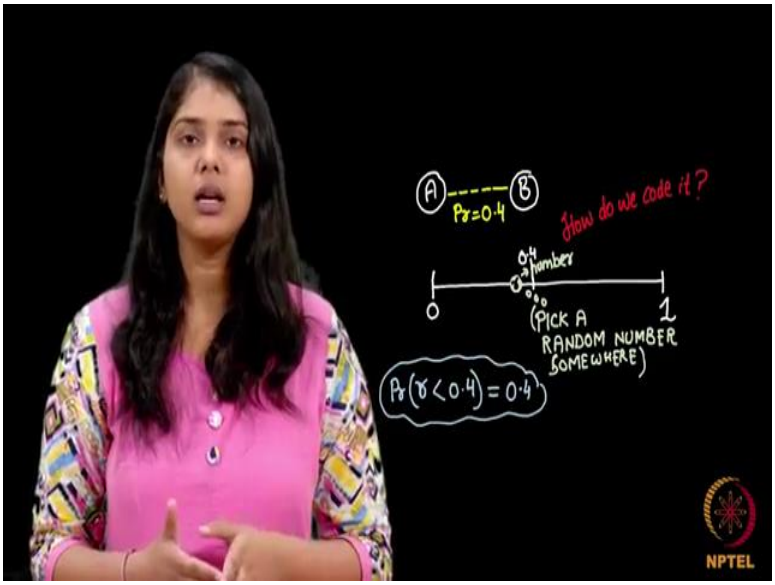
So, more is the difference between your BMIs less probable two people are to become friends with each other. So, if let say a person is underweight having a BMI of 15 and a person is overweight having a BMI of 40. So, it is very unlikely that they will become friends. And if two people they have almost the same BMI. So, let us say 25 each then they are likely to become friends. So, we can directly use a formula that probability of two people becoming friends with each other is inversely proportional to the difference between their BMIs.

So, probability of an edge between u and v is proportional to one divided by the absolute difference between them. So, we take absolute to avoid a negative there. So, we have absolute there and we add an extra factor of plus 1000 here. Why do we add an extra factor of plus thousand here is if you see by this formula if we do not have a 1000 here and let us say the BMIs of two people are equal, let us say 25 each then the probability of them becoming friends becomes 1 by 0 which is infinity, so we have a 1000 here which not only help us deal with this infinity issue what it does is it reduces all the probabilities.

So, although the probability of one person becoming connected to another is remains inversely proportional to the difference between their BMIs, but we want this probabilities to be less because since we want to see the network to be evolving we do not want a lot of edges to enter at the same time. So, adjust should come slowly and

slowly. So, this plus 1000 helps us reduced all the probability value, it scales all the probability values down. So, we use this formula for implementing homophily.

(Refer Slide Time: 03:16)



And one interesting concept here which I would like to tell you, you see here what we are going to do is the here are two people. Let us say A and B and we are saying that assume the probability with which A and B become friends is 0.4. So, there is a 40 percent chance that A and B become friends and 60 percent chance that they do not become friends. How do we implement this in our coding? So, this random package helps us do this. So, what I do is assume, I pick a random number random real number from a number line from 0 to 1 as shown here.

I want some event to happen with a probability of 0.4. So, now, I have a random number here between 0 and 1. So, I pick a random number let us say it is here, what is the probability that this random number is less than 0.4. So, since this random number can be anywhere on this number line if you see the probability that this random number is less than 0.4 is actually 0.4 itself. So, 0 to 0.4 is a smaller slot, 0.4 to 1 is a bigger slot. So, there is a lesser probability of this number falling below 0.4 and there is a higher probability it lies in the range 0.4 to 1. So, we will be using this method to implement what we are saying. So, assume there are two nodes u and v and we want them to become friends with probability 40 percent 0.4. What will do is generate this random

number are from 0 to 1 and if its value is less than 0.4, we make an edge between these two people otherwise we do not make an edge.

(Refer Slide Time: 04:55)



So, now before implementing homophily I want to do one extra thing. So, we have added colour to a node. So, we have added blue colour for the person node and red colour for the foci nodes.

(Refer Slide Time: 05:05)

```
                i=i+1
def get_colors(G):
        c=[]
        for each in G.nodes():
                if G.node[each]['type']=='person':
                        if G.node[each]['name']==15:
                                c.append('green')
                        elif G.node[each]['name']==40:
                                c.append('yellow')
                        else:
                                c.append('blue')
                else:
                        c.append('red')
        return c

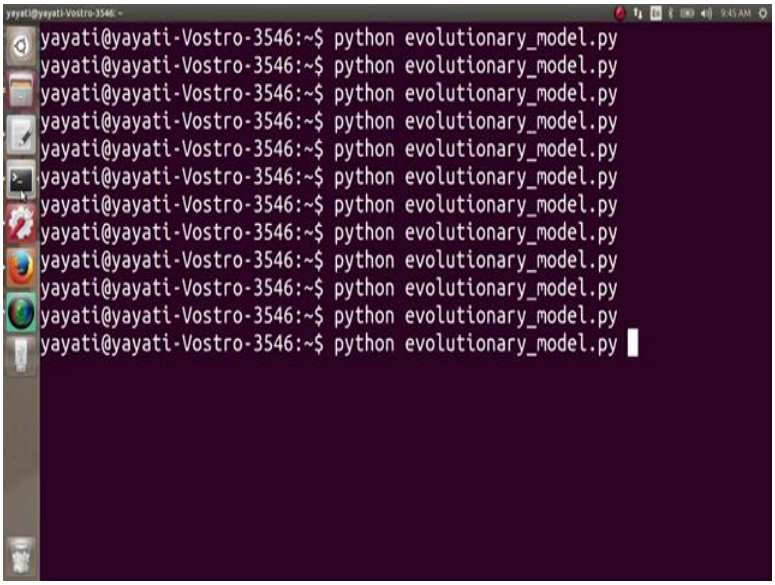
def get_foci_nodes():
        f=[]
```

What I want to see I want to see underweight and overweight people separately, I want the underweight people to appear in let say green colour and overweight people to appear

in yellow colour, so that when we look at this graph across different timestamps we can see that how the number of people are changing and how the clustering between them is happening.

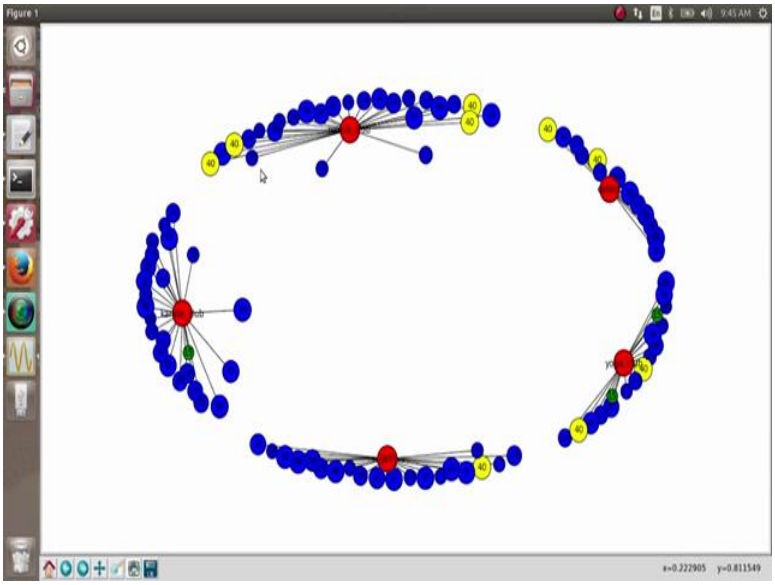
So, what we do here is if G node each type equals to equals to person and then further we have if G.node each name equals to equals to 15. So, what we do is in this case c.append what do we append here is a green and then we have else if which is written as elif in python. So, elif G.node each name equals to equals to 40, if it is 40, c.append and what do we append here is a yellow colour. And still if some node is left; obviously, many nodes are left having the BMI in between these two then we do c.append(blue). Let us save it and next let us look at the output.

(Refer Slide Time: 06:55)



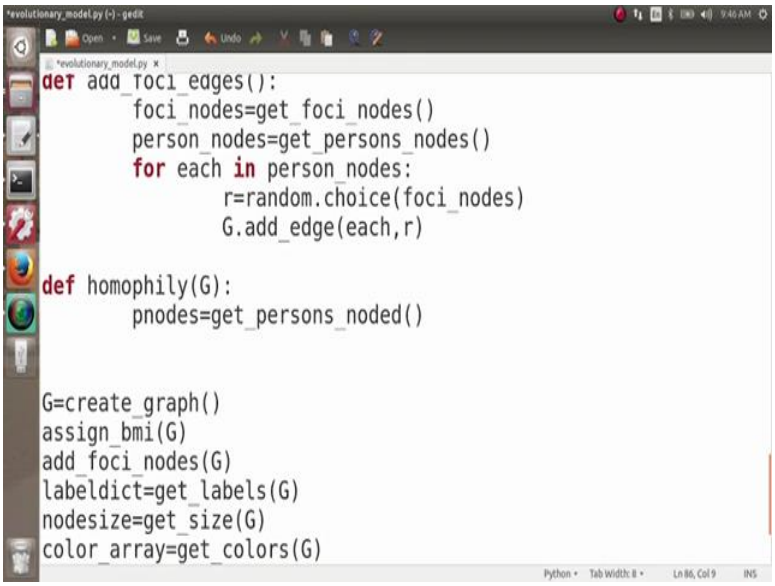
So, you see here the graph looks I think all the more beautiful now. So, we have these nodes. So, blue nodes are mostly the normal nodes red nodes are the foci nodes. So, here you see all the nodes having BMI of 40 comes in the yellow colour and all the nodes having this BMI of 15 comes in the green colour.

(Refer Slide Time: 06:59)



Next if we go head and implement homophily in this network, let us see how we implement homophily. So, after adding foci edges what we want to do if we want to implement homophily.

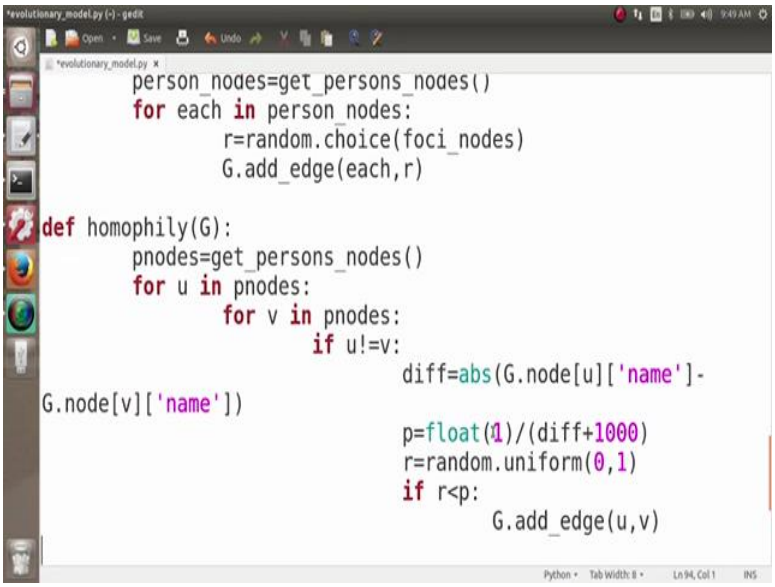
(Refer Slide Time: 07:30)



Now what does homophily do? So, what will be implementing here will be one iteration of homophily what does homophily says nodes which are alike similar to each other they tend to become friends with each other, and we have already looked at the formula for this.

So, we define homophily here, define homophily G and what does it do first of all it gets all the person nodes. So, person node equals to get_persons_nodes because you see homophilies only going to happen between the people not the social foci. So, this function is explicitly defined for the person nodes. So, we get all the person nodes and for simplicity, let us simply name this array as pnodes. pnodes refer to person nodes pnodes equals to get_persons_nodes. You can also notice here that I am nowhere pass the parameter G here because G here is being considered as a global parameter - pnodes equals to get_persons_nodes.

(Refer Slide Time: 08:57)



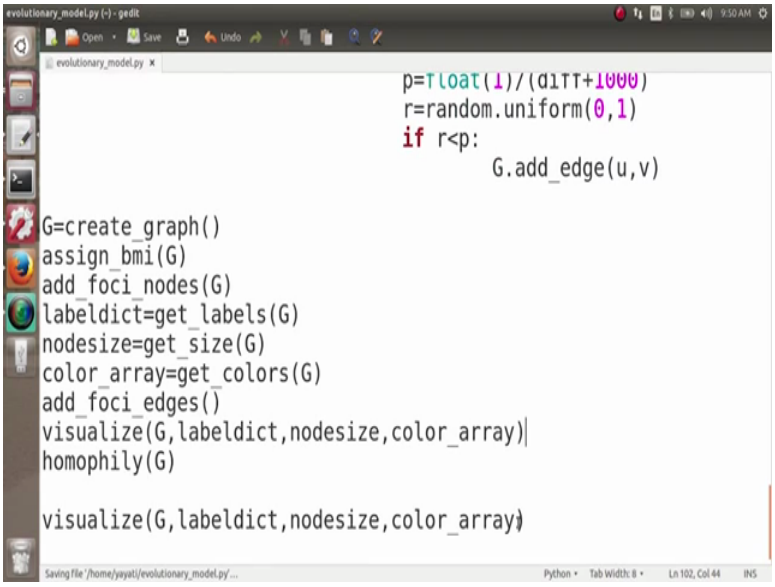
So, I have all the person nodes, now I want to see whether there BMI similar to each other and if it is similar to each other I will add edges between them what I do for u in pnodes. So, u is my first node for u in pnodes and then I have for v in pnodes, so it picks each of the person nodes every possible combination of two person nodes one by one and obviously, my u should not be equal to v because the person is not going to make tie with himself. So, if u != v what do we do is we calculate this difference in their BMI.

So, what is the difference in their BMI? It is the absolute value of, absolute value of G.node each sorry G.node[u][‘name’] which carries the BMI of this person minus G.node[v][‘name’]. So, I have this difference here, after having this difference what is the probability that these two nodes will connect to each other as we have seen is inversely proportional to this difference - greater the difference lesser the probability,

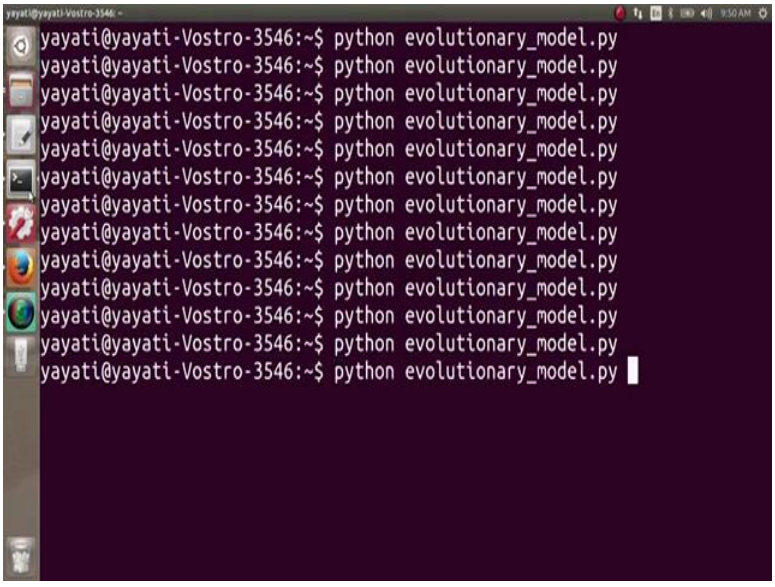
lesser the difference higher the probability. So, probability of these people connecting is one divided by this difference plus 1000. So, we have this 1000 here to keep these probabilities small.

And next as discussed before we have a random number r, random real number r which takes its value from 0 to 1 and if this r < p what do we do; if G do not add edge between u and v. I think it is clear. So, what we do is we calculate the difference between two nodes, probability of these nodes connecting is inversely proportional to this difference.

(Refer Slide Time: 11:28)

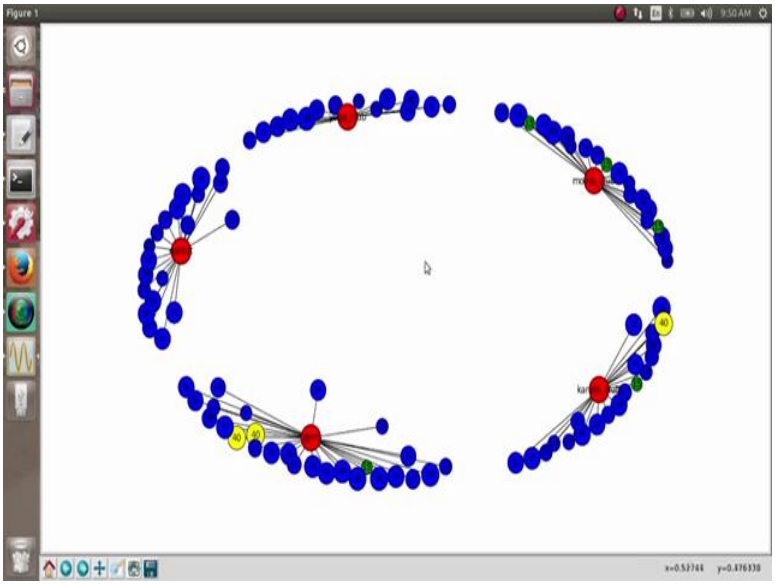


(Refer Slide Time: 11:33)



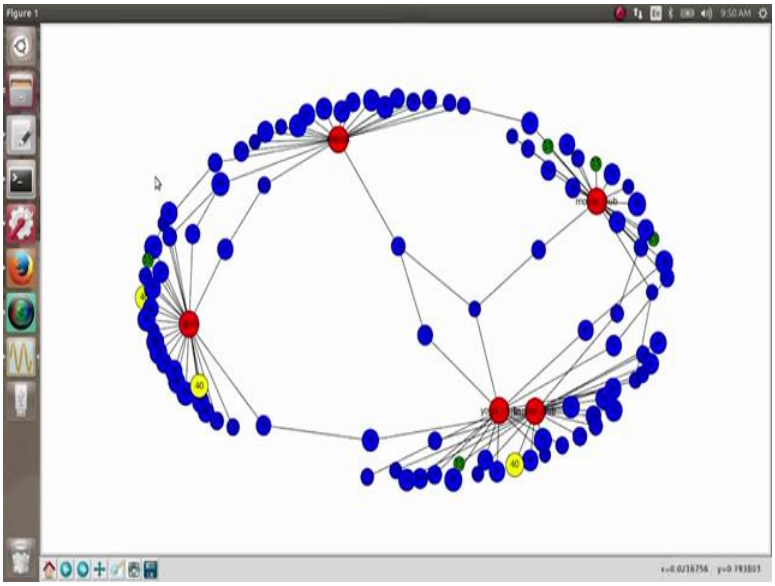
And with this probability these two nodes connect in every iteration. Let see what we want to do is let us visualize the graph once before homophily and once after homophily.

(Refer Slide Time: 11:39)



So, this is our graph before homophily, this is our graph before homophily, and this is our graph after homophily.

(Refer Slide Time: 11:45)

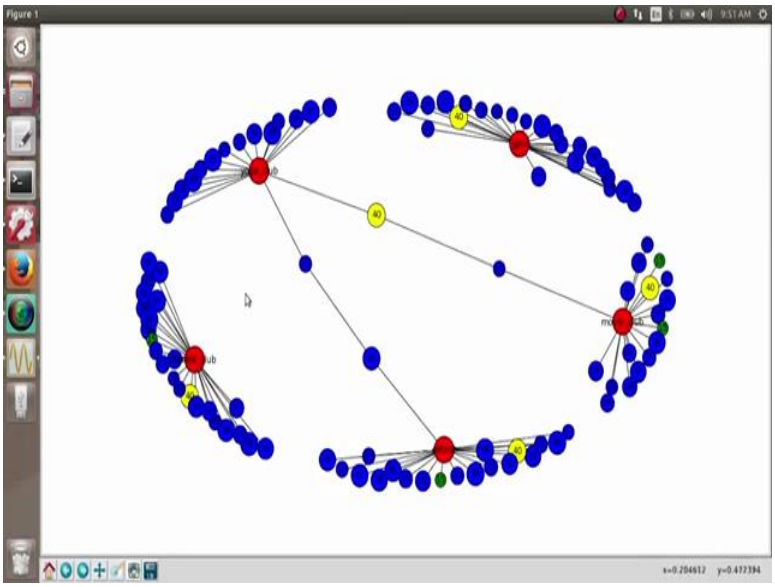


So, you can see here now we have many relationships across different clusters. So, these nodes previously they existed in different clusters, but you can see now our graph has become connected and more and more people has connected with each other. So, here

we have an edge between 35 25, 25 39, 39 27 and so on and these edges are in accordance with their body weights BMIs.

So, we can make this probability actually a little lesser if you want a smaller number of edges to be added at every step that is totally up to us. So, we can increase this probability of connection, we can decrease this probability of connection, but the probability should be inversely proportional. So, if you want to decrease the number of edges being added at every step what we can do is we can make this value all the higher and this probability gets less here. So, you see here that this was our initial network.

(Refer Slide Time: 12:51)

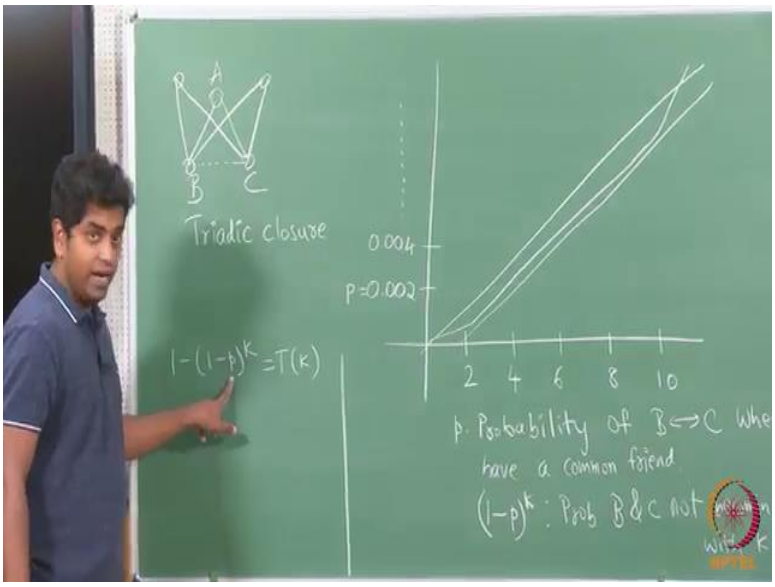


And this is our new network. So, you see here a smaller number of edges have been added, a smaller number of edges have been added it here.

Social Networks
Prof. S. R. S. Iyengar
Department of Computer Science
Indian Institute of Technology, Ropar

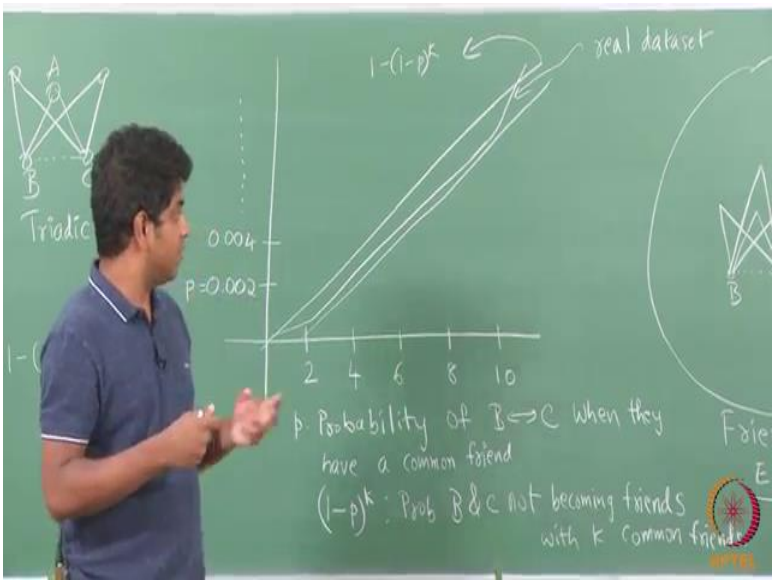
Lecture – 50
Strong and Weak Relationships (Continued) and Homophily
Quantifying the Effect of Triadic Closure

(Refer Slide Time: 00:14)



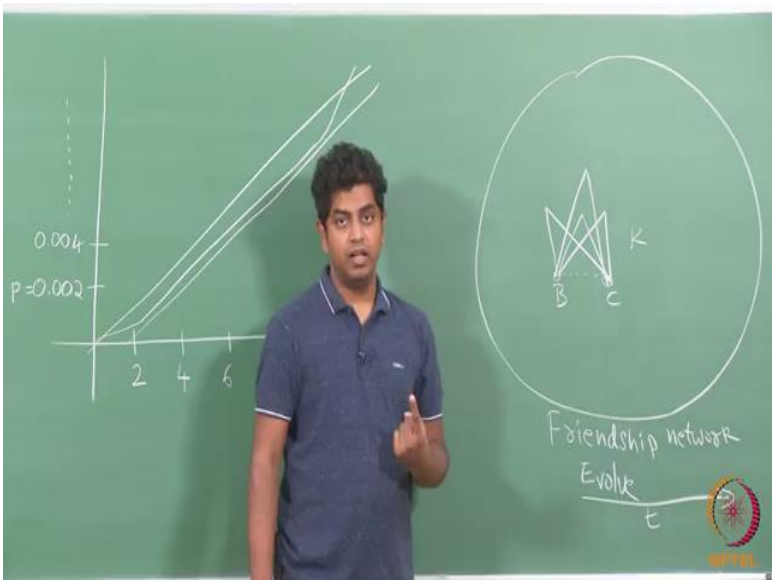
So, we saw about triadic closure where a person A has two friends namely B and C, and there is enormous pressure for B and C to become friends, this is called the Triadic closure perfect. So, now, as you can see if B and C have a common friend there is pressure for them to become friends. As they say a friends friend is a prospective friend right now what if there are many such common friends do you think the pressure increases for B and C to become friends, if there are a lot of common friends you just now watch the video clip which said two people are sitting and then they discussing they should probably become friends, simply because they have a whole lot of common friends.

(Refer Slide Time: 01:28)



What is this whole lot of common friends mean do you think two people becoming friends has something to do with their common friendship? There is a space of free such that answers this question, people considered the email communication network and then tried plotting the following. They said if there are two common friends 4 common friends, 6 common friends, 8 and 10 common friends let us say and if you look at the probability of them becoming friends let us say 0.002, 0.004 and so, on right you observe it is a very small probability.

(Refer Slide Time: 02:05)



So, what we do is we look at our friendship network over a timeline; friendship network you look at it evolve with time T, and then we observe that two people who have K number of friends K number of common friends

Let us say what is the probability of them becoming friends in the next time step? For example, today I look at this friendship network this B and C are not friends, but they have K common friends with them, tomorrow I will observe are they still friends may be not I keep going like this and I observe that eventually they become friends. So, now, I will consider the following, given two people B and C if they have K number of common friends and if they become friends right tomorrow then I will make a note of that and across this timeline which is sort of this friendship network which is evolving across this timeline I will try to draw this plot of what is the probability with this data I can write a plot here what is the probability of we having our closure between B and C, when the number of common friends are 2, the number of common friends are 4, the number of common friends are 6 so on and so forth.

They observe that the plot looks something like this very linear which means increase in the number of common friends results in the increase in the probability of them becoming friends. Although as you can observe the probability is actually on the very small side its very small 0.0002 and we probably we can go up to 0.008 or 0.010 or something right and this looks linear what does it signify they tried looking at the fact if this can be fitted over some linear curve and they were successful. How did they do this? This, simply it the following they tried looking at the fact that if there are K number of friends with a probability of me becoming friends with someone if I have if the probability of B and C becoming friends if they have one common friend let us call that p.

If p is the probability.

If p is the probability of B and C becoming friendship when they have a common friend 1 - p is the probability of them becoming not becoming friends. If p is the probability of them becoming friends when they have a common friend 1 - p is the probability of them becoming them not becoming friends right and assume you have K such common friends as we know 1 - p whole to the case the probability of you having K common friends, at B

and C are not friends this is the probability let me write this down, this is the probability of B and C not becoming friends despite the fact that they have K common friends.

Now, what I will conclude here, what will 1 minus 1 minus p whole to the K denote this will be denote the probability of this not happening, because I am considering 1 minus of that. So, what this signify this signifies probability of B and C becoming friends with K common friends, let me denote this as T of K. I repeat T of K signifies the probability of two people becoming friends when they have K common friends provided p is the probability of B and C becoming friends when they have a common friend, they observed that for an appropriate p you will basically get this straight line and this straight line and this curve that you observed that I wrote here sort of agree.

This is the curve that they observed and real-world data set real data set and this straight line is the curve $(1 – (1 – p))^K$. As you can see it is a pretty straight forward analysis they observe its linear and there then try to see what could be this p they observe that the p is very small whatever B it was observed the moral of the story is that it was observed that the probability of two people becoming friends goes higher and higher, as the number of common friends go higher and higher.

Social Networks
Prof. S. R. S. Iyengar
Department of Computer Science
Indian Institute of Technology, Ropar

Lecture – 51
Strong and Weak Relationships (Continued) and Homophily
Fatman Evolutionary Model - Implementing Closures

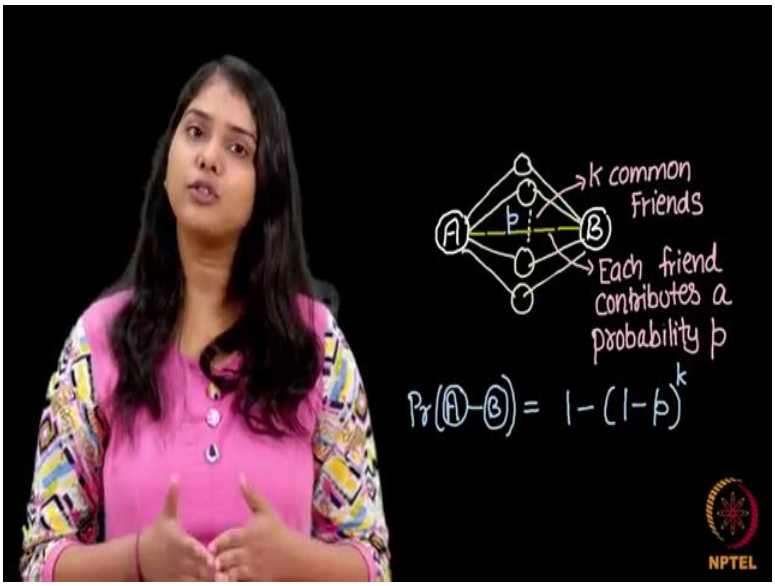
We next want to implement closures in our model, whereas we have seen there are 3 kinds of closures; triadic closure we both have a common friend we become friends.

(Refer Slide Time: 00:13)



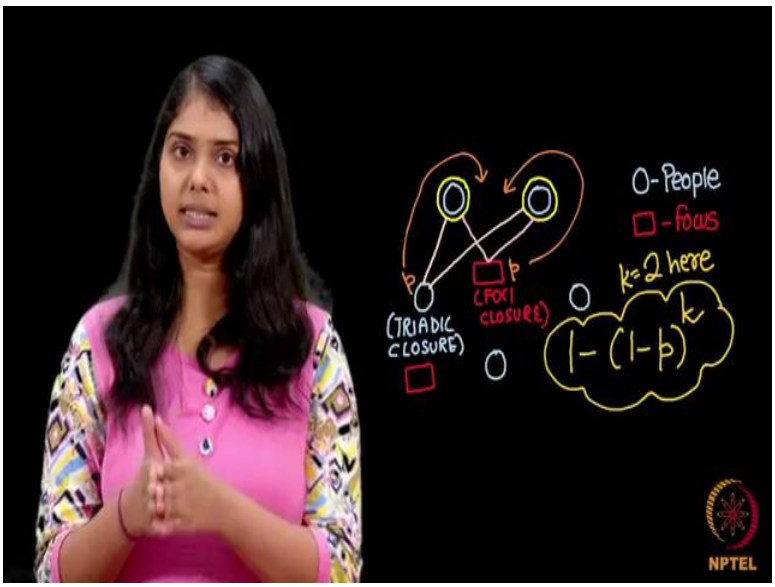
Foci closure: we both are going to the same gym, so we become friends; membership closure I am your friend, you go to gym, so I start going to gym. Now, how do we implement these closures in our model is now the question. So, how have we implemented these closures before? So, we have used a formula if you remember.

(Refer Slide Time: 00:44)



So, what we have done is; if there are 2 people A and B and these people have let us say some K number of common friends, then assume that each of these friends; each of these common friends contributes to a probability p. So, one common friend contributes to a probability of p that these 2 people will become friends. Assume the; it is 0.4, so one person says that with 40 percent probability, these 2 people A and B will become friends and then we looked at this formula that the probability that these 2 people will become friends in the next iteration is 1 minus; 1 minus p to the power k.

(Refer Slide Time: 01:38)

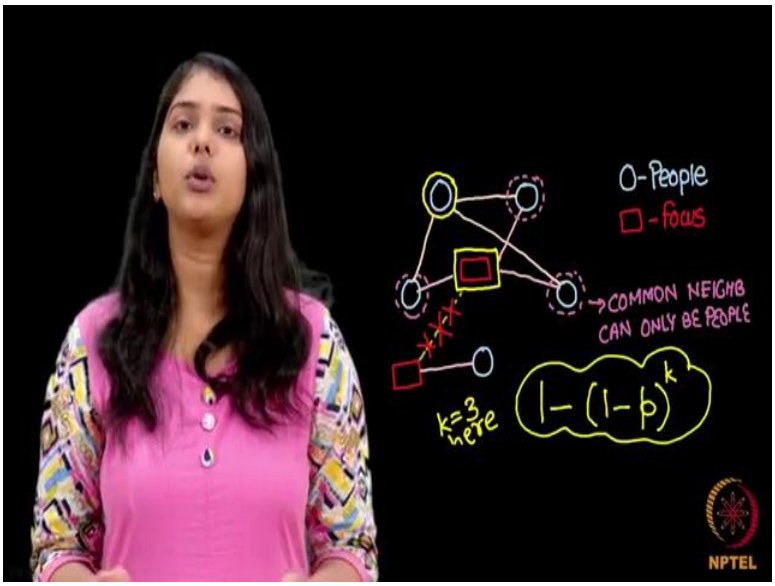


So, we will be using the same formula for implementing all 3 kinds of closures. Now, how do we implement it? First, we will look at every 2 possible combination of nodes. So, when we pick 2 nodes from this network; we will pick every possible pair of 2 nodes. So, when we pick these 2 nodes from the network what can be their types; so, both of these can be the person nodes; person A, person B or one can be a person node, and another can be a foci node.

So person and foci node or vice versa foci node and person node, so, mainly one person node and one foci node and in the last case both of these can be the foci node. How do closures happen in all of these 3 cases? So, in the first case when there are 2 people node; so if we look at the common neighbors, so these 2 nodes. So, we know that in our model the common neighbors can be a person as well as the foci. So, we use the same formula for both of these things, so we capture both the triadic closure and the foci closure using the same formula.

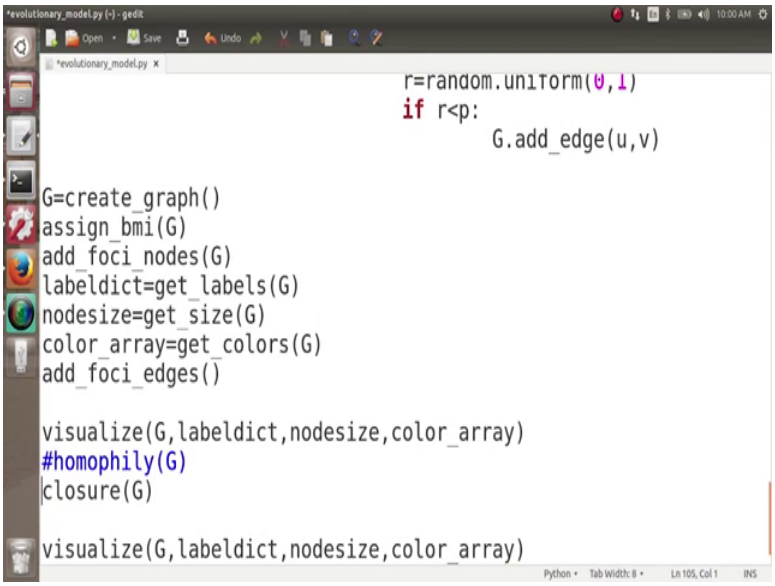
So, we assume that whether it is a person which is your common friend, or it is a foci where both of these are going contributes to a probability p of them becoming friends. So, as I told you that this is a very; we are trying to keep things very simple here. So, once we write the complete coding; we can actually change both of these probabilities, we can assume that or we can code it that if there is a person node; it has a higher probability of making them friends and if it is a foci node, it has a less probability of making them friends, but for now we keep both of these the same. So, whether it is a common friend or common foci; it contributes to a probability p of both of these becoming friends and then again, we use the same formula. So, that formula helps us in achieving both things here triadic closure and foci closure.

(Refer Slide Time: 03:38)



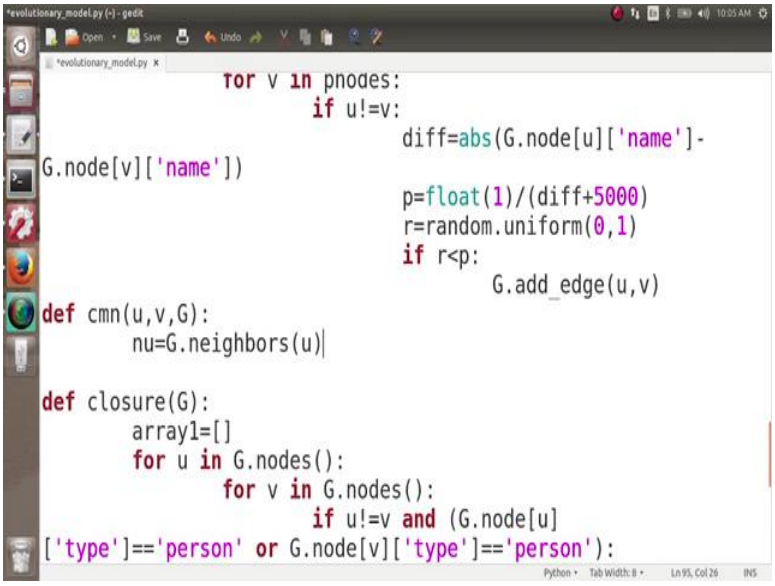
Let us now look at the second case, so here is a person and then here is a social foci; now we look at their common neighbors. Now, if you look at their common neighbors; you observe that their common neighbors can only be persons, their common neighbors cannot be foci, why, because there are no edges between the foci nodes, so 2 foci nodes are never connected to each other, it is the edges are only between person to person and person to foci. So, we have a person and we have a foci here and all their common neighbors are different-different persons. So, this means that this person here can have k of his friends going to the social foci and again we assume that each of these common friends contributes to a probability of p.

(Refer Slide Time: 04:44)



For this person also to go to this social foci and again we use the same formula. So, you see how we just use one formula for implementing all these 3 kinds of closures; once we call it we can actually change the things here and make it more advanced. Next, we want to implement closure property in this graph, in the meantime; we comment the homophily. So, that we can clearly see what is happening in closure and we have here closure(G).

(Refer Slide Time: 05:03)



So as we have discussed there are 3 kinds of closures which we have to implement become here define closure(G) and what you have to do for closure is we know that in one iteration everybody is going to look at their common friends to any 2 people or any 2 people there going to look at all their common friends and they are going to decide whether in the next iteration, they are going to connect to each other or not.

So, what we do here is take an array let us say array1 and this array will have 3 values. So, how will this array look like; this array will be composed of sub arrays and each sub array will be having 3 values. The first value is the first node, second value is the second node and third value are the probability of connection. So, when we are going in an iteration of closure(G); we do not keep adding edges side by side like we did for homophily. Because, if you add an edge between you looked at 1 and 2 and you added an edge between them, it can have complications for the addition of the coming up edges.

So, the addition of every edge should be independent of what has been added before. So, to maintain that independence we have here this array; array1 and so these are the kinds of values this array1 will holds, so here it can be 2, 5 and then 0.7 and so on. So, how do we go about doing this, so initially we have this array1 which is empty and then now by after looking at all the nodes and their common neighbors, we have to decide whether we have to add an edge between them or not.

As we have seen that for any kind of closure to happen, the only condition required is whatever nodes we are picking both of though node should not be the foci nodes. Otherwise, if both of them are the people nodes; it results in a simple closure and otherwise the other 2 cases result in a membership closure and a foci closure; the only condition is both of the node should not be foci nodes.

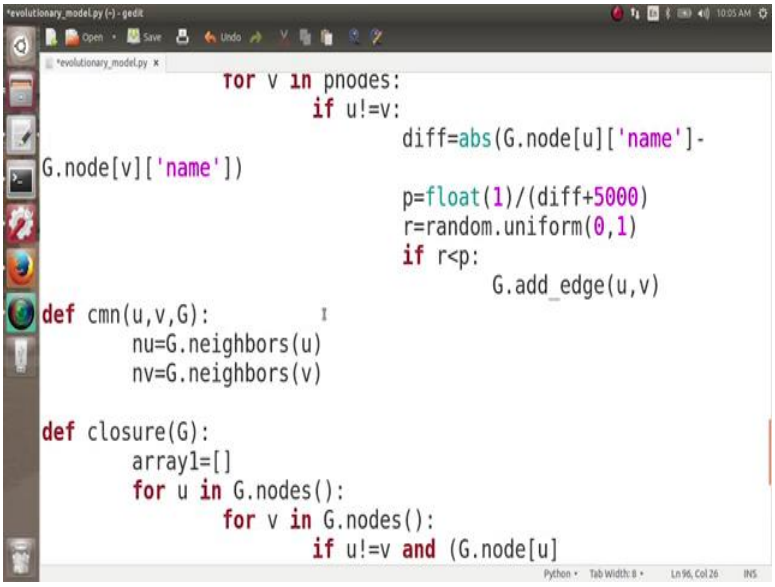
Now, what do we do is for u in G.nodes and then for v in G.nodes; now both of these nodes should be the foci nodes at least one of these should be the person nodes. So, if first of all u != v and either G.node[u][‘type’] == ‘person’ or G.node[v][‘type’] == ‘person’. So, one of they should be person; in that case we have to implement that closure. We already know; what was our formula for closure, and we are using that same formula for all 3 kinds of closure.

So, here we find out the value of a probability p which is nothing, but our formula so our formula was $(1 – (1 – p))^k$. So, for having $(1 – p)^k$; we have this function math. power.

So, we have to add this package math at the (Refer Time: 08:47) which will do, so 1 - math.pow(1 – p, k), but now what is k here; k is the number of common neighbors . So, we need the number of common neighbors between u and v. So, we have here k equals to; find k equals to common neighbors.

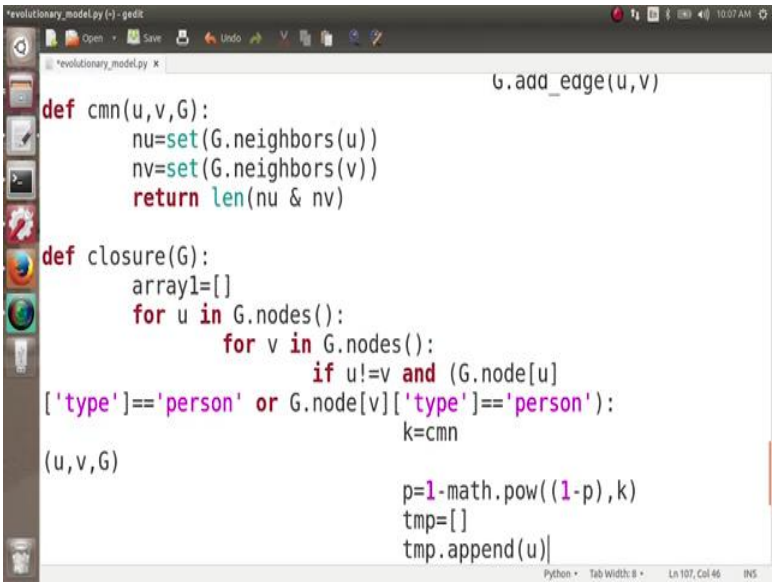
So, let us have simple function in cmn common, common neighbors between u and v in the graph G. So, now we need to define this function here; define cmn common neighbors of u, v, G and how do we find its value is first we need the neighbors of u.

(Refer Slide Time: 09:57)



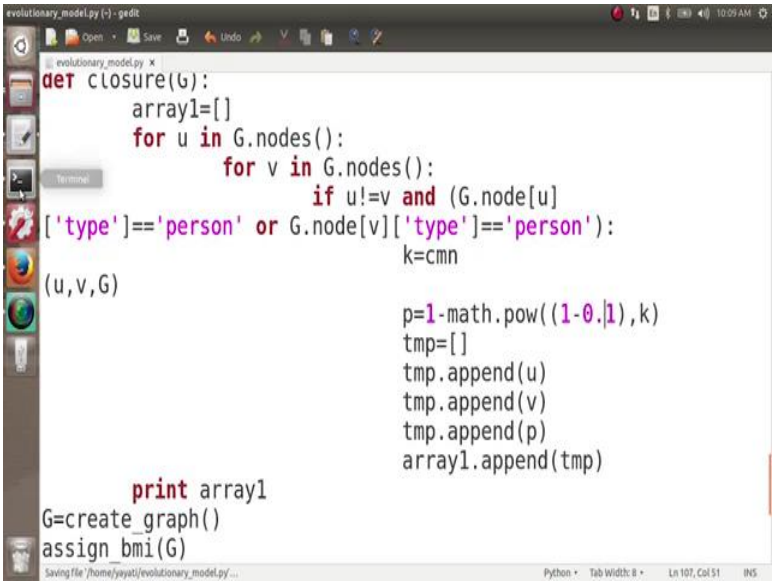
So, neighbors of u = G.neighbors(u) and then neighbors of v = G.neighbors(v) and we want the common one.

(Refer Slide Time: 10:28)



So, for having common one we need a set intersection, so let us make both of these value a sets. So, nu is the set having the neighbors of u and n, v is a set having the neighbors of v and what we have to return is the length of the set which set n, u and n, v which only looks at the common elements of u and v; to be written length of nu and nv. So, we have this probability here; math.pow(1 - p, k). Now, we have to feed these values inside our array which has to had 3 parts u, v and the probability of connection.

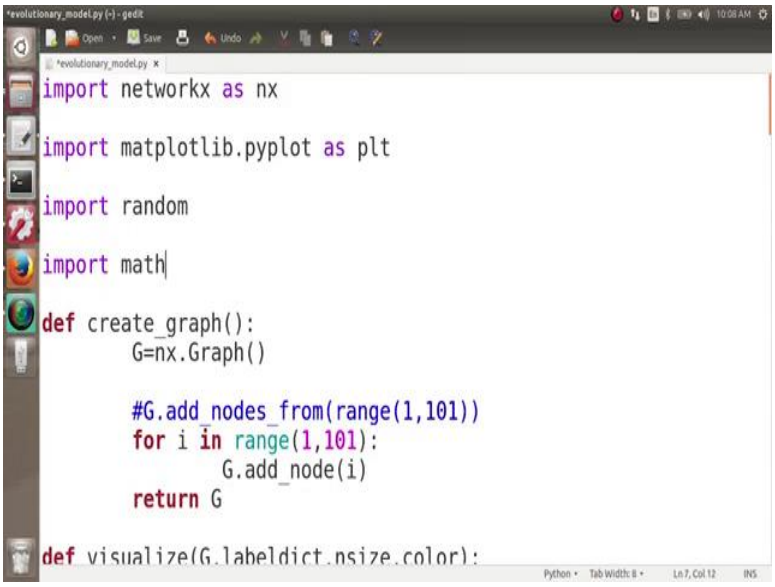
(Refer Slide Time: 11:33)



So, we need a sub array here let us name is strength, so the first value which we are going to append(tmp) phase u and the second value which we are going to append(tmp) is v and then the third value which we append(tmp) is p.

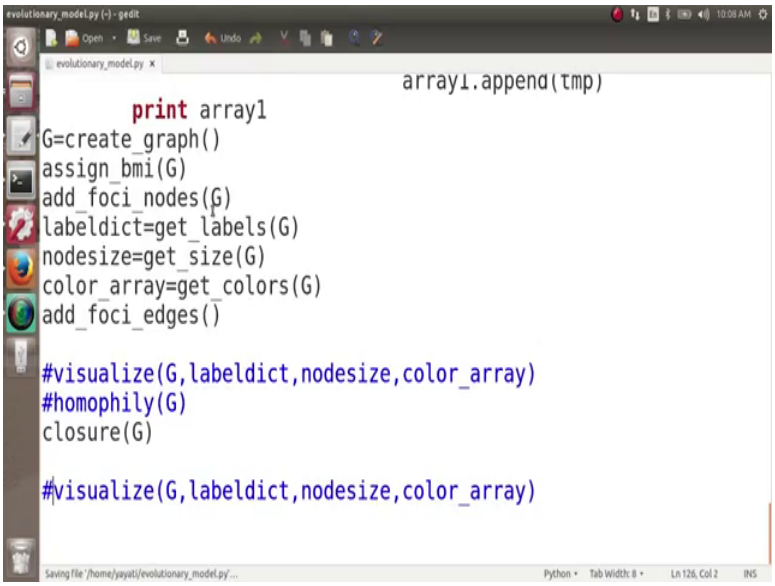
So, tmp has these 3 values and at the end we append(tmp) in array1; array1.append(tmp). So, we have made here this array1; for all possible pairs of nodes, let us quickly look at how this array1 looks like.

(Refer Slide Time: 12:14)



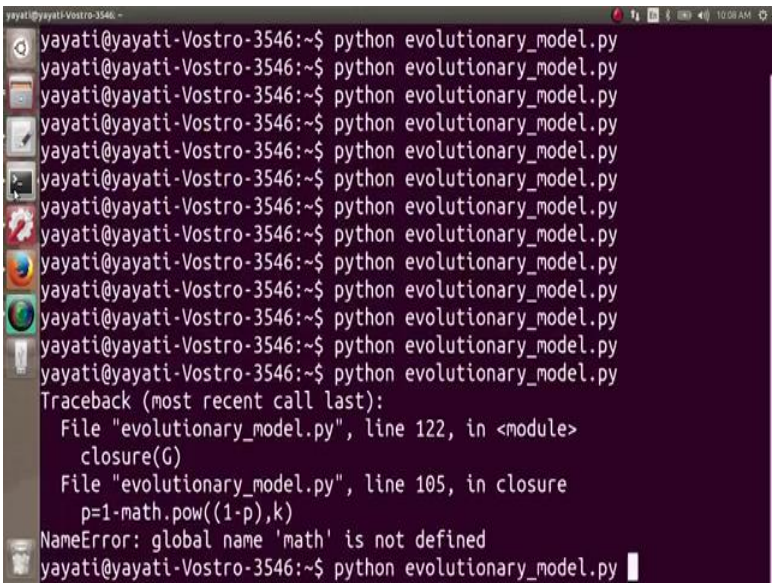
So, let us print array1 and see how it looks like; now we forgot to import the package math, we go up and here we import math.

(Refer Slide Time: 12:35)



And at the end is this is become a (Refer Time: 12:37) code, so at the end for the time being we comment and visualize functions as well and now let us look at the array.

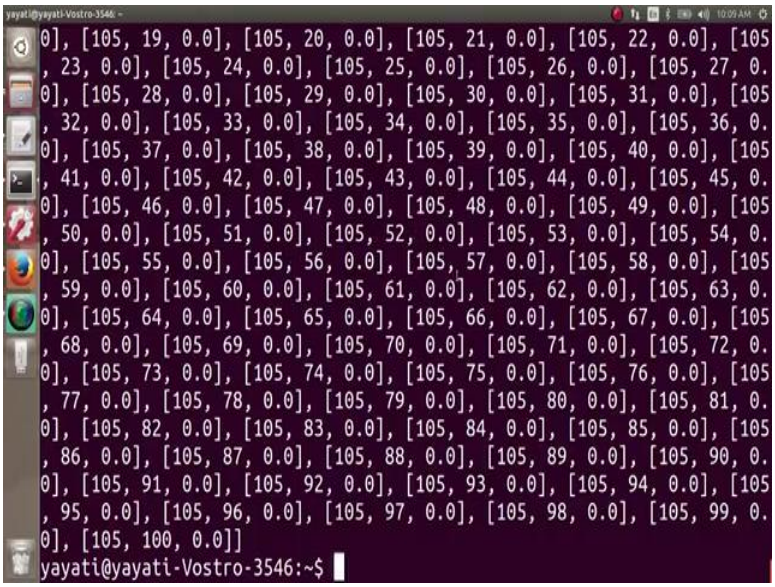
(Refer Slide Time: 12:42)



There is some problem, so this p here; p in the formula, this p here was supposed to be a constant. What was that constant value of probability? That is the probability each of your common neighbor contributes to u forming a connection. Let us come back, when we come here to closure p equals to this probability is supposed to be some value here.

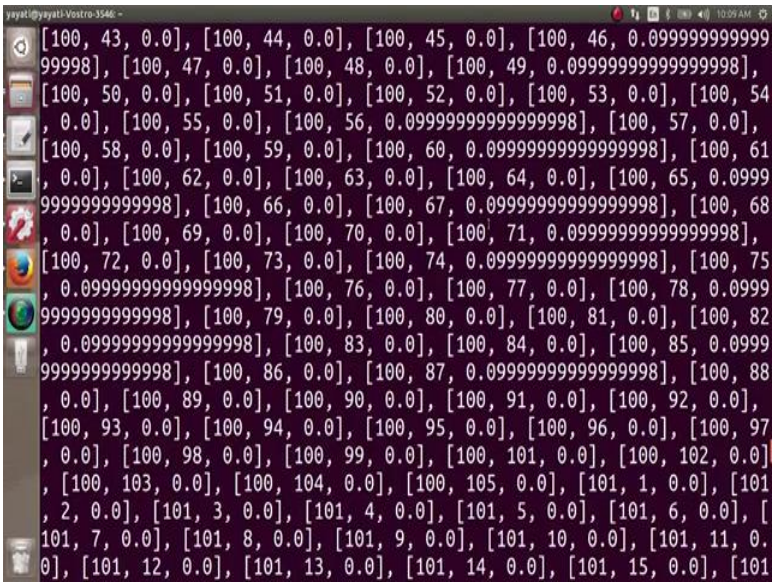
So, let us say each of my neighbour contributes 0.01 probability of connection; let us rather say 0.1 probability of connection; then the more common neighbors we have, more probability of connection becomes; let us go back execute it.

(Refer Slide Time: 13:52)



So, now you can see here your array and as we can see that as of now most of the values are here 0 and there is a reason for it to be 0 because there are very fewer common neighbors here.

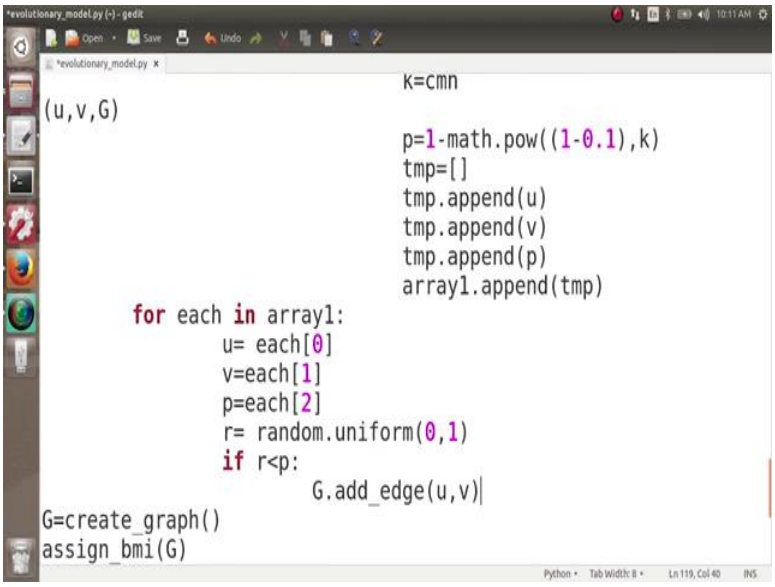
(Refer Slide Time: 14:10)



So, you can see here in some cases we have here this value here 0.09998, so since it is the very beginning of our codes. So, initially we had seen that the common neighbors only exist between, so there is no homophily as of now; we have commented the function homophily. The common neighbors will exist only when 2 people are the part of same social foci.

So, you can see here if you look at the nodes here 100, 87 and then 100, 74; 100, 56. So, these are mostly the people node, which are connected to the same social foci and this is the probability of these 2 nodes getting connected with each other. So, we apply a closure here, it is mostly since there is no edge between 2 people; it is mostly going to be happening a foci closure only. So, here was this array1; now we must make edges in accordance with this array1.

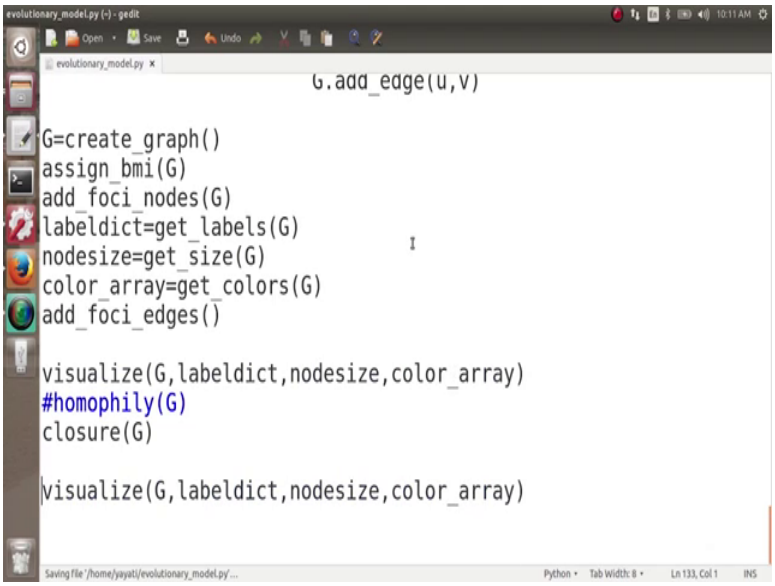
(Refer Slide Time: 15:21)



So, that is very simple; so, we go for each in array1; so, for every entry in array1; what do we do; our first value, first node is each 0 and then our second node is each1 and then our probability of connection is each 2. As we have done before, we choose our random real number from 0 to 1; r = random.uniform(0, 1) and if r < p, we add an edge G.add_edge; edge betweenu and v.

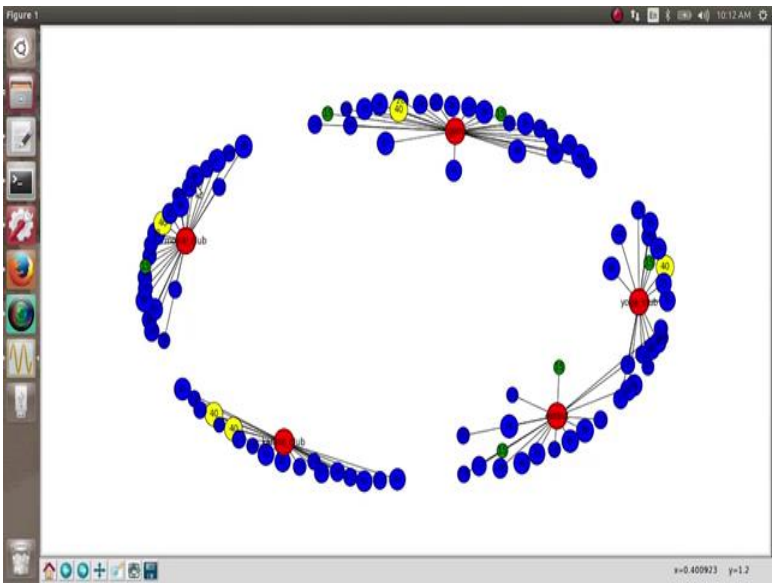
So, this is one iteration of our function closure let us execute and see it.

(Refer Slide Time: 16:10)



So, here we will visualizing our graph as well, so this is our initial graph looks like; let us see what is going on here. So, actually there are no intersections here it has just, ok.

(Refer Slide Time: 16:13)

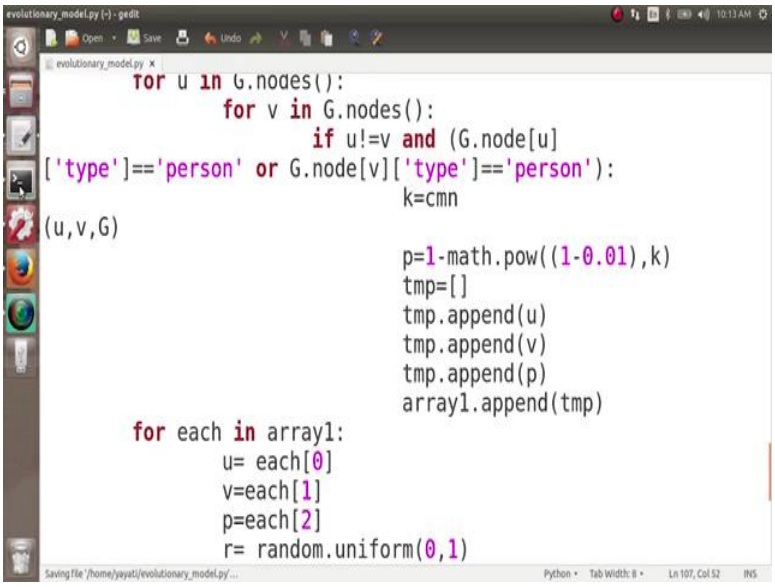


So, you can see here every node is actually the part of exactly one social foci here, we will zoom this graph and see and now this is what happens after closure. So, let us see what is happened after closure; let us look at this. So, you can see that there are more and more edges which have been added between people. So, 18 has now become connected

to 28; 28 has become connected to 24 because they were all the part of the same social foci gym.

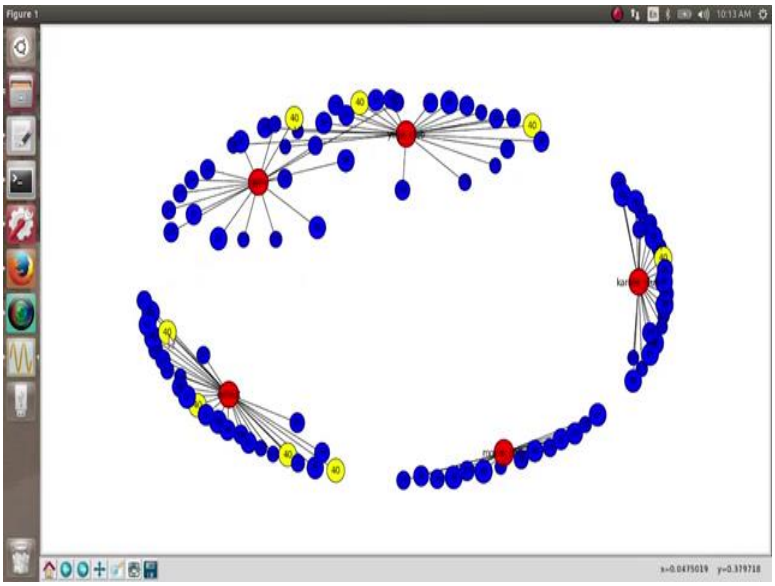
So, you see how here the edges between people have been added, so as we can see here that probability of connection seems to be very high. We do not want this high probability of connection, so for that what we can actually do is; we go back and we deduce this probability here.

(Refer Slide Time: 17:22)



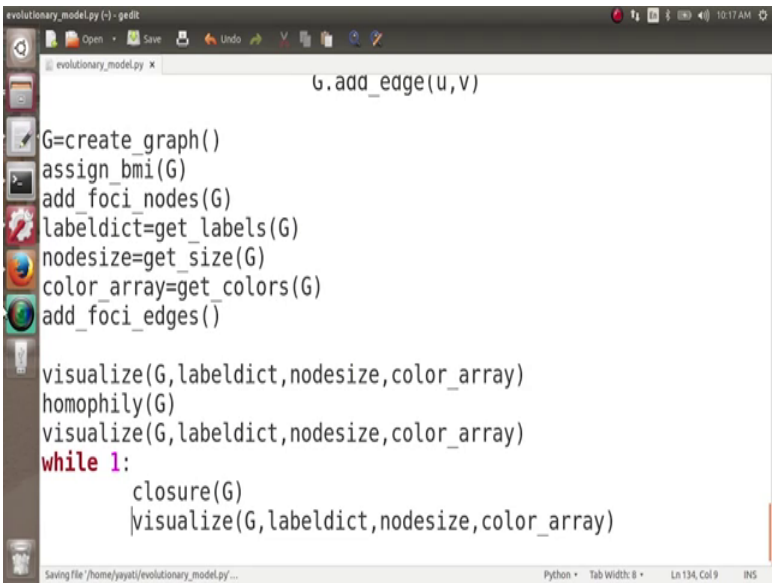
So, let us say one common neighbor contributes 0.01 probability of connection. Let us now look at, so this is our initial graph and let us see what happens here now. Let us again pick 1 here; let us say we pick this karate club and then you can, now here see that the probability of connection has reduced. So, this karate club leads to the formation of some number of edges between 2 people.

(Refer Slide Time: 17:31)



We might want to see other kinds of closures as well, so till now here only 2 people were getting connected to each other because of having a common social foci.

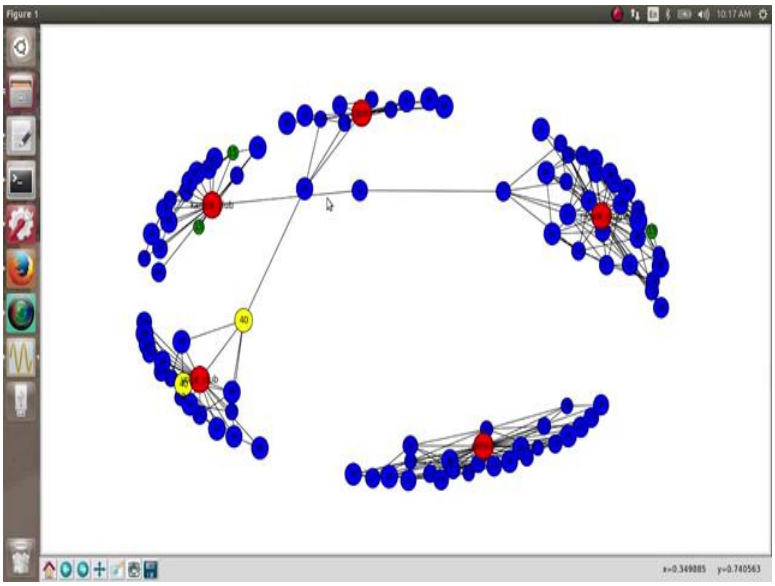
(Refer Slide Time: 18:05)



So, for that we would like to do homophily first so that we have some edges between the people as well and then we visualize and then we apply closer. So, for looking at, so let us for the meantime reduce this value and differentiating between the triadic closure and foci closure is a little bit difficult because there we edge edit between 2 people only. We would like to look at the membership closure, where one person becomes a part of social

foci because his friend is a part of social foci. So, let us try to look at that; we put this coding in a loop. So, while one closure happens and it keeps visualizing it again and again; now this is homophily, so, this is the graph. Now you can see here, this node 35; it is going to the karate club as well as it is going to the gym.

(Refer Slide Time: 19:02)



So, this node 35; it has 2 social foci is here in which it is participating, but according to our code initially node was assigned to exactly one social foci. So, most probably 35th node was initially a part of karate club only as we see then it has identified that one of its friends 1920 or somebody was going to gym and hence 35 also decides to go and be a member of gym. So, we can actually see a membership closure also happening here; you can write this code, play around with it and look and verify it further and it will keep happening till I do not stop it.

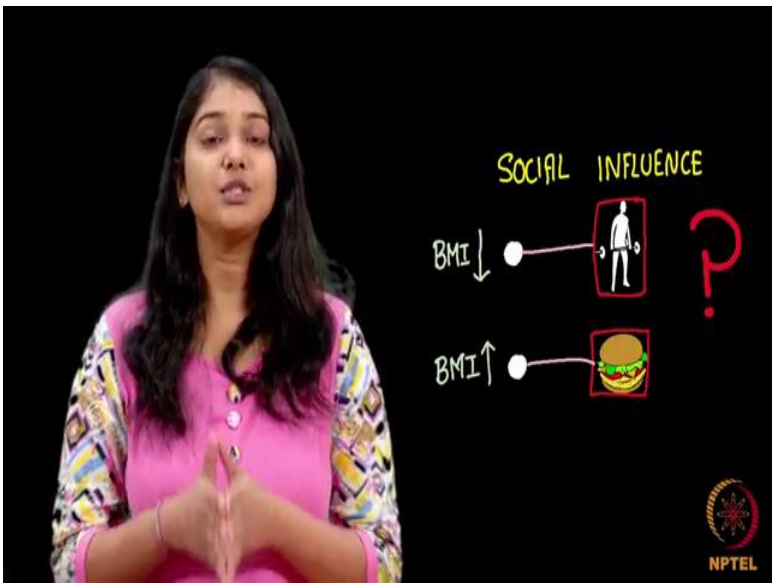
So, let us just stop it here control z; I will stop this code here. So, till now what we have done is; we have made a graph having 100 nodes with different BMIs where every node is part of exactly one social foci. Then we implemented homophily here and then we implemented closures here and we implemented all 3 kinds of closures; triadic closure, foci closure and membership closure and we have also seen them happening on this graph.

Social Networks
Prof. S. R. S. Iyengar
Department of Computer Science
Indian Institute of Technology, Ropar

Lecture – 52
Fatman Evolutionary Model – Implementing Social Influence

Finally, we have implemented homophily, we have seen how do implement three kinds of closures scenario evolutionary model and the last and the most interesting thing which remains is the social influence and it is actually the easiest. We need not do anything here in this model to achieve social influence, as I have told before.

(Refer Slide Time: 00:20)



So, there is a very subtle way in which we capture these social influence happening; how do we do that is, assume that if a person is connected to a gym so; obviously, he is being connected to gym should result in a decrease in his BMI and if a person is connected to an eat out place, they should result in increase in the BMI of this person.

So, what do we do; for every node which is connected to a gym, we reduce its BMI by 1; every iteration and every node which is connected to an eat out place, we increase its BMI by one every iteration and we just do this and the social influences captured; How? So, as I have told you; we have seen here now, when a person gets connected to a gym; he is losing weight. Now, I have a friend and this friend is going to the gym; now

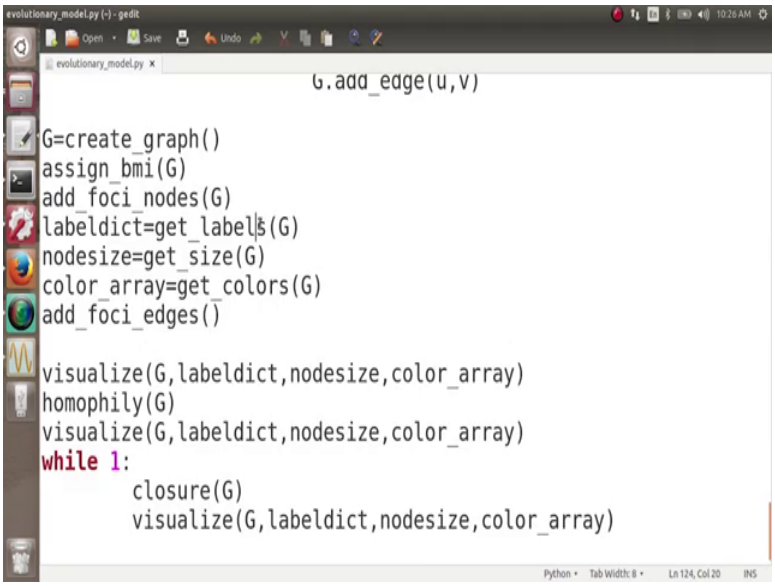
because of membership closure, I tend to become a part of the gym and when I become a part of the gym; as you saw I will start losing weight.

(Refer Slide Time: 01:22)



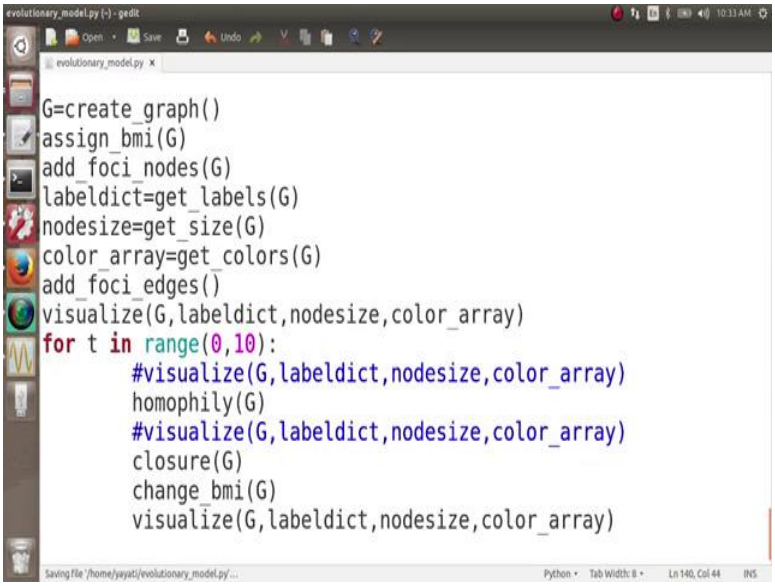
So, this is one aspect of social influence, so we have mainly looked at three reasons why in which social influence can happen, but we capture one of those here and that reason is the sheared context. When 2 people are the part of the same social foci, it leads to an influence. So, we have seen that it is like; if 2 people both have a brilliant teacher, both of them will start having good marks. Similarly, if 2 people here both of them are part of gym; so one person who was going to gym, another also started going to gym will start losing weight and similarly with an eat out place; the effect is reversed, so this is how we capture social influence here.

(Refer Slide Time: 02:25)



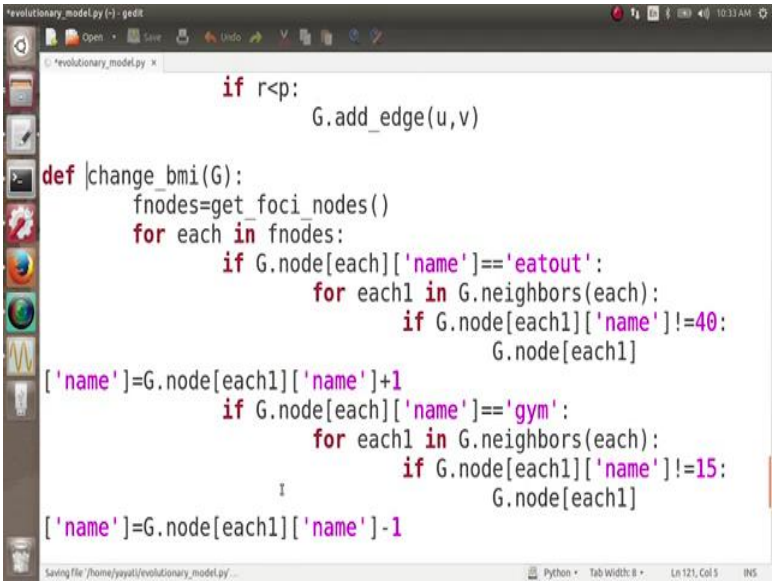
Finally, what we want to do here is; we want to implement social influence. How we are implementing social influence? We have already discussed before; well it is an indirect implementation of social influence. So, it is mainly at every iteration a person who is associated with gym; lose his weight and a person will associated with an eat out, gains weight and then as we know that membership closure is happening. So, let us say I am a part of gym and then I have a friend, who is fat, then he looks at me and I have joined gym, he also joined gym and hence he started losing weight. So, this the kind of social influence we are going to implement here.

(Refer Slide Time: 03:18)



So, for meanwhile let us just comment everything and now what we want to do here is; we want to change weights, we want to change BMIs of people.

(Refer Slide Time: 03:41)



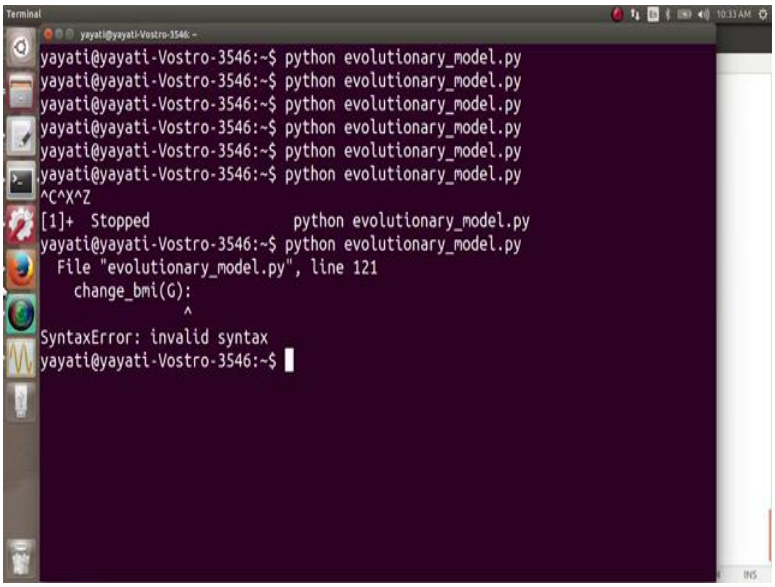
So, we called change_bmi(G) and what this function change_bmi(G) does is change_bmi(G). So, for each in f nodes; what do we do for each1 in G.neighbors(each); what do we do, if G.node[each1], but again we are doing it for the foci node which are either an eat out or a gym. So, we want to see that so for each in f nodes; we again need to put a node here, if G.node[each]. So, if this each node its name; its name == ‘eat out’.

If its name = ‘eatout’ only then we are going to insecure this code, so if its eat out we look at each of its neighbors and if each G.node[each1][‘name’]. So, now since we are going to increase the weight, first this name should not be equals to 40 because if it is equal to 40, we cannot further increase the BMI of this person. So, if each if G.node[each1]; name != 40; what do we do is we increase it by 1.

G.node[each1], name = G.node[each1][‘name’] + 1 and then we again do this same thing for gym. It is essential we have to copy paste the same code here, if G.node each, name = ‘gym’. So, if it is a gym we look at all of its neighbors and if for a neighbour this value should not be equals to 15; if this value is not equals to 15; we decrease its BMI by 1, so this is how we are going to change the BMI of people.

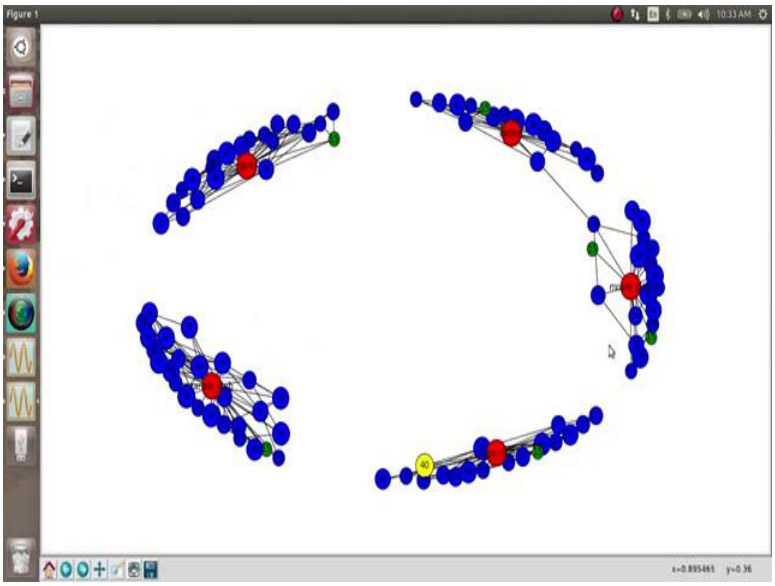
Now, what are we are going to do is; we have all three components in our network. We have homophily, we have closure and we have social influence. We are going to put all of these three together and we are going to visualize our network. So, let us take time for t in range; let us do it over a time period of; let us do it for 10-time instants. So, for t in range 0 to 10; what are we going to do is, we do homophily we apply (Refer Time: 07:00) closure, we apply social influence and then we visualize this graph and add the beginning when nothing is done that time also we want to visualize our graph. So, we have the statements here; now let us execute our code and see what is happening.

(Refer Slide Time: 07:16)



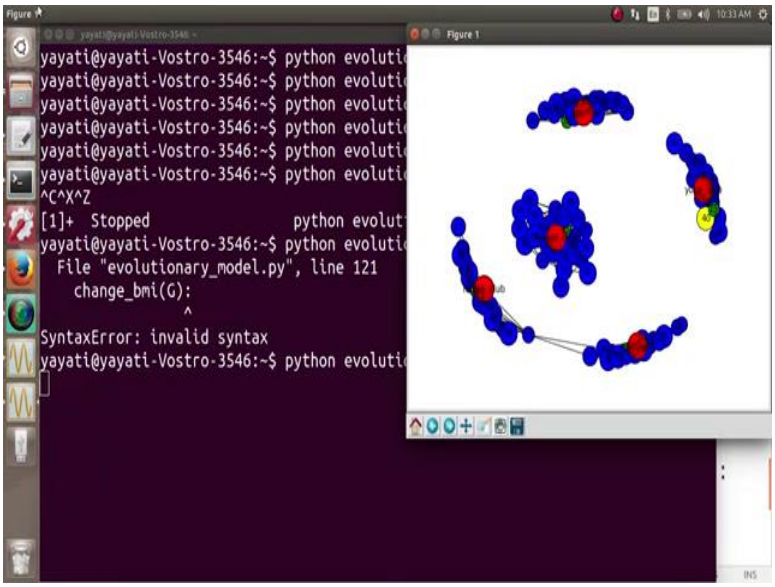
change_bmi(G); so, this is a function definition; so, we need a define here; go back and secure it. So, this is our initial network where we have five social focis and we have different nodes; I mean different BMIs.

(Refer Slide Time: 07:31)



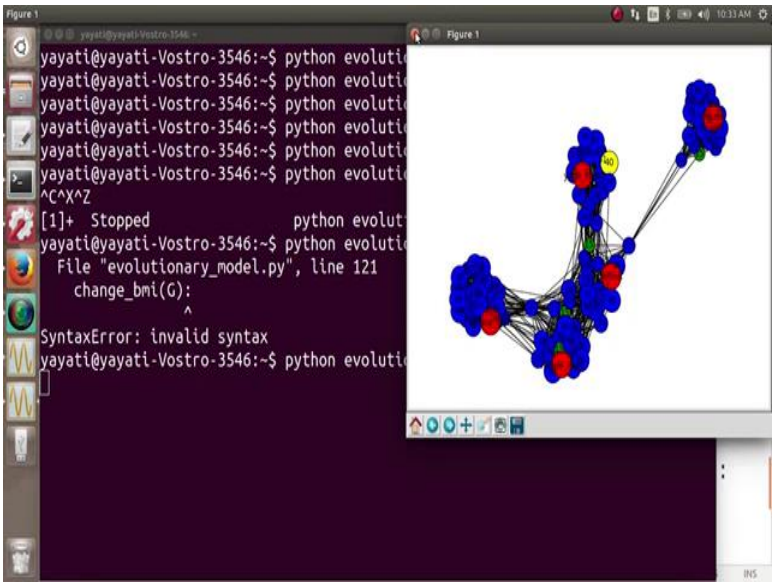
Now, we see here that some nodes have become friends with each other and there are more number of edges between these nodes and let us look at the number of obese people in this network, so currently we have one obese node which is 40.

(Refer Slide Time: 08:00)



We again have one obese node; which is 40 and is a greater number of nodes are getting added; Do you see some problem in this code? I see some problem in this code.

(Refer Slide Time: 08:10)

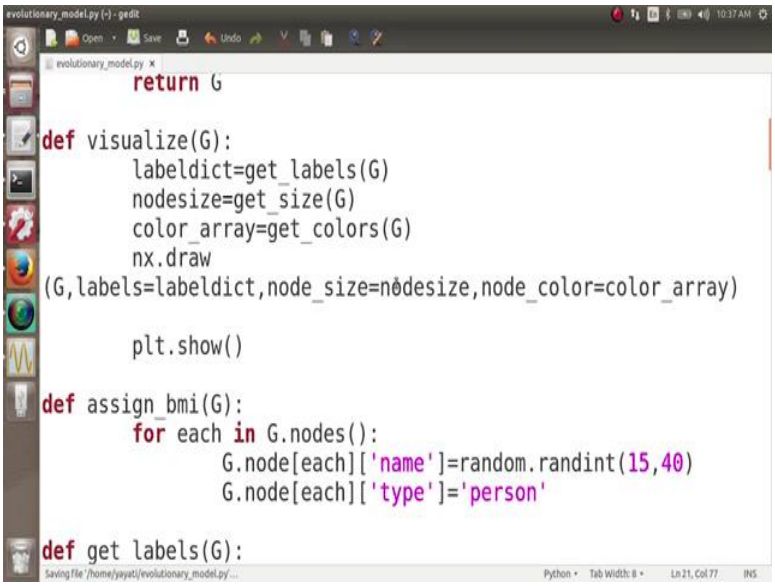


So, as we can see the number of obese people is now changing in this network that should actually happen because we have implemented social influence. So, what can be the problem with this code, so let us try to troubleshoot it together when we write a piece of code, we should be able to troubleshoot this code nicely? So, you will see here when we were visualizing a graph G here; we have passed these various arrays here; color array, node size, label dict. So, here when we have here when the BMI of people have changed and then the social influence has occurred. Do not you think we need to again calculate all these values?

So, it is like in the beginning we saw which nodes were yellow, we faded it here and we visualize we fed it here and we visualized it and now when we are running this graph; we are again and again using the same color array, the same size array; so, we need to change this. So, best way to change is to have these values; took all these functions inside our visualize.

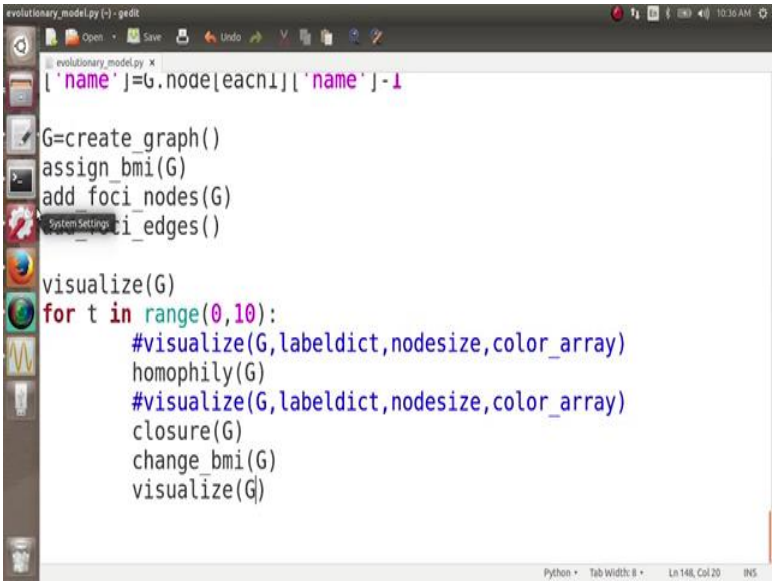
So, this functions like let us say labeldict function and then the node size function and the color array function. So, all these three things need to be calculated in the visualize function because whenever you visualize it, these values are keeping, these value that changing. So, what we do is we just delete these values from here and we put all these values inside all these function calls; inside a visualize function. So, here is our function visualize; so, what we do here is we have all these values here.

(Refer Slide Time: 10:04)



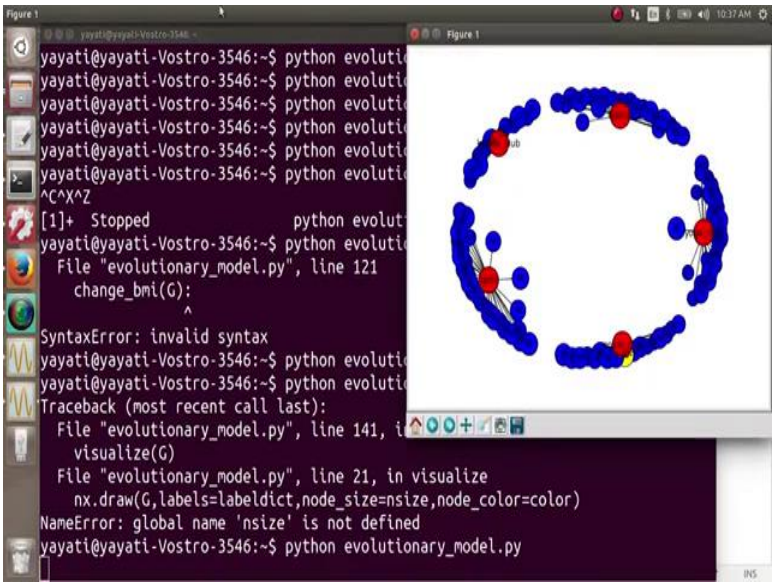
So, whenever we want to visualize a graph; we again look at the labels, we again look at the size of nodes and we again look at the colors and then we do not need to have all these things here, we just have a define visualize(G) and when we call these function towards the end, there also we just need a visualize(G). So, whenever you will try to code sometimes when weird things will happen. So, the best ways to keep printing your different parameters at different places and see what is going wrong where.

(Refer Slide Time: 10:32)



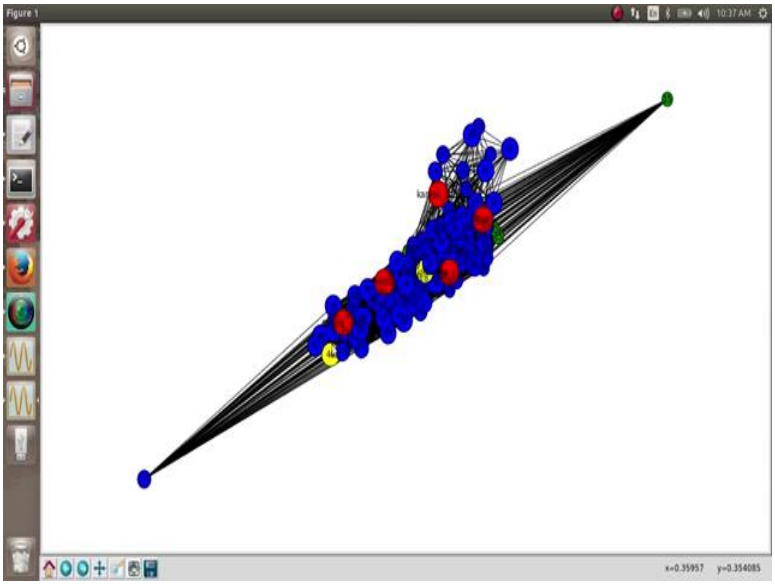
Now, let us again execute it.

(Refer Slide Time: 10:21)



Line 21; there seems to be a problem, so let us go to the line 21, so we have calculated labeldict here, we have calculated node size here. So, instead of n size here; it should node size and then node under scroll color is color under scroll array node color. Now, let us look at this graph, so this is our initial graph and then you see some 2 friendships have been added to this graph.

(Refer Slide Time: 11:56)



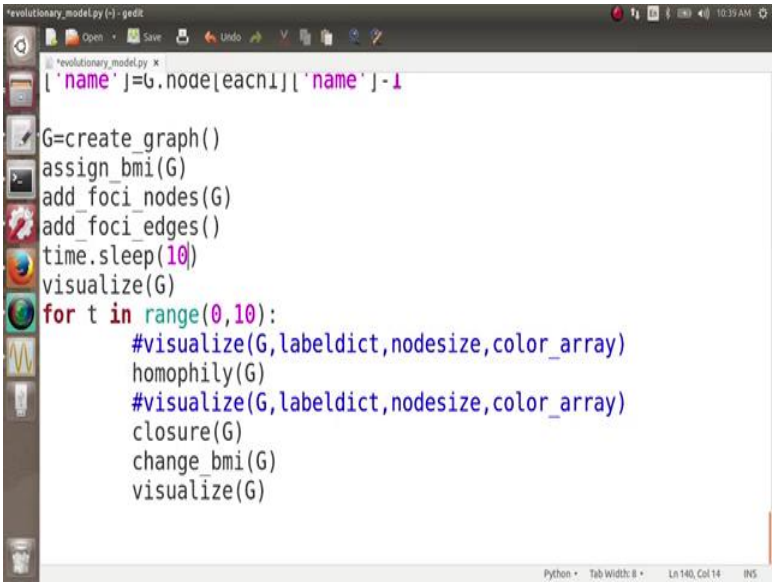
So, now you see there is one yellow node and there is another yellow node, so you see that the BMI of one node has been increased and it is hit the obesity parameter. So, this

how a graph will keep changing; to have a battle visualization, we can actually change the parameters, we have used in the code; which you will see that with time more and more number of people will keep becoming obese because of social influence and yes people will use their weights also because of this.

So, let us quickly have a look on the parameter, so, this was for homophily and then this is for closure. So, let us keep it small for the time being let us keep it 0.01 and then we can again visualize this graph.

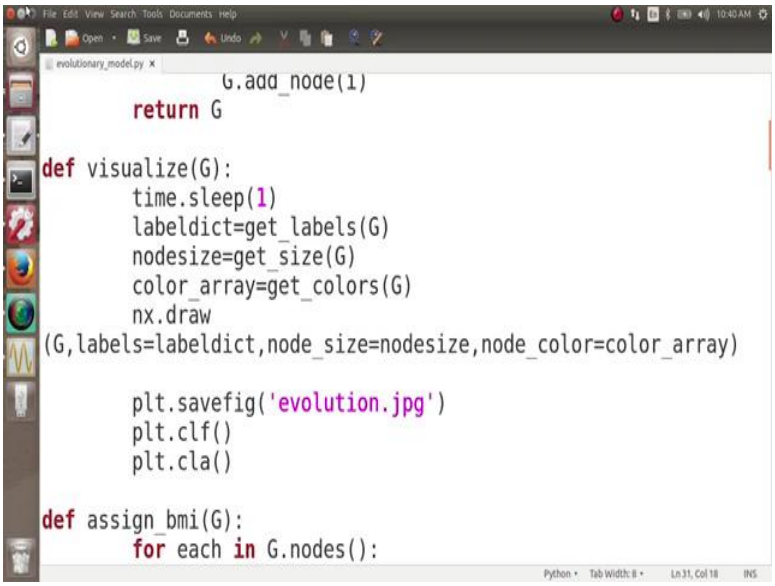
Now, let us visualize this graph in a very nice way, so what we are going to do for this visualization is we store this graph in a file jpg file and when we run this code at jpg file, we will keep changing with time and we observe that change in that jpg file. So, for that what do we do is; when you call this function visualize(G), this function will store your graph; will save your graph as a jpg file; instead of plotting it here.

(Refer Slide Time: 13:19)



Now, first let us put a sleep statement here so that before making your file, the code sleeps for some time.

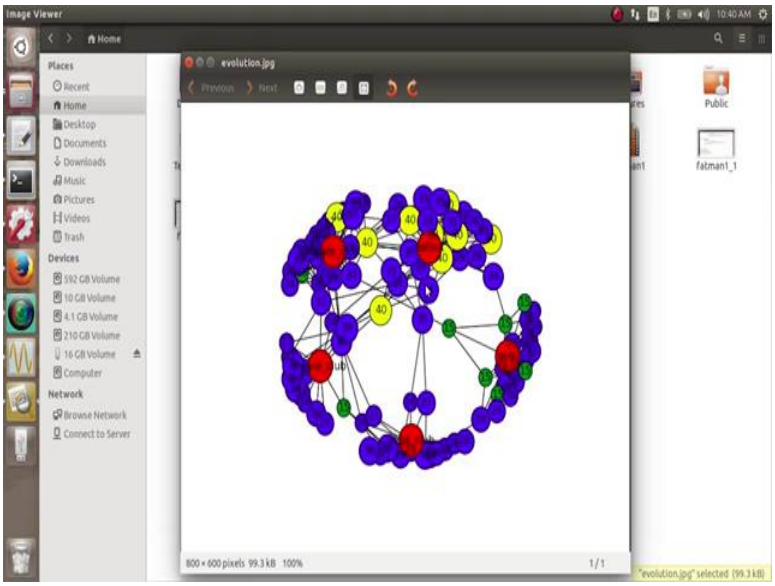
(Refer Slide Time: 13:39)



So, let us say some 10 seconds and whenever you call your function; visualize(G), what this function is going to do is; it will wait for let us say 1 second and for this, use this sleep statement we need to import time. It may put some delays in your code, so time.sleep(1) and here instead of showing our plot, what do we do is; we save our figure plt.savefig and let us say evolution.jpg.

So, this plot is saved and once this plot is saved; we need to clear this plotting screen for the next plotting to occur. So, what we have done is; we have removed the plt dot show statement here and whatever the output graph is, we are having that output graph in a separate file and that separate file keeps changing with time.

(Refer Slide Time: 14:50)



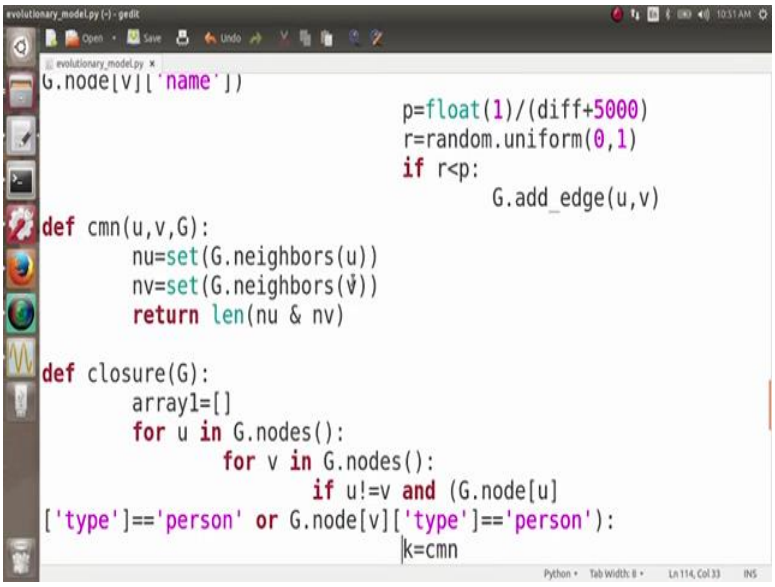
Let us execute this code now and see what is happening. So, let us come back, so here we are and let us wait. So, here is our file evolution dot jpg let us open it and you can see that now this code has started changing with time. Now this figure has started changing with time and you can see that how more and more number of yellow nodes are popping up in the network; how more and more number of green nodes are popping up in the network, which shows that with time the number of obese people in this network is increasing and with time the number of underweight people in this network is also increasing, so it mainly depends on how you execute this code.

So, this was one way of looking at this animation; there actually milli nines python functions to look at this animations. So, for the sake of simplicity we have done it like this time, but next time we will try to use a better approach, we will try to use a inbuilt functions of python.

Social Networks
Prof. S. R. S. Iyengar
Department of Computer Science
Indian Institute of Technology, Ropar

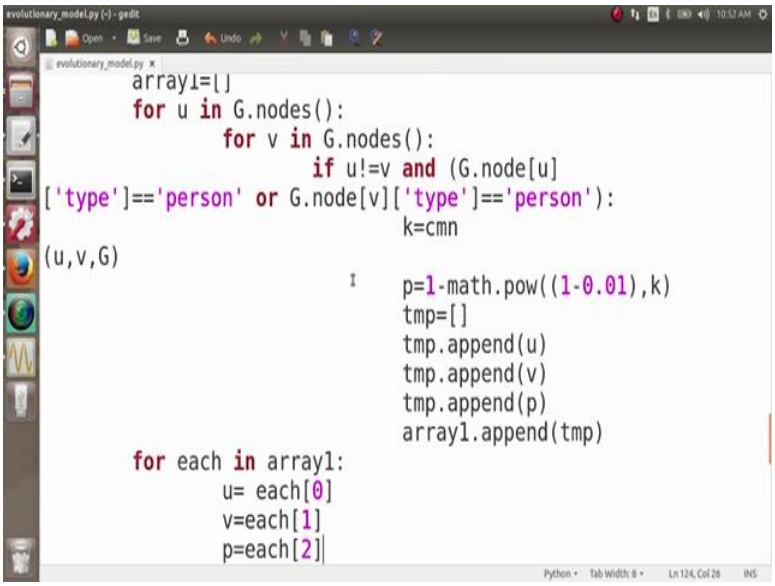
Lecture - 53
Strong and Weak Relationships (Continued) and Homophily
Fatman Evolutionary Model - Storing and analyzing longitudinal data

(Refer Slide Time: 00:05)



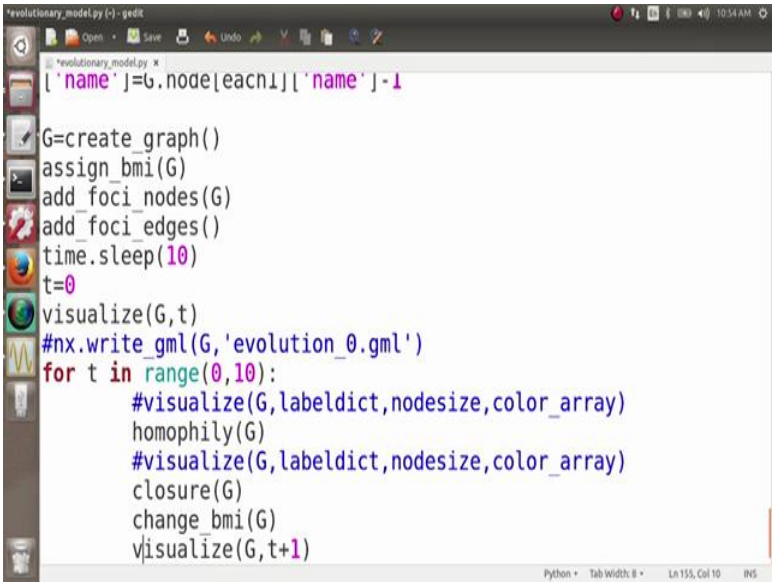
Next what we want to do assume we want to analyze what all is happening in this graph. So, it is not actually possible to analyze it very nicely by visualizing the graph and looking it as a figure every time. So, some nicely written functions can help us in the analysis of this graph.

(Refer Slide Time: 00:25)



So, for that what we will do is, first of all we will store whatever is happening in this graph as different snapshots. So, we will have different network store for different snapshots, next we will look at these networks one by one and look at how this network is changing over time.

(Refer Slide Time: 00:41)

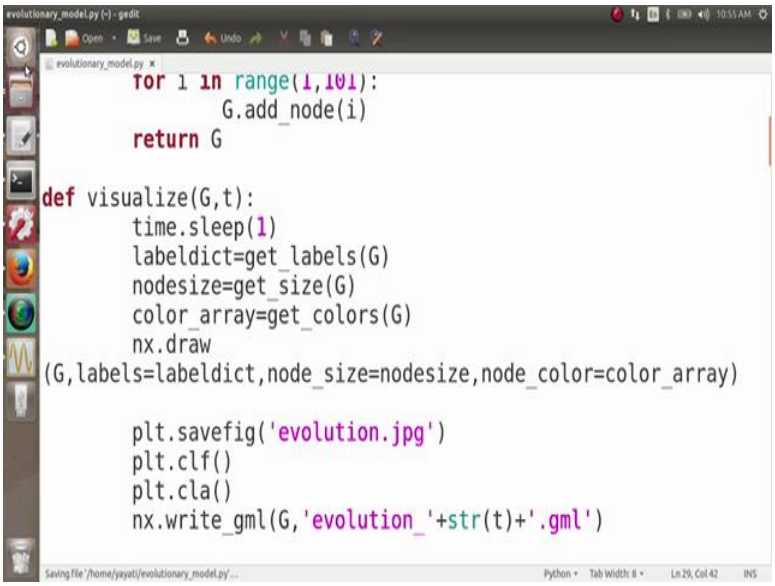


So, what we do here is along with visualizing G we try to store this network. So, we call a function here let us say store G, and this store G takes a time value with itself let us say its t or let us say the time value is i. So, initially this i is 0 when nothing is done. So, we

call store G , i, and a better option is when we visualize this graph that time only, we can store this network what do we do is. So, for the very first of all outside the loop we store it manually. So, for storing it we use we stored it in a gml format. So, for storing it in a gml format we use the function nx.write_gml as you know we have done it before nx.write_gml(G) and let us say the file name is gain evolution_0.gml. So, this is a first file name.

Next when the time runs from 0 to 10 what we are doing is we are calling these three functions and we are visualizing this network. So, let us pass the value t + 1 here. So, why t + 1? t was here t starts from 0, but we want the indices of this network from 1. So, that is evolution_1.gml, evolution_2.gml. So, we have here visualize (G , t + 1) and actually we should be doing it here also because the format of the function is same. So, we need to pass the same parameters here. So, let us first have t = 0 and then we call visualize(G , t) you can actually come and take and then our t then we called visualized (G , t + 1).

(Refer Slide Time: 02:56)

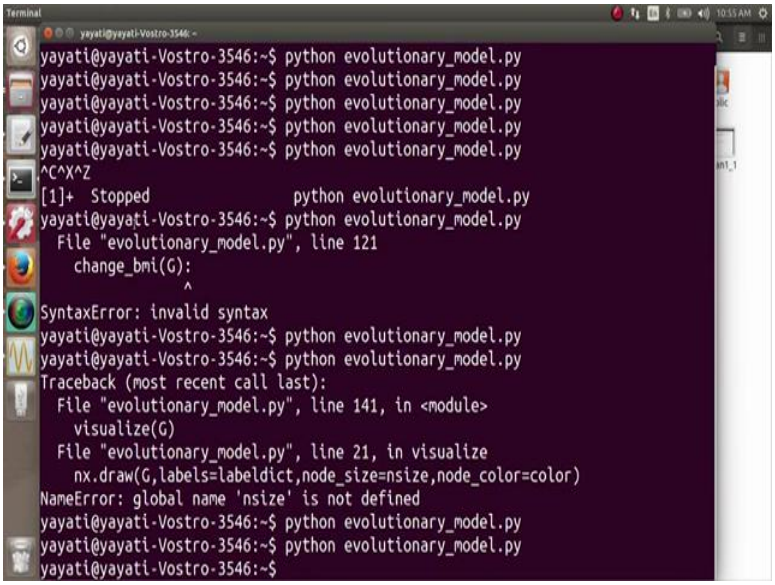


And what happens here coming back to our visualization function. So, it now it is taking an extra parameter which is the t at which the time t the graph is to be stored. So, what we do here is to store the graph after doing all this, we have a function here nx.write_gml and then we write this graph G as what should be its name its name should be evolution underscore, and then we have this value of t here. So, we do a string append operation

and what do we append to this string is the value of t as a string. So, it is evolution_0 then evolution_1 and we append.gml to it.

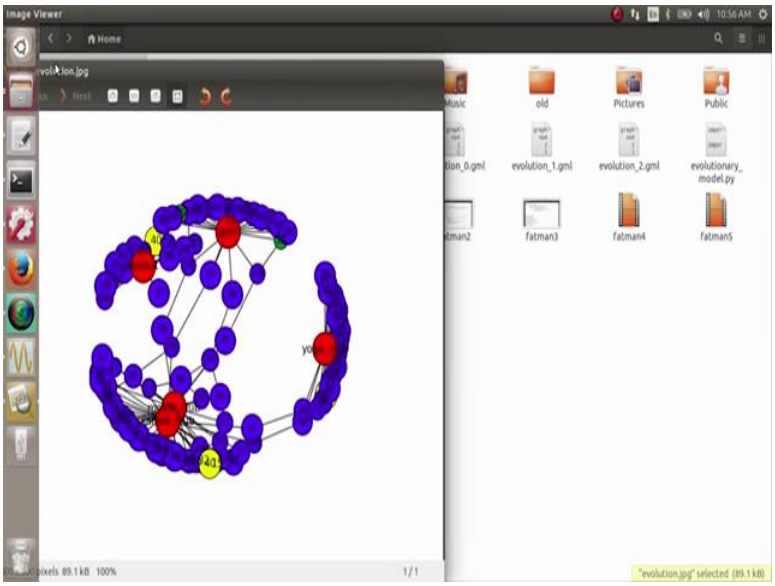
So, how this graph gets stored is G evolution_0.gml, evolution_1.gml and so on and then will be loading this graphs and we look at some properties of this evolution of this completes evolution. So, let us see let us try to run it.

(Refer Slide Time: 04:03)



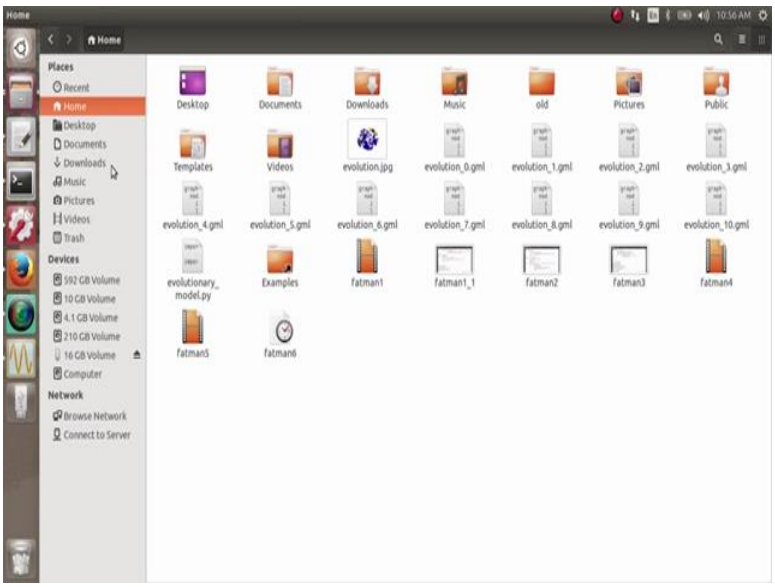
So, let us see how this graph gets generated here we let us (Refer Time: 04:06) python evolutionary_model.py and let see what happens here. So, we have seen as before this file evolution.jpg is going to change with time it will keep changing.

(Refer Slide Time: 04:16)



So, and if you see here let us just let us just close it. So, you see here that we have here these different gml files for me evolution_0.gml, evolution_1.gml, 2.gml.

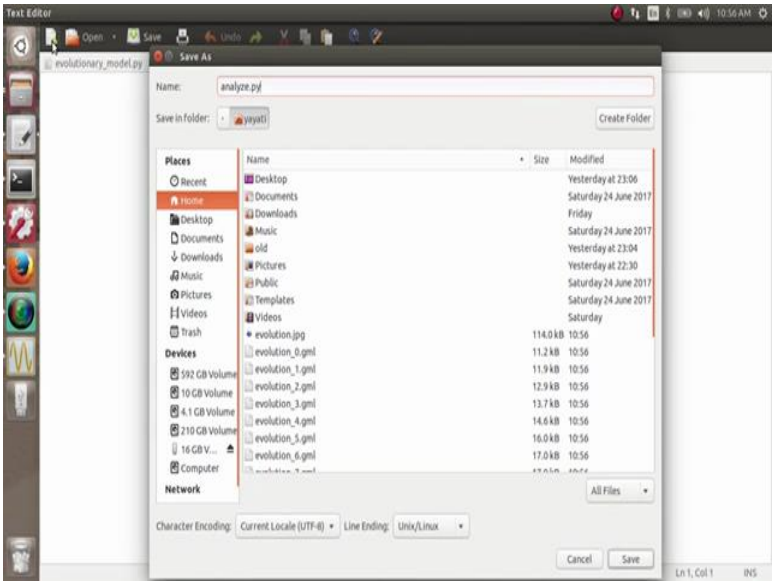
(Refer Slide Time: 04:24)



So, we have done it over some ten snapshots. So, you can do it over as many number of snapshots as you want. So, here we have 10 networks here I do not say ten networks it is the same network over 10 times stumps. So, it is my initial network there is some homophily some closure etcetera are happening on this network and then we have these different gml files coming up here.

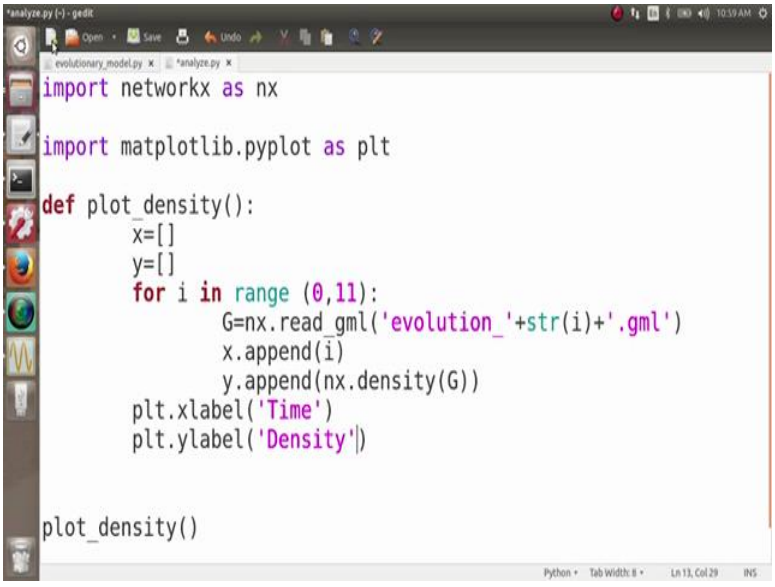
Now, I want to analyze these files why I want to analyze how this how the change in this graph is happening. So, to analyze this file we create a new file.

(Refer Slide Time: 05:10)



And let us name it analyze.py, and what I want to do in analyze.py see I want to analyze three things I want to see how the number of obese people in the network change then I want to see how the number of edges between these obese people change, and then I want to look at the change in density on this network. Let us keep it simple for the timing let just look at 2 things.

(Refer Slide Time: 05:44)

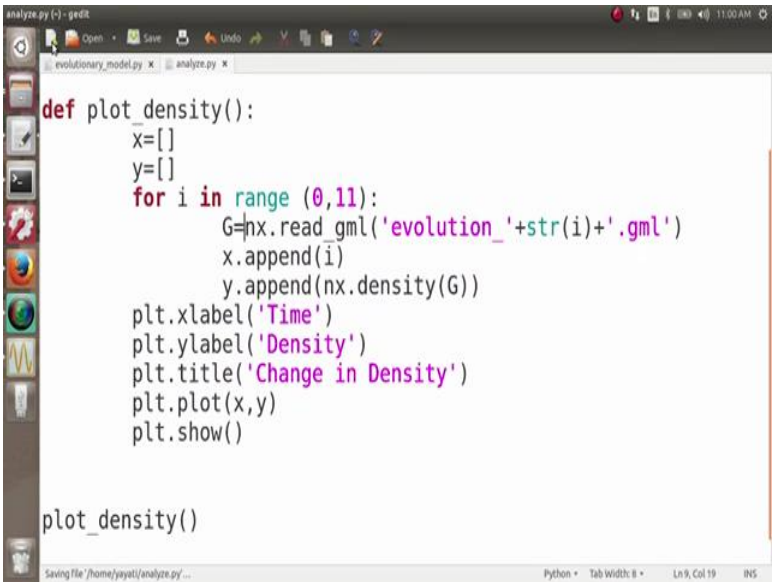


Let us look at how the number of obese people change and how the density of these networks changes. So, we need to import network x as nx and we also need to import matplotlib.py plot as plt.

And next what we want to do is let us say our function is let us have our function plot density. So, what plot density does? It plots how the density in a network is changing. So, we defined here plot underscore density and what this function is going to do is. So, we need have. So, you know that for plotting we need an x axis, we need a y axis; x axis here is the iterations and y axis here is the density of these different networks. So, our x axis will be the iterations and our y axis will be the density.

So, what we do is we know that for i in range we have 10 snapshots. So, we take from range 0 to 11 we need a graph here. So, what is our graph G = nx.read_gml and what is the file name? File name we know is evolution under score and whatever is the value of i plus and we have here gml. When we read this file and then we want to look at the density of this network. So, what was in the x axis? x.append its very simple. So, we append i here and what do we appending y is the density of this network. So, we have a direct function for density let us call that nx.density(G). In y we append the density of this graph. So, we have an x axis, we have a y axis and at the end we plot both of these or we can actually put the titles for the plot plt.xlabel and let us see it is the time and then we can have plt.ylabel.

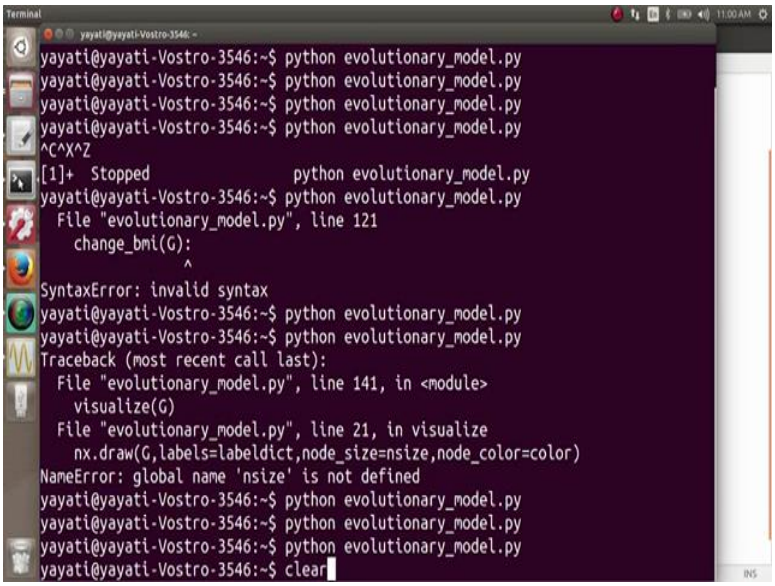
(Refer Slide Time: 08:13)



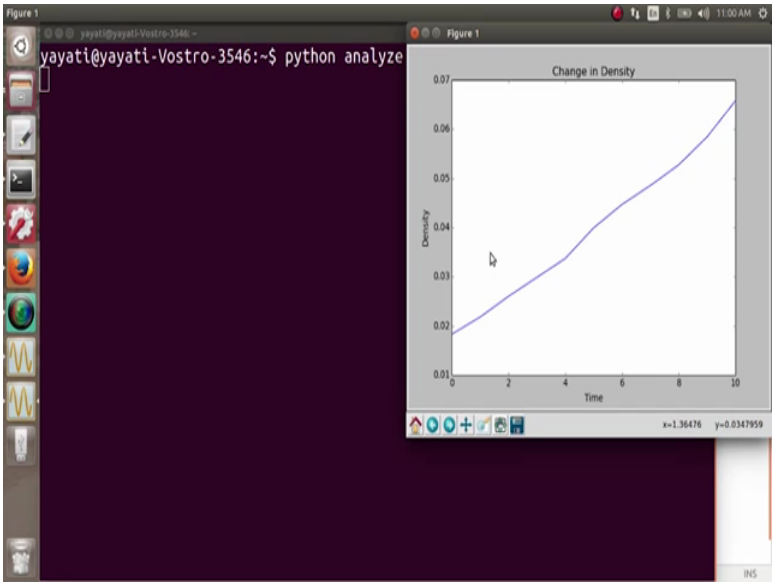
And let us have it as density, and then we have a title for this plot will it change in density; and then what we do is plt.plot(x, y) as we do, and then we do a plt.show.

So, this is simple right we have these 10 gml graphs we read each of these graph one by one look at their density and then we clued their density let us execute it for.

(Refer Slide Time: 08:47).



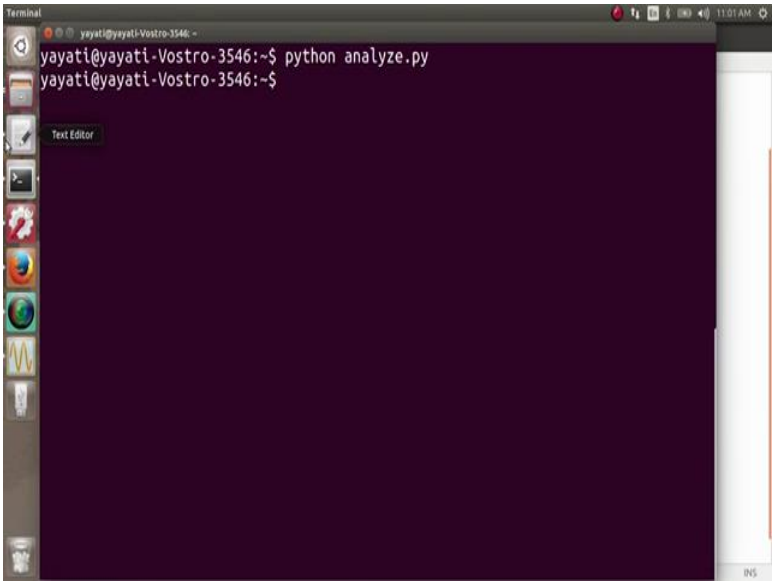
(Refer Slide Time: 08:50)



So, we have here python and we have analyze dot p y. So, you can see here how the density of this graph is changing. So, we trailed over ten iterations the density is increasing very fast, not we cannot say very fast densities increasing almost linearly this

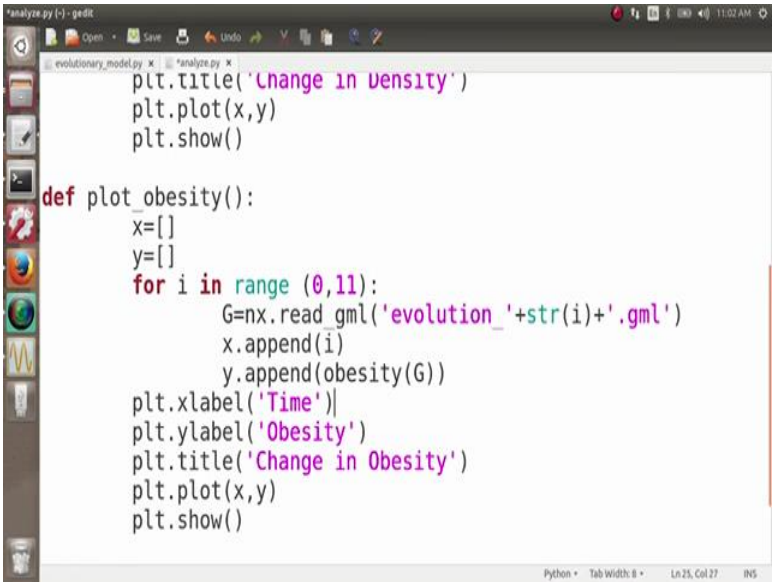
is about the density of this network more and more links are being formed because of homophily and because of all these three different kinds of closures.

(Refer Slide Time: 09:17)



So, you can actually see whatever you want to see across this evolution with the while loading this graphs and looking at them.

(Refer Slide Time: 09:35)

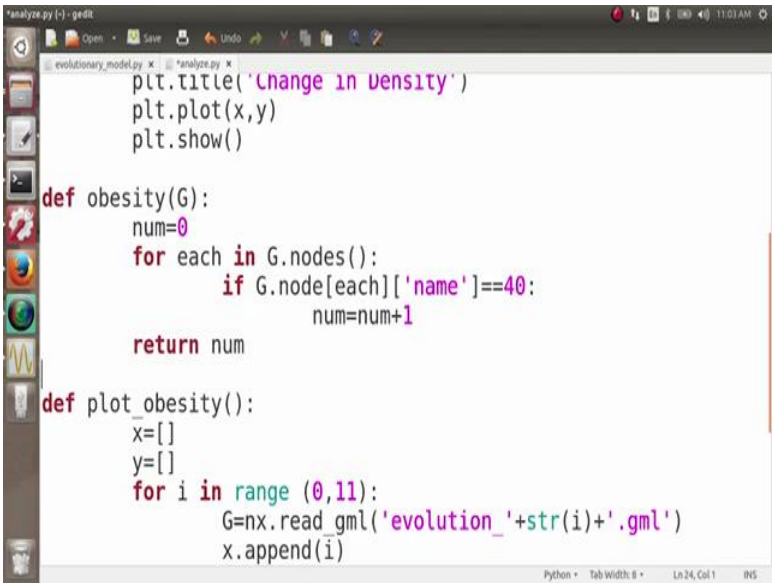


Let us say next I want to plot the obesity in this network and how do I plot the obesity by looking at the number of obese people in this network. So, we can write a similar code for this. So, we have defined plot underscore obesity and actually it is going to be almost

the same. So, let us just copy paste we use copy paste again and again in computer science. Define plot no (Refer Time: 10:10) computer science in coding define plot underscore obesity and we have this till here it is perfect.

Now, the problem is only here y dot append we have appended here the density of graph G, but what we want to append here is let us call our function obese and let us name this function as obesity. So, obesity is a function which will take as input your graph G and return you the number of obese people in this graph, and we just remove the density here by obesity, but yes we need to define this function obesity G.

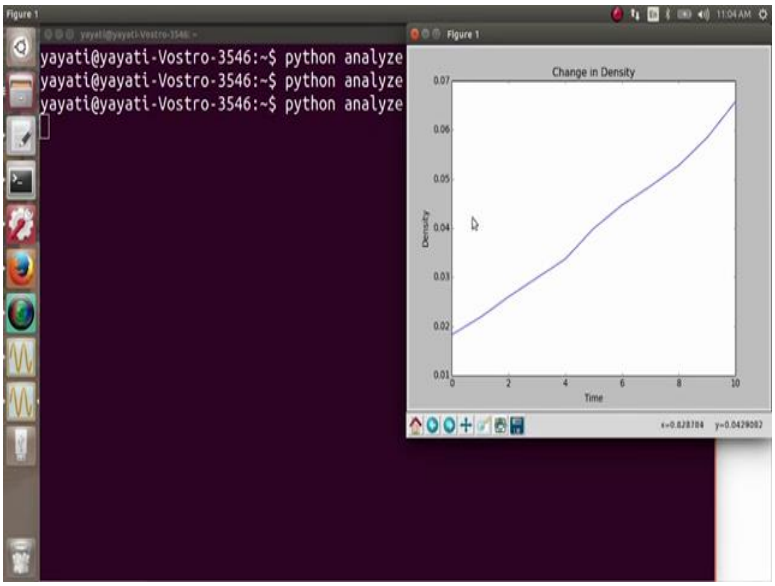
(Refer Slide Time: 10:51)



Lets quickly define this function obesity G and it is quite easy to find the number of obese people in this graph. So, let us say the number is 0 then for each in G.nodes we see whether the node is obese or not and if the node is obese, we increase num by 1.

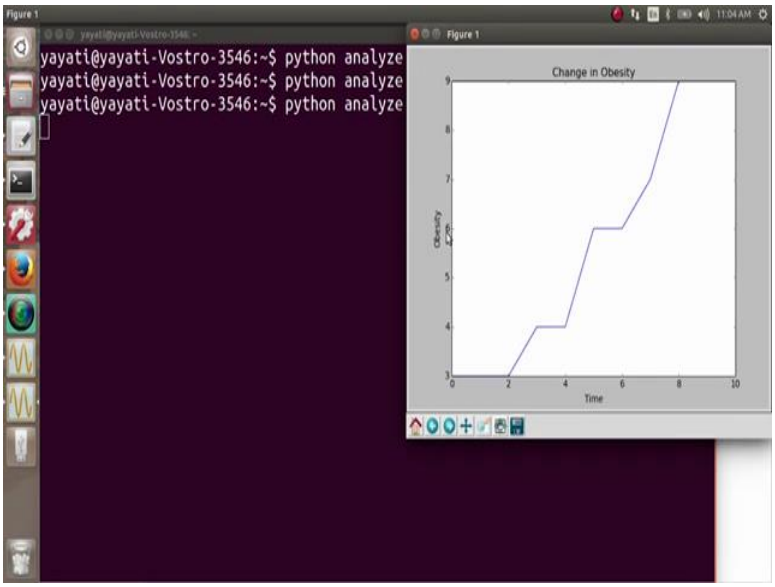
So, for each in G.nodes if G.node[each] and we look at its name right if its name equals to equals to 40 that is if its BMI is 40, we increase num by one and then we return num. So, this is how it goes.

(Refer Slide Time: 11:36)



Let us plot it and let us see it now. So, let us plot it and see. So, here we have python analyze.py. So, this first plot here is for the change in density, which we can see is increasing with time.

(Refer Slide Time: 11:51)



And now let us look at obesity, how the number of obese people in this network increase is. So, you can see here. So, it is an interesting curve you can see that it increases that then it remains constant for some time it then increases, then again remains constant for some time and so on.

We can actually see a lot of the change in a lot of interesting parameters in this graph you can actually see how the number of membership closures is changing in this graph, you can look at the number of triads in a lot of other things. So, that is a little bit of more complicated coding, but; obviously, fun to do if you are interested in any of that coding you can try it yourself and leave your questions on discussion forum will be always happy to help you.

So, I will leave you here with these very 2 simple observations. So, over this complete evolution and you can try your own coding and let us see if there is any problem. I hope you enjoyed this screen cast.